

アナログ開発ツールの使い方 (OpenSUSI-TR10)

ISHI会

森 瑞紀 (Mizuki Mori) , <https://github.com/3zki>

今村 謙之 (Noritsuna Imamura)

Rev.1

アジェンダ

- なぜ、半導体を学ぶのか？
- 半導体の基礎知識
- 開発ツールの紹介、PDKの中身
- アナログ半導体の設計フロー
- アナログ半導体設計デモンストレーション（CMOSインバータ）
- フレーム・パッドとテープアウト（製造依頼）

なぜ、半導体を学ぶのか？

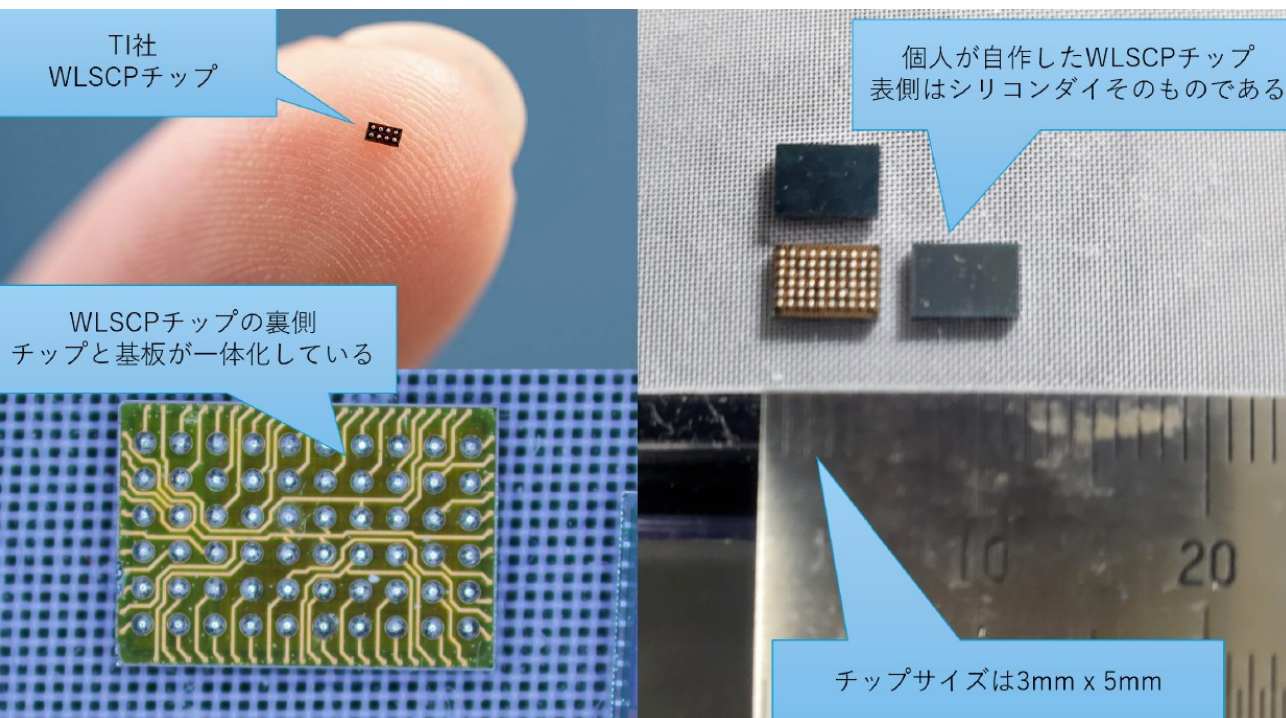
次世代電子工作の世界へ

現在は、専用チップ化が進む世界

.....
超小型パッケージがもたらす大きな可能性～TI 社の世界最小マイコン MSPM0C1104 のプレスリリースより～

消費者は、電動歯ブラシやスタイラス ペンなど日常的に使用する電子機器に対し、より小型で低コストかつ多機能な製品を常に求めています。こうした製品の小型化が進む中で、エンジニアは限られた基板面積で新たな機能を追加できる、コンパクトな統合された部品を求めています。MSPM0C1104 マイコンは、搭載する機能を厳選し、TI のコスト最適化戦略を取り入れるとともに、WCSP のパッケージング技術の利点を活かしています。8 つの接点を持つ WCSP のサイズはわずか 1.38mm² で、競合製品と比べて 38% の小型化を実現しています。

.....



チップは「小型」と「低コスト」
がキモ！

- 最終到達点は「超小型の専用チップ化」

PCB基板はチップと一体化！

- 最終到達点は、チップと基板は一体化し、センサーやアクチュエーター、筐体と統合される

次世代電子工作は
「基板設計」から「専用チップ設計」になる！

半導体を自作する世界がやってきた



ICHIKEN

イチケン / ICHIKEN ✓

@ICHIKEN1 ・ チャンネル登録者数 50.9万人 ・ 307 本の動画

ものづくり系の動画をアップロードしています。...さらに表示

ichiken-engineering.com/ichiken-youtube-ad、他 4 件のリンク

登録済み ▼

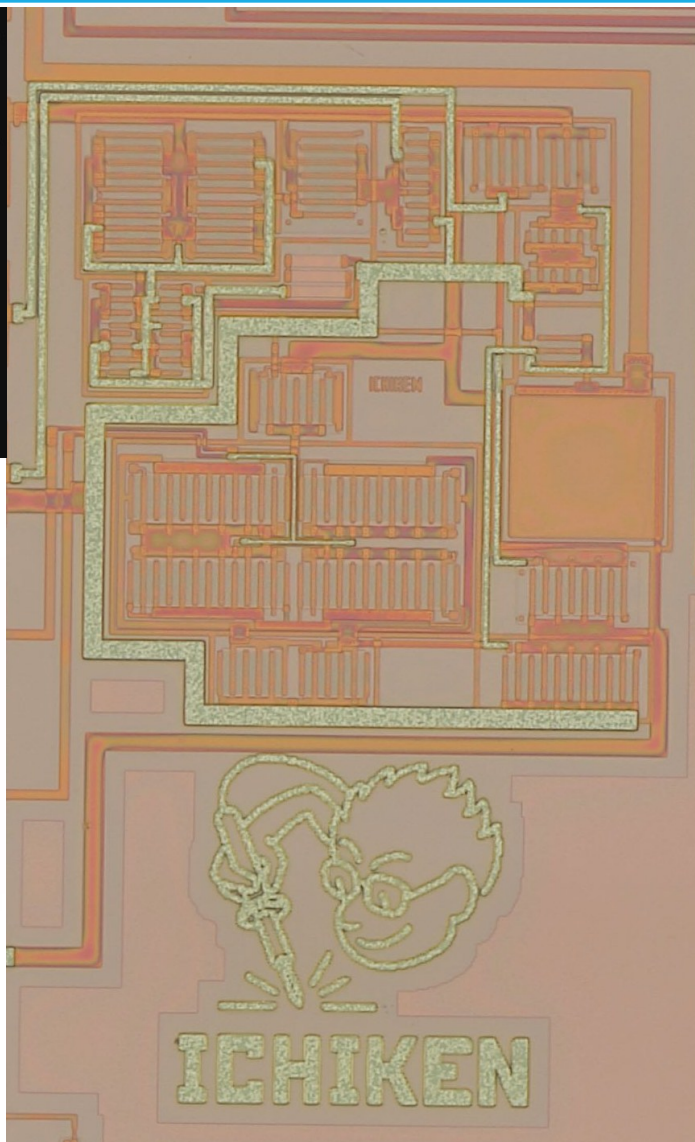
メンバーになる

コミュニティ

半導体を自作する

米粒サイズ

本当に動いた!



こんにちは、イチケンです。

普段は YouTube で電子工作やものづくりの動画を投稿しているのですが、今回ご縁があって、この「Open Source Silicon Magazine Vol.2 と Vol.3」の冒頭の挨拶を書かせてもらうことになりました。

2025 年に、オープンソースの設計ツールを使ってオペアンプをゼロから設計し、実際にシリコンチップとして製造するということをやりました。ISHI 会の皆さんにサポートしてもらいながら、回路図を書いて、シミュレーションして、レイアウトを作って、検証して、ファウンドリに出して——そして数ヶ月後、実際にチップが届きました。届いたチップをマイクロスコープで覗いたとき、自分が設計したレイアウトがそのままシリコンの上に再現されているのを見て、素直に感動しました。そしてブレッドボード上でオペアンプとして動作したときは、「ああ、本当に動くのだ」と。正直に言えば、出力の駆動力が足りなかったり、高い周波数で波形が歪んだり、改善点はたくさんあります。しかし、自分で設計したトランジスタが、自分で描いたレイアウト通りに製造されて、ちゃんとオペアンプとして増幅している。この体験は、既製品の IC を使った電子工作とはまったく別の面白さがありました。

さて、この本を手にとってくれた皆さんの中には、「半導体の設計って自分には関係ないだろう」と思っている方もいるかもしれません。僕も最初はそう思っていました。半導体は、巨大な企業が何百億円もかけて作るものというイメージがありますよね。しかし、今は違います。オープンソースの設計ツールがあります。無償で使える PDK (プロセスデザインキット) があります。TinyTapeout のようなシャトルサービスを使えば、1 タイルあたり数百ドルで自分の設計をシリコン化できます。国内のファウンドリでも、シャトル便を使えば数万円の負担でチップが作れる時代です。設計環境のセットアップも、有志の方々がまとめてくれていて、かなり楽になっています。つまり、電子工作で Arduino やマイコンを使ってモノを作るように、半導体チップそのものを「設計して作る」ことが、個人の手の届くところまで来ているのです。

本書 Vol.2 と次作の Vol.3 では、普段の電子工作で何気なく使っている IC チップの中身が、どのように設計されているのかを解説しています。論理回路と MOSFET の関係から始まって、スタンダードセルと配置配線、SystemVerilog での CPU 設計、TinyTapeout での IC チップ設計体験記。アナログ側では、OPAMP の増幅回路、バンドギャップリファレンス、ADC、DAC、VCO といった回路の設計原理からレイアウトまで。「何を作ったか」だけでなく「どのように作ったか」——設計の手法、苦労したポイント、工夫したポイントを、実際に手を動かした人たちが書いています。Vol.1 でツールのセットアップや基礎を学んだ方は、この Vol.2 と Vol.3 で実際に「何が作れるのか」を体感できると思います。

実際にオペアンプを設計してみて特に実感したのは、レイアウト設計の奥深さです。回路図上では線をつなぐだけですが、レイアウトではコモンセントロイドで温度特性を揃えたり、ダミーポリシリコンで製造バラツキから回路を守ったり、ダブルビアで信頼性を確保したり。回路図には載っていないノウハウが山ほどあります。そして、これがパズルのようで面白いのです。いくら時間があっても足りないくらいです。

電子工作の世界では、かつて Arduino が登場したことで「マイコンは一部の専門家だけのもの」から「誰でも使える道具」に変わりました。その結果、Maker Faire のような場で、想像もしなかったような多様な作品が生まれるようになりました。

半導体の設計も、今まさに同じ変化の入り口に立っていると思っています。「こんなセンサーがあったらいいのに」「この機能を 1 チップに収めたい」「既製品にはない特性のアナログ回路が欲しい」——そういったアイデアを、自分の手でシリコンにできる時代が来ています。今はまだ、L チカならぬ「最初の 1 チップを設計して動かす」段階かもしれませんが、しかし、電子工作も最初は L チカから始まって、そこから無数のプロジェクトが生まれていきました。半導体設計も同じ道をたどるはずですよ。

この本がその最初の一步になれば嬉しいです。いつか皆さんの設計したチップの話を聞ける日を楽しみにしています。

イチケン

次世代電子工作の世界へ

- マイコンは、大雑把に分けると・・・
 - 周辺機器部（ペリフェラル部 + IO 部）
 - クロック部（内蔵発振器 or 外部発振器）
 - コンピュータ部（プロセッサ部 + メモリ部 + バス部）

今日はこれら専用チップ設計の基礎を身につける講座



マイコンを実際の半導体として
自作するための知識を詰め込んだ
一冊！
半導体という新しい電子工作の
世界へ踏み出そう！

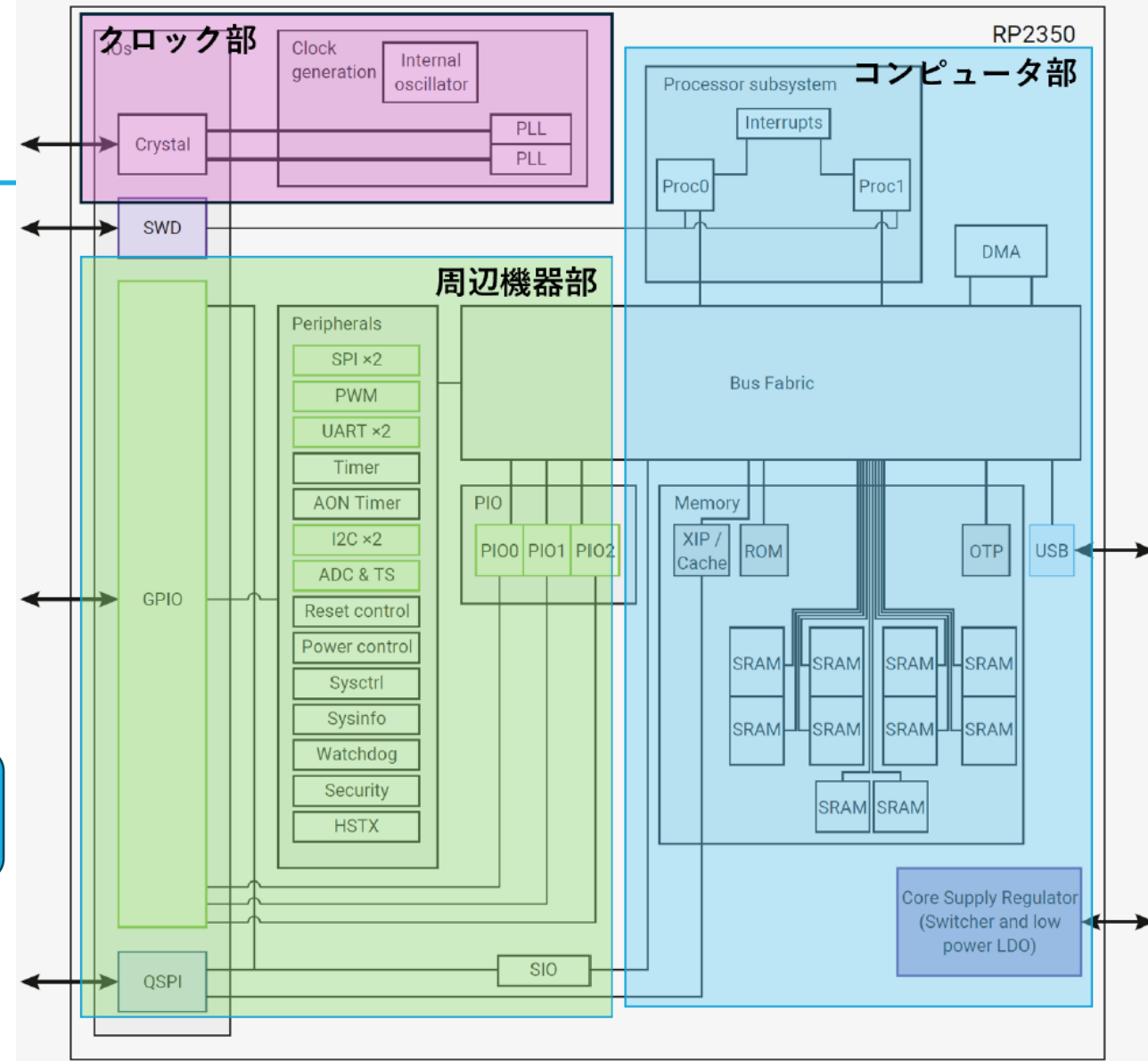
半導体設計の基礎知識を得るための
OpenSourceSiliconMagazineVol.1は
無料配布中！

イチケンさんが
挑戦した
OPAMPも解説！

オープンソース半
導体コミュニティ
ISHI会は、Discord
やリアルイベント
にて活動中！

電子版：2000円
紙版：2500円
お得なセット版は
2500円です !!

詳細を知りたい方は
こちらをどうぞ



ラズベリーPico マイコンのブロック図

半導体の基礎知識

プレーナ型FETプロセス

ICとは？

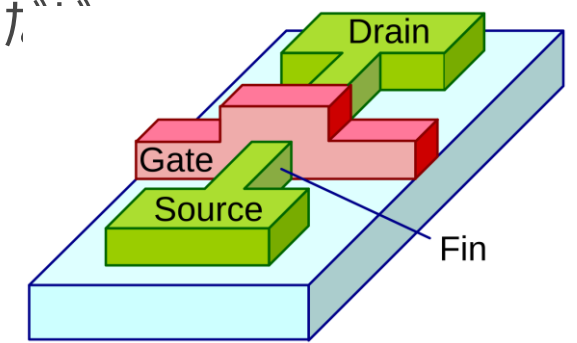
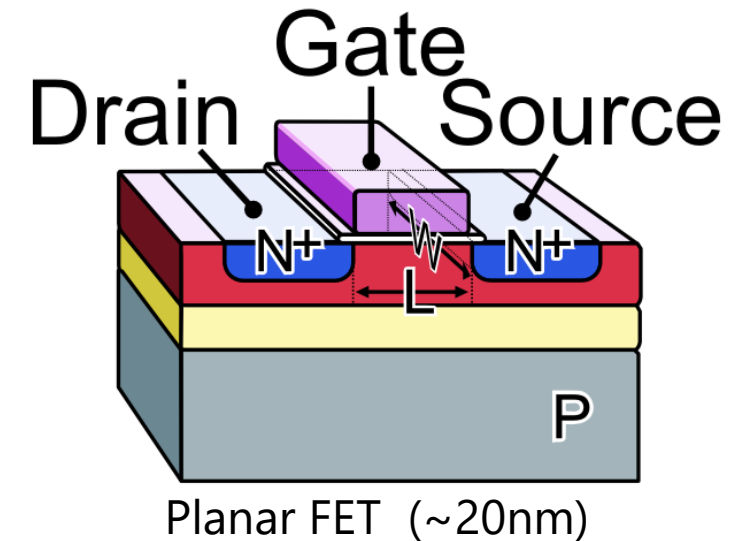
- Integrated Circuit, 集積回路 には様々な形態がある。
 - 我々が目にするICは“**モノリシックIC**” 1枚のシリコン基板に回路が構成。
- 複数の部品を集積させたICを“**ハイブリッドIC**”という
 - 例: 水晶発振器 = 水晶発振子 + 発振回路
- モノリシックICの集積密度を高めたものが **大規模集積回路**、**LSI** (Large Scale Integration)



Crystal Oscillator (Photo: my own work)

プレーナ型FETプロセスとは

- 一言に半導体と言っても様々な種類があり、設計手法もそれぞれ異なる
- 東海理化(TR-1um)は**プレーナ型FETプロセス** 
P型MOSFETとN型MOSFETが作成可能
- プレーナ型FETは20nm程度が限界。
CPUのような高密度集積回路ではFinFETやGAA-FETといった全く別の構造をしたプロセスを現在は使用している。
- プレーナ型 45~130nmはロジック回路用としては時代遅れ (2000年前後) が
FinFETよりも安く製造でき、アナログLSIとしては現役。



https://commons.wikimedia.org/wiki/File:Multigate_models.png

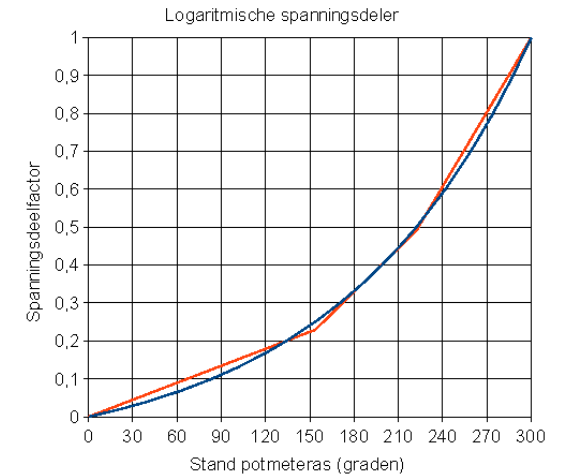
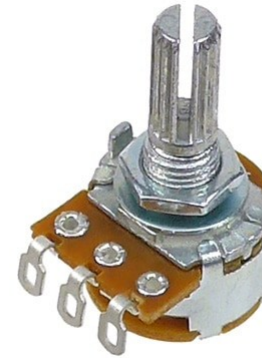
[https://commons.wikimedia.org/wiki/File:MOS-FET_gate_model_\(n-channel\)_E.PNG](https://commons.wikimedia.org/wiki/File:MOS-FET_gate_model_(n-channel)_E.PNG)

プレーナ型FETプロセスで作れる素子

- プレーナMOSFET: PMOS, NMOS
 - 耐圧により種類があることがある : 1.8V, 3.3V 5.0V, 10.5V, 16V, 20V など
- ダイオード
- 抵抗: ポリ抵抗, 拡散層抵抗 (アクティブ抵抗) , Nwell抵抗
- キャパシタ: ポリ容量, 拡散層容量, MIMCAP (プロセスによっては無い), 配線層の寄生容量
- バラクタダイオード: プロセスによっては無い
- ラテラルバイポーラ (PNP, NPN) : プロセスによっては無い

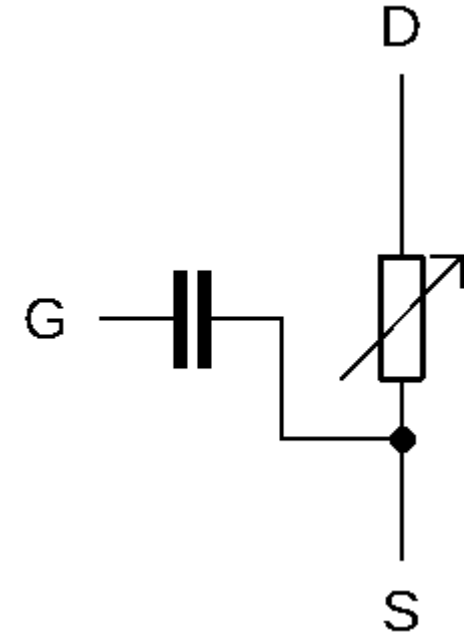
PMOS, NMOSとは

- 可変抵抗（ボリューム）に似ている
 - 0Ω から摺動子(ツマミ)を回すと、抵抗値が増加する
 - スペックでは最大抵抗値とカーブが記載
 - 抵抗値の増え方にバリエーションがある（カーブ）

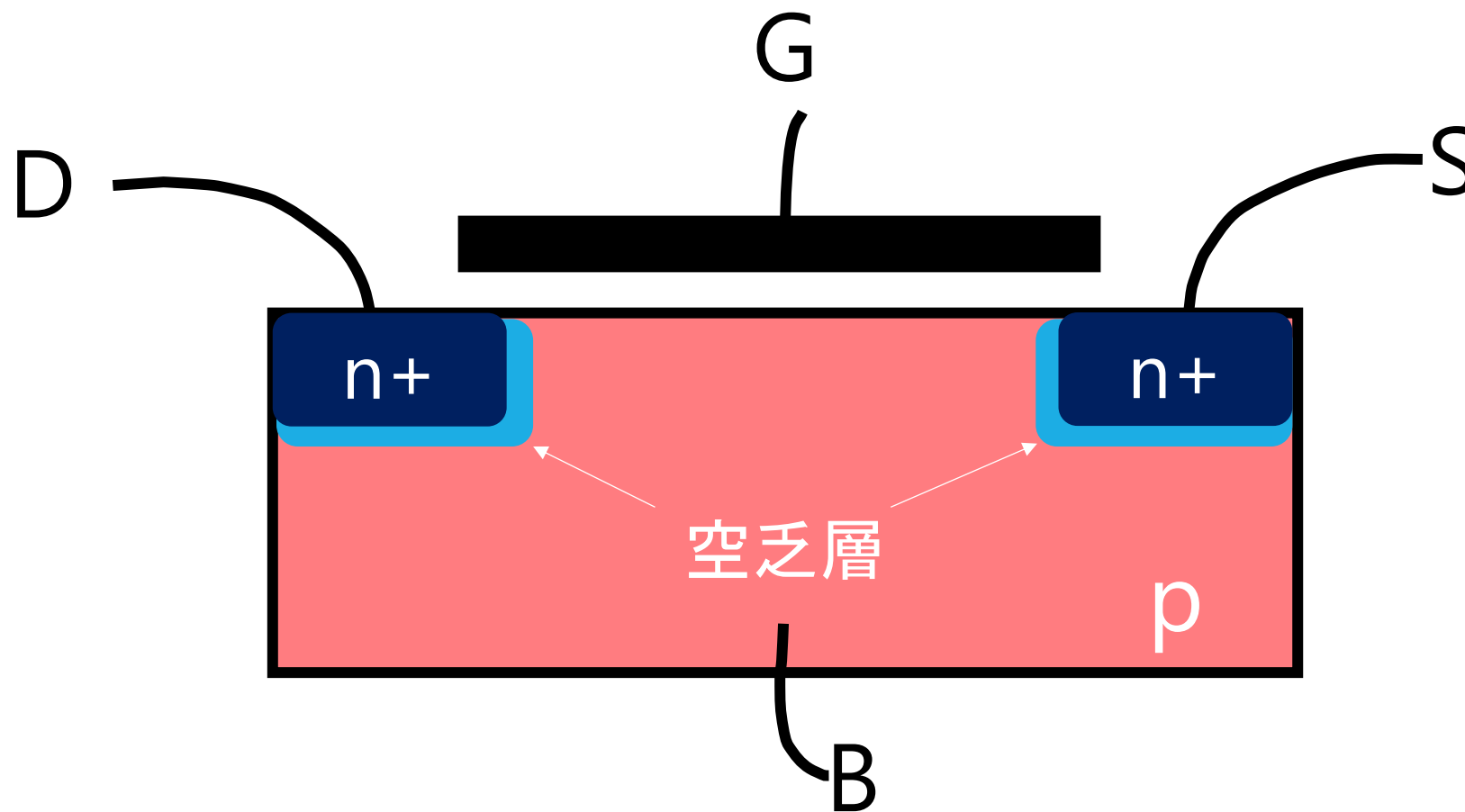


PMOS, NMOSとは

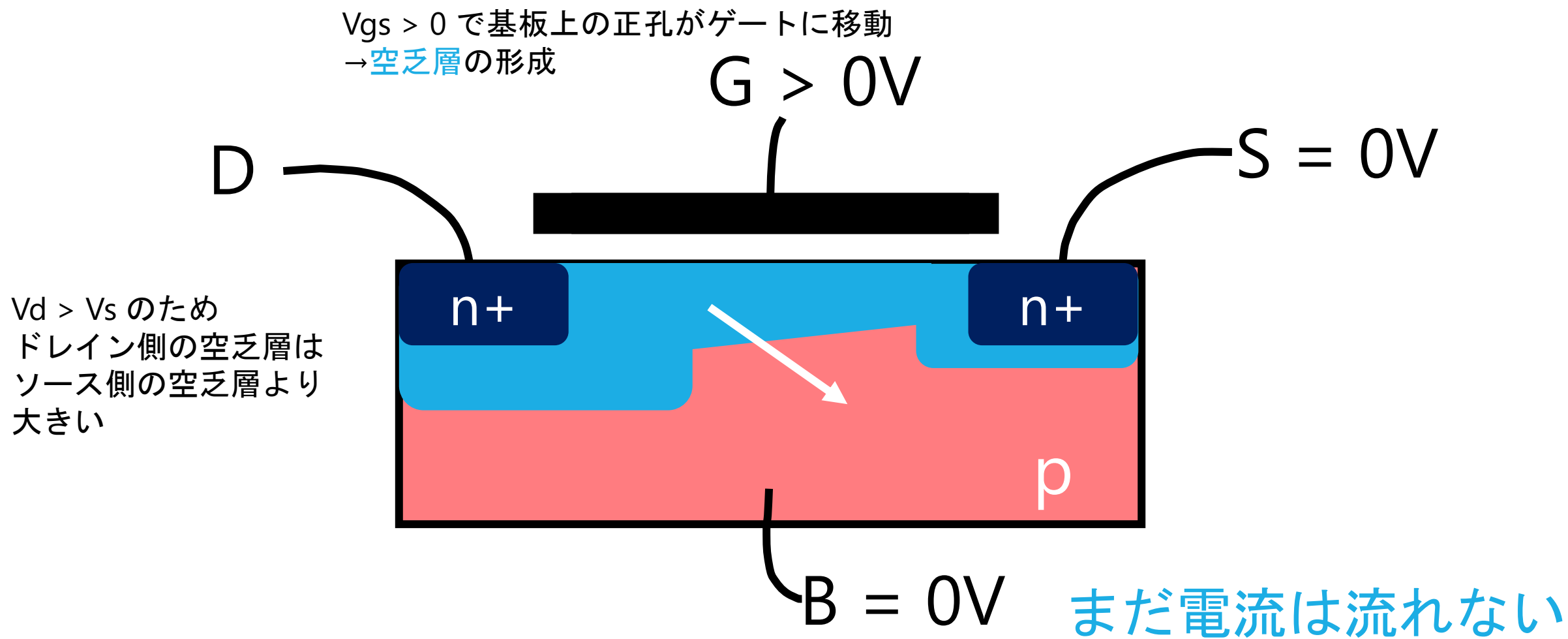
- MOSFETは電流をベースにして考える（抵抗値に置き換えて考える事は基本的でない）
- MOSFETは可変抵抗の逆
 - 摺動子（ツマミ）ではなくゲート電圧 V_{GS} によってド레인ソース間の電流を制御
 - $\infty\Omega$ からゲート電圧を調整すると抵抗値減少
 - スペックでは電流カーブが記載(ド레인ソース間電流 I_{DS})
 - 最小抵抗値は電流カーブから計算（オン抵抗 R_{on} ）
 - 抵抗値の増え方はプロセス依存（閾値電圧 V_{th} 、トランスコンダクタンス g_m ）



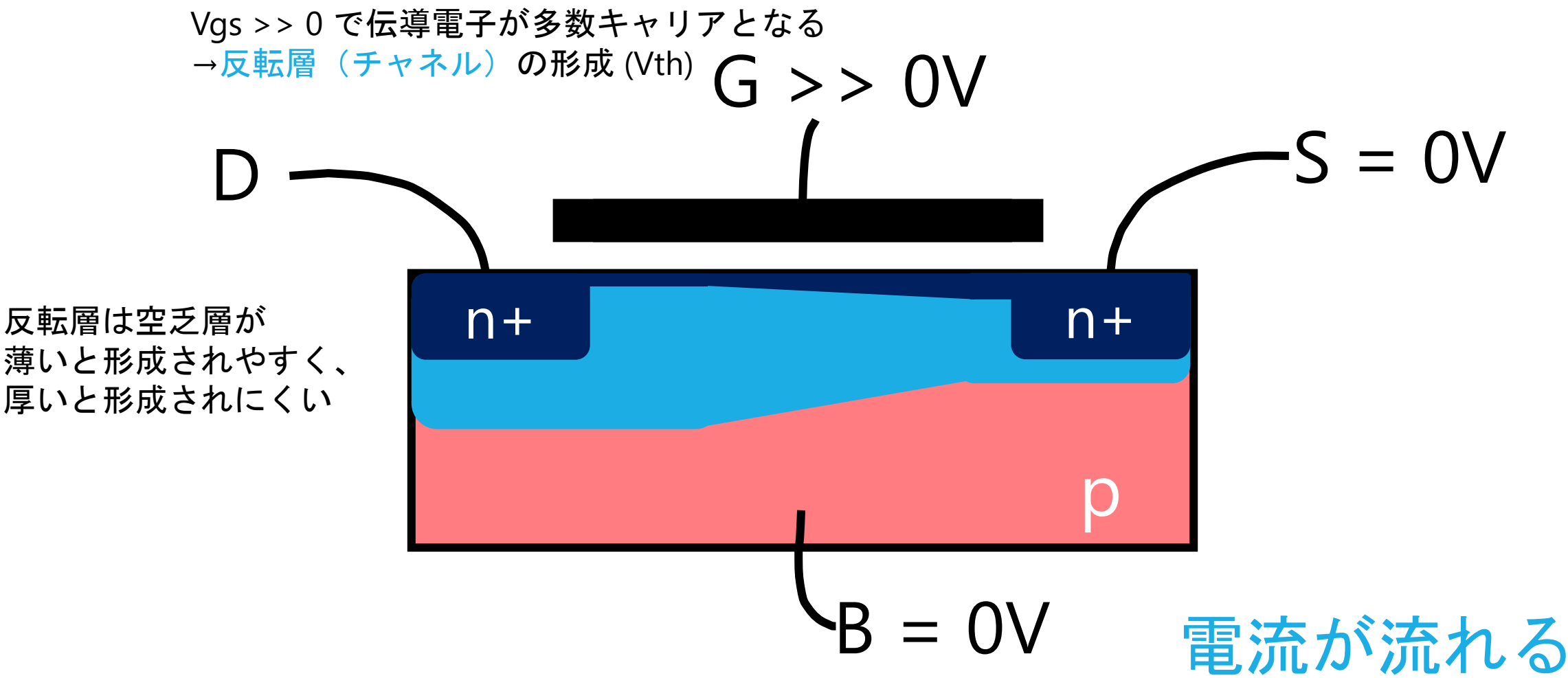
NMOS 簡単な模式図



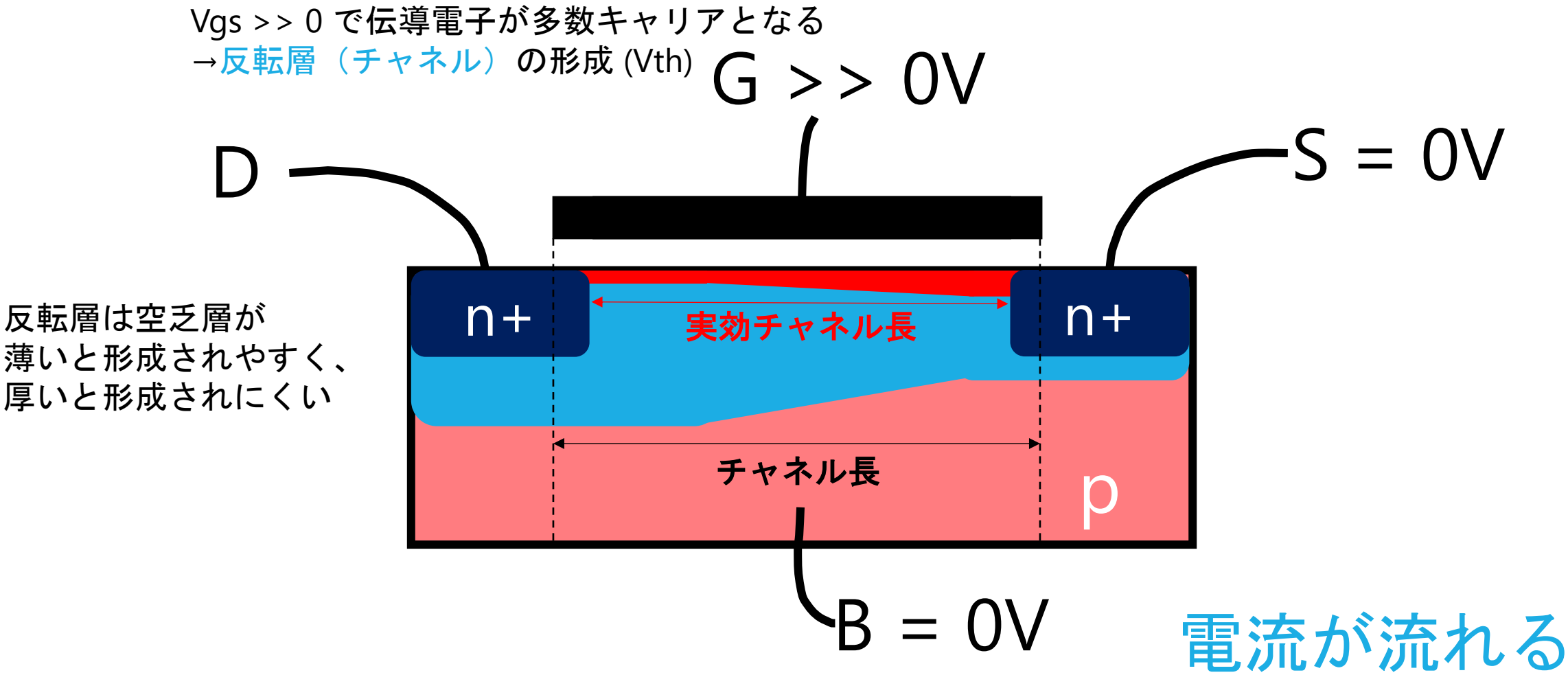
NMOS 簡単な模式図



NMOS 簡単な模式図

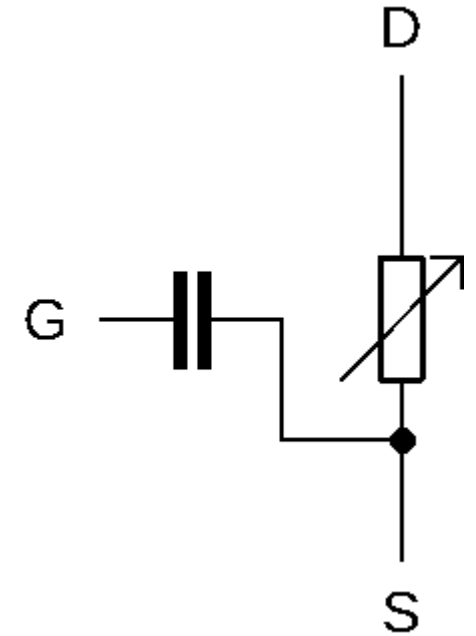


NMOS 簡単な模式図

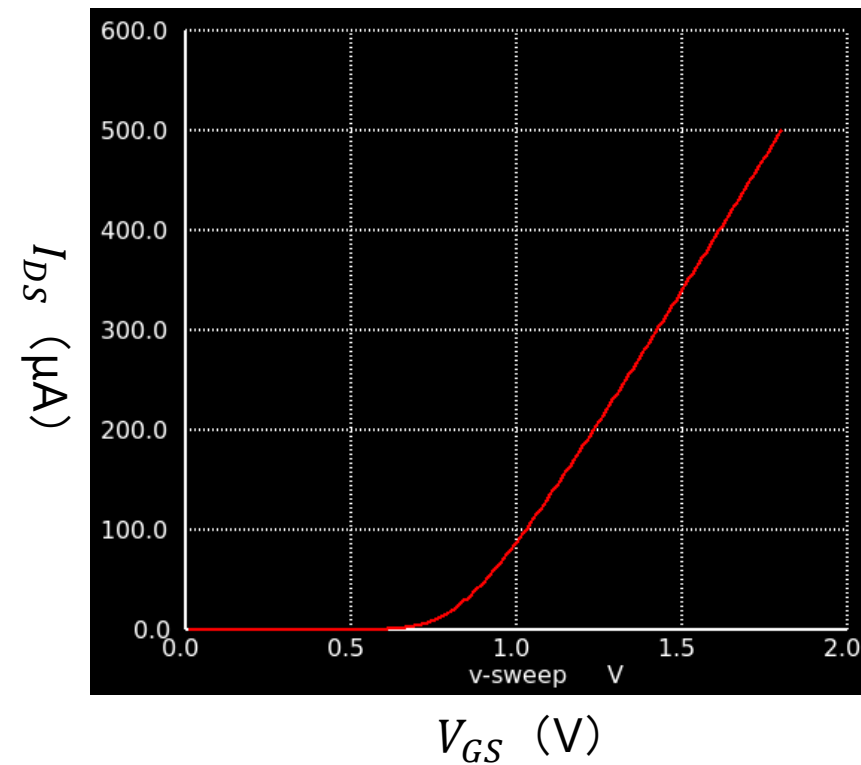
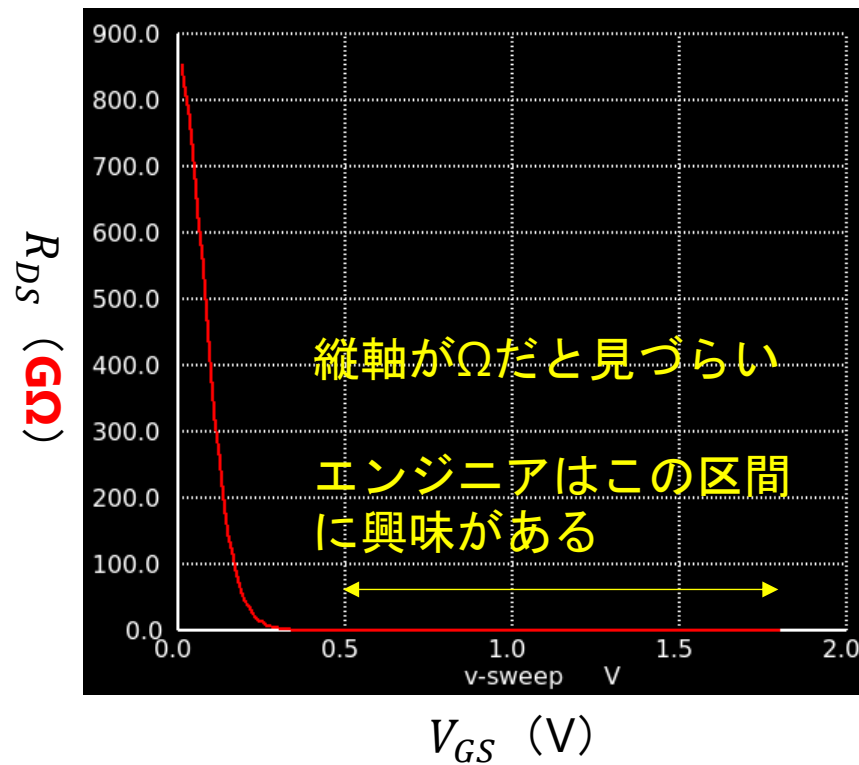
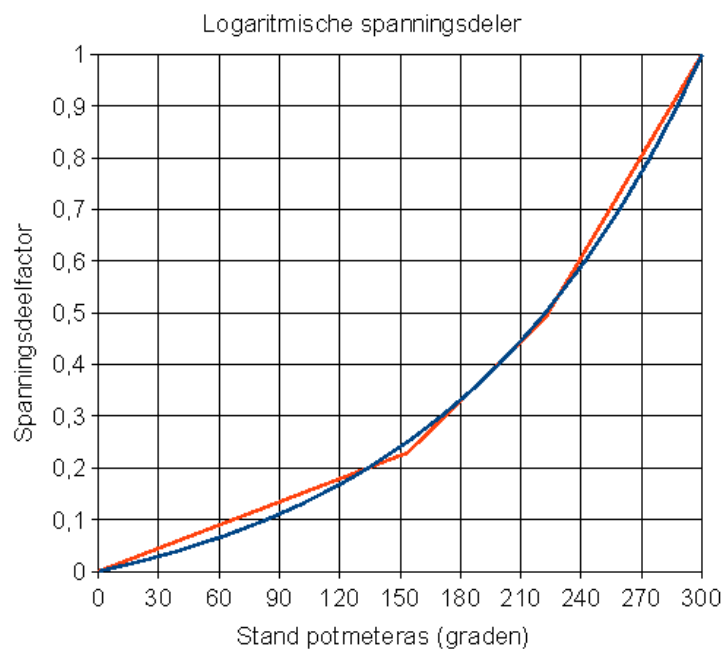


PMOS, NMOSとは

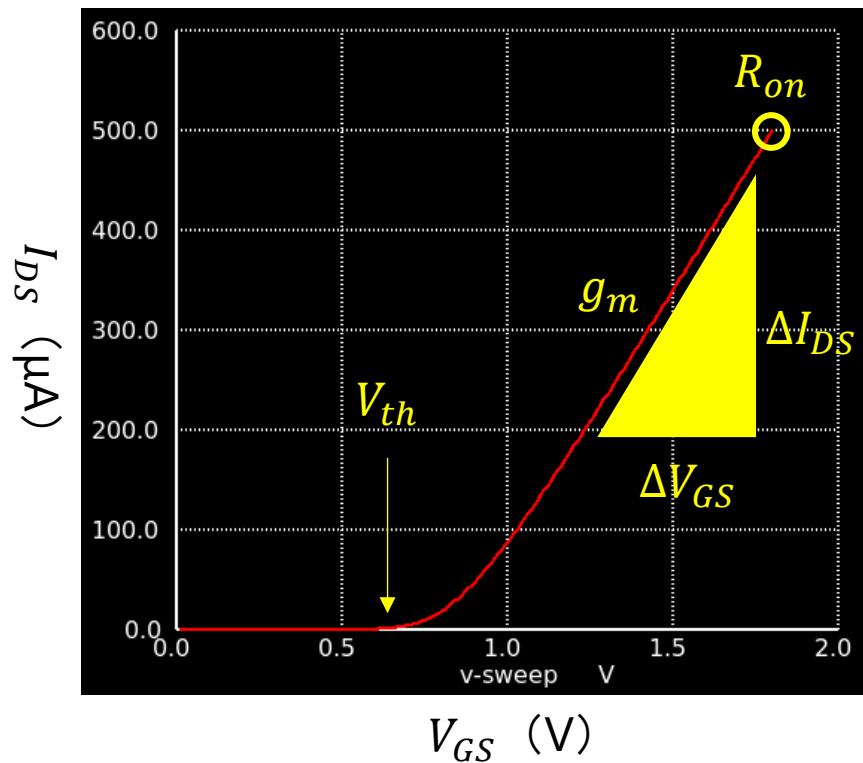
- 右図の等価回路は「MOSFETを取ってRLCで表すならこうだろう」というもの
 - ドレイン・ソース間は「電流源」を置くのが正しいと思われるが、初心者にはあまり馴染みがないので可変抵抗とした
 - MOSFETの重要な特性は **トランスコンダクタンス**とよばれるもの
 - また特定のドレインソース間電圧領域において、電流源として動作する **飽和(saturation)**とよばれる特性も非常に重要。
 - ゲート容量は**ゲート・ドレイン間**と**ゲート・ボディ間**にも存在するが、単純に見映えの問題でゲートソース間のみの記載とした



可変抵抗 vs NMOS



NMOSの特性



- V_{th} 閾値電圧
電流が流れ始めるゲートソース間電圧の閾値
- g_m トランスコンダクタンス
ゲート電圧の変化分 対 ドレインソース間電流の変化分
- R_{on} オン抵抗
オン状態での抵抗値

設計する前に…

- CMOSの特性を知りたい！
 - $V_{gs} - I_{ds}$ カーブ
 - 動作点解析 (Operational Point)

これらの詳細は、オペアンプ編にて詳しく解説します。

アナログ半導体の設計フロー

どのような設計をするか、どのような開発ツールを使用するか

アナログ半導体の設計フロー

1. 回路図（テストベンチ）を描く
2. シミュレーションをする
3. 回路図を基にレイアウトを描く
4. レイアウトを検証する (DRC, LVS)
5. ~~レイアウトを基に寄生成分を考慮したシミュレーションをする~~ TR-1um では未対応
6. フレームに載せる

そもそもアナログ半導体回路設計に必要なものとは？

• プロセスデザインキット PDK

- シミュレーションモデルライブラリ (SPICE)
- 検証ツール用ルールファイル (DRC, LVSなど)
- スタANDARDセルライブラリ
 - レイアウト (GDS)
 - ネットリスト(SPICE)
 - 自動配置配線ルール(LEF)
 - Verilogシミュレーションライブラリ (LIB, V)
- 一部指定されたレイアウト (フレームなど)

• PDKで指定された各種ツール (EDAツール)

- 回路図エディタ or Verilogコンパイラ
 - 回路図エディタ : xschem, Glade, KiCAD
- レイアウトエディタ or 自動レイアウトツール
 - レイアウトエディタ : klayout, Glade, (KiCAD)
- SPICEシミュレータ or Verilogシミュレータ
 - SPICEシミュレータ : ngspice, LTspice
- 検証ツール
 - 検証ツール : klayout

PDKの場所

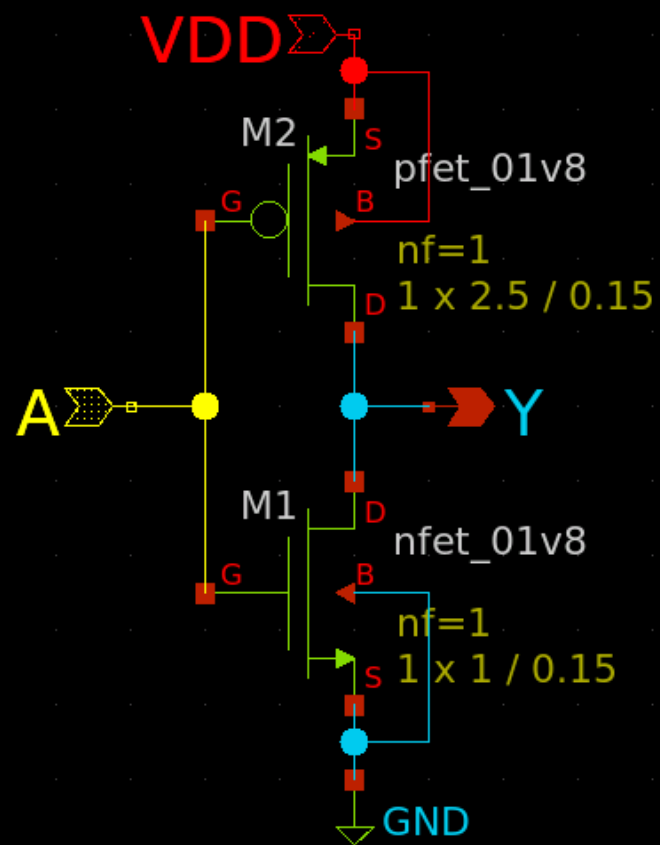
公式レポジトリ

- TR-1um PDK
<https://github.com/OpenSUSI/TR-1um>

ISHI-KAIオリジナル

- EDAツールセットアップスクリプト
https://github.com/ishi-kai/OpenEDA-PDK_SetupScript/
- Mac用のVmwareイメージやWSLイメージなどもあります。

回路図(ベンチマーク)を描いて...



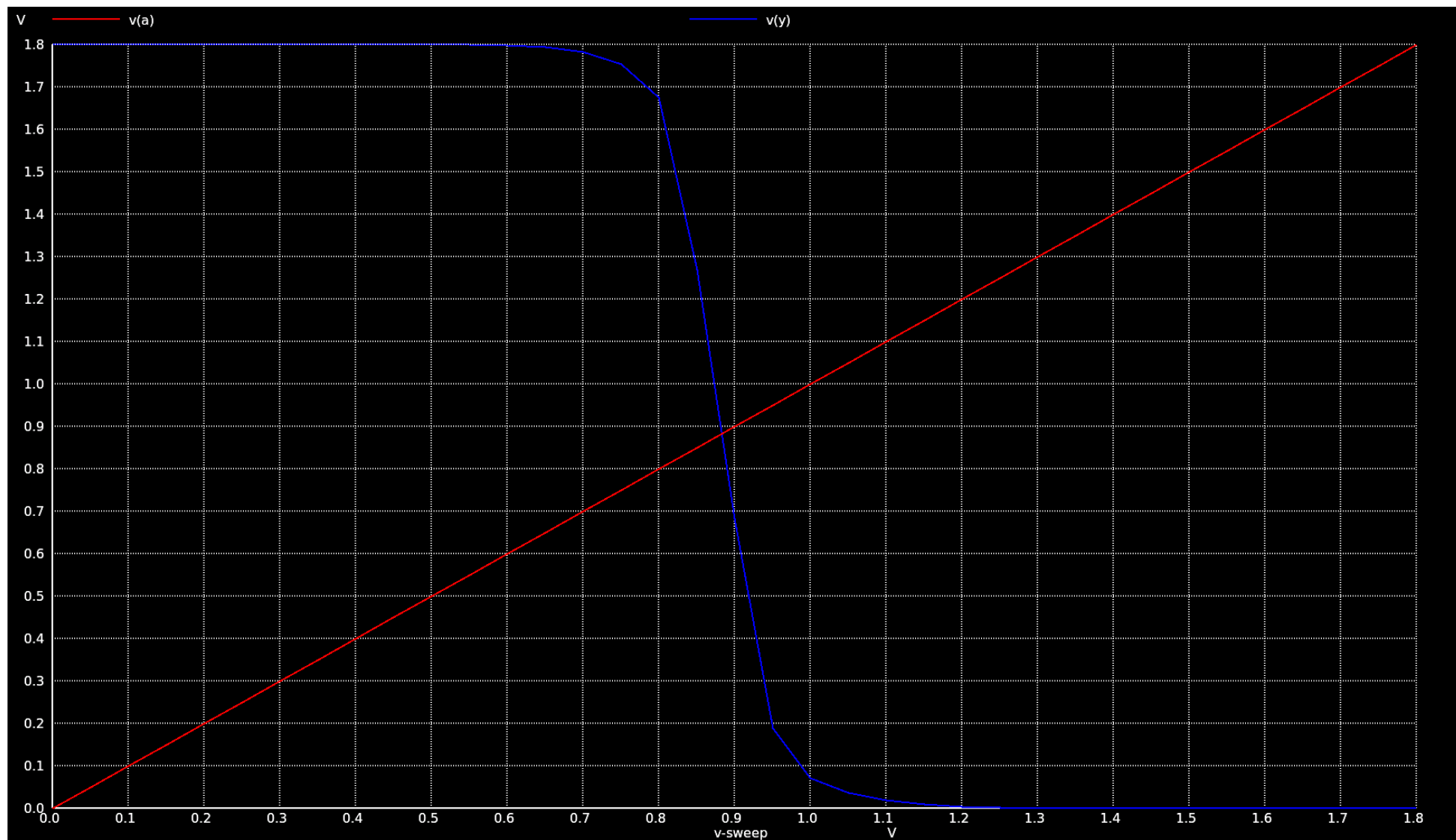
COMMANDS
SIM=ngspice

```

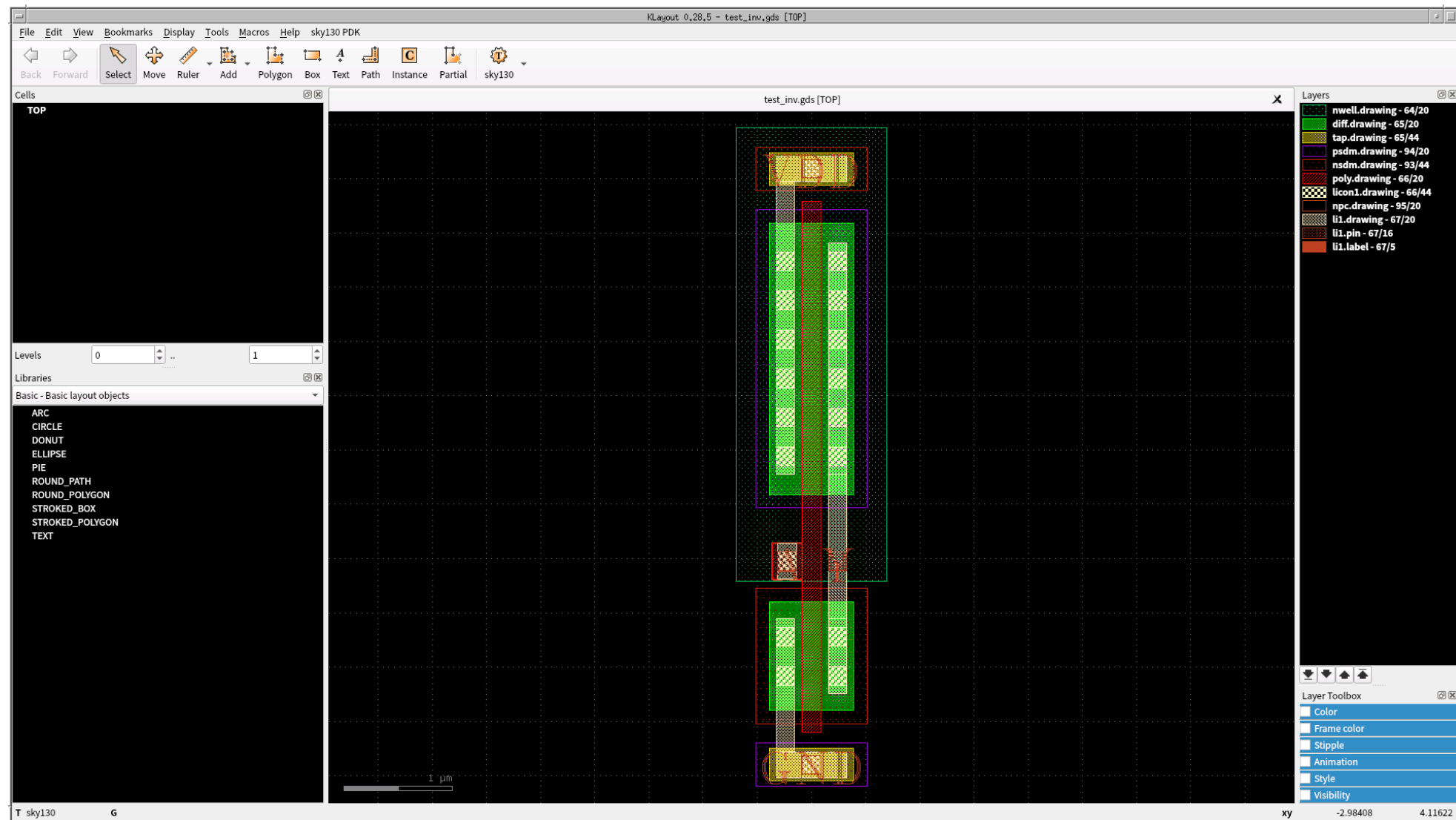
VD VDD 0 dc 1.8
VA A 0 dc 0
.control
save all
dc VA 0 1.8 0.05
plot v(a) v(y)
.endc

```

論理検証をして...

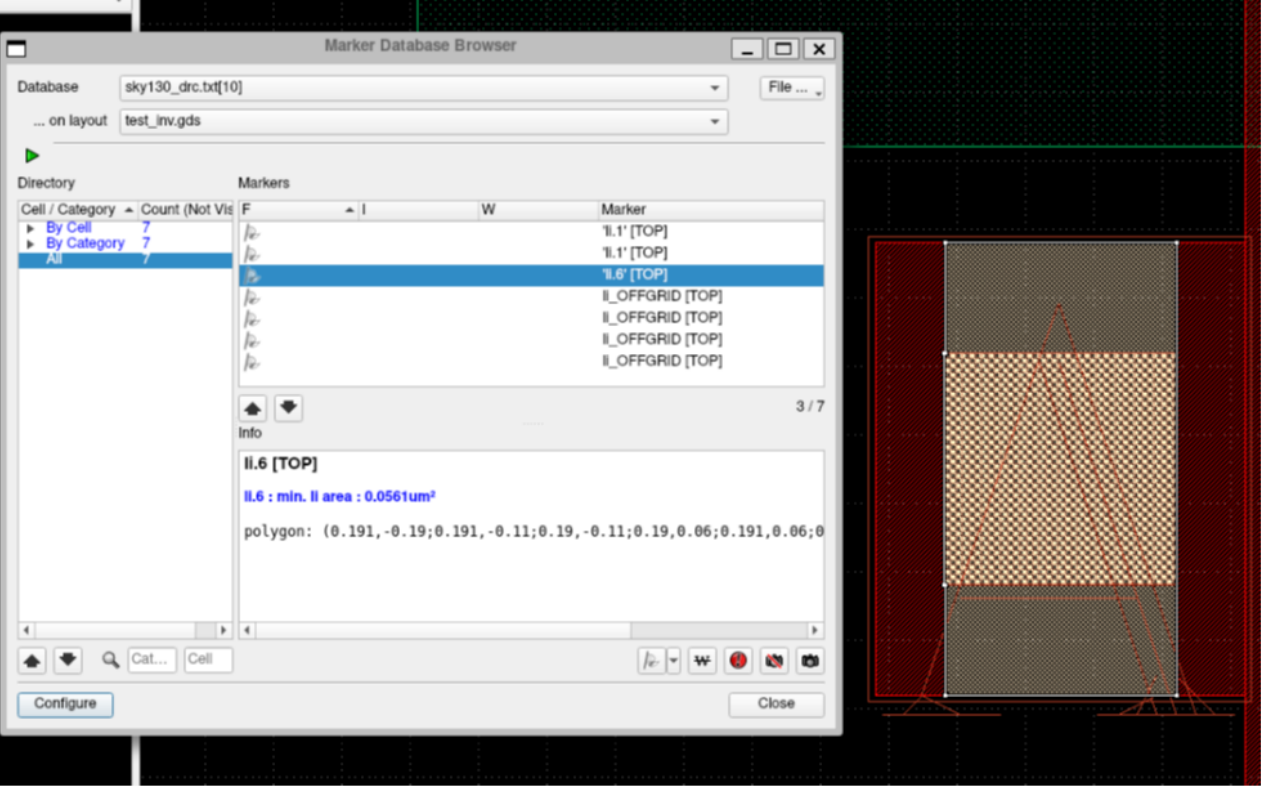


レイアウトを描いて...

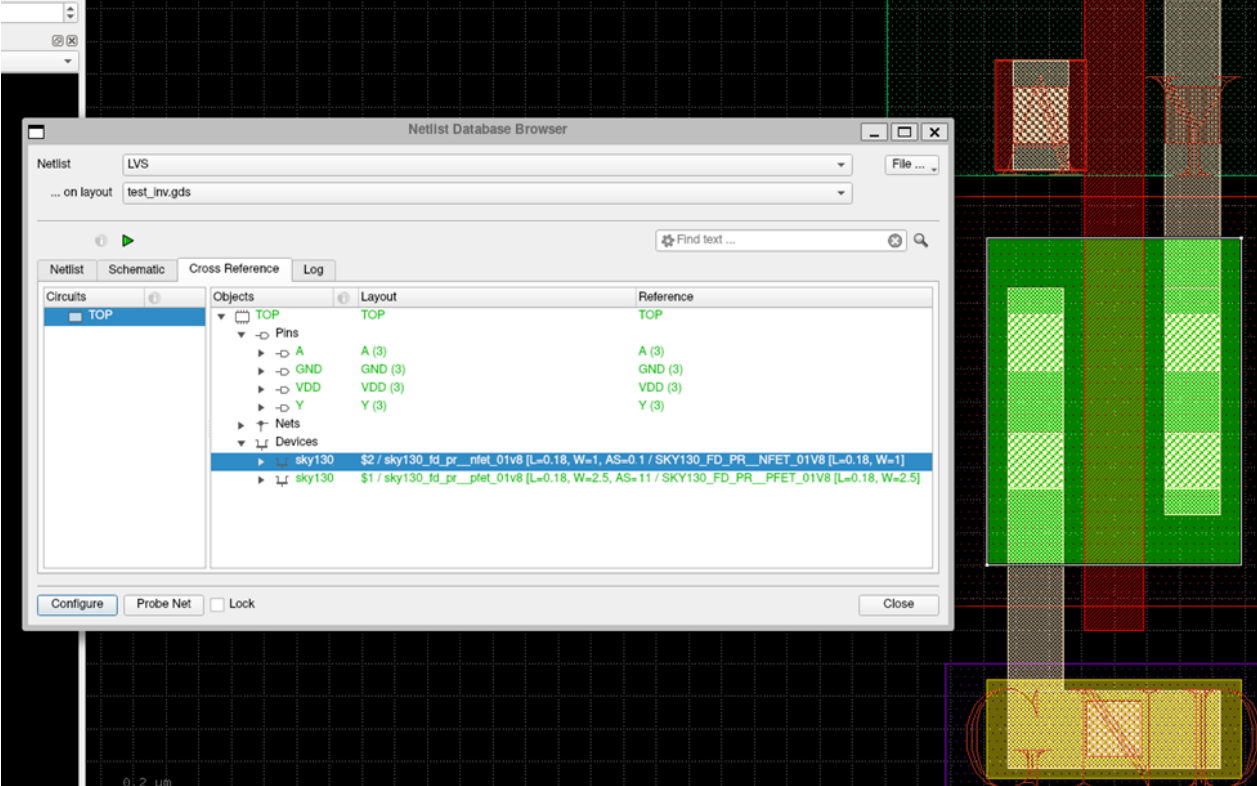


レイアウトを検証して...

DRC



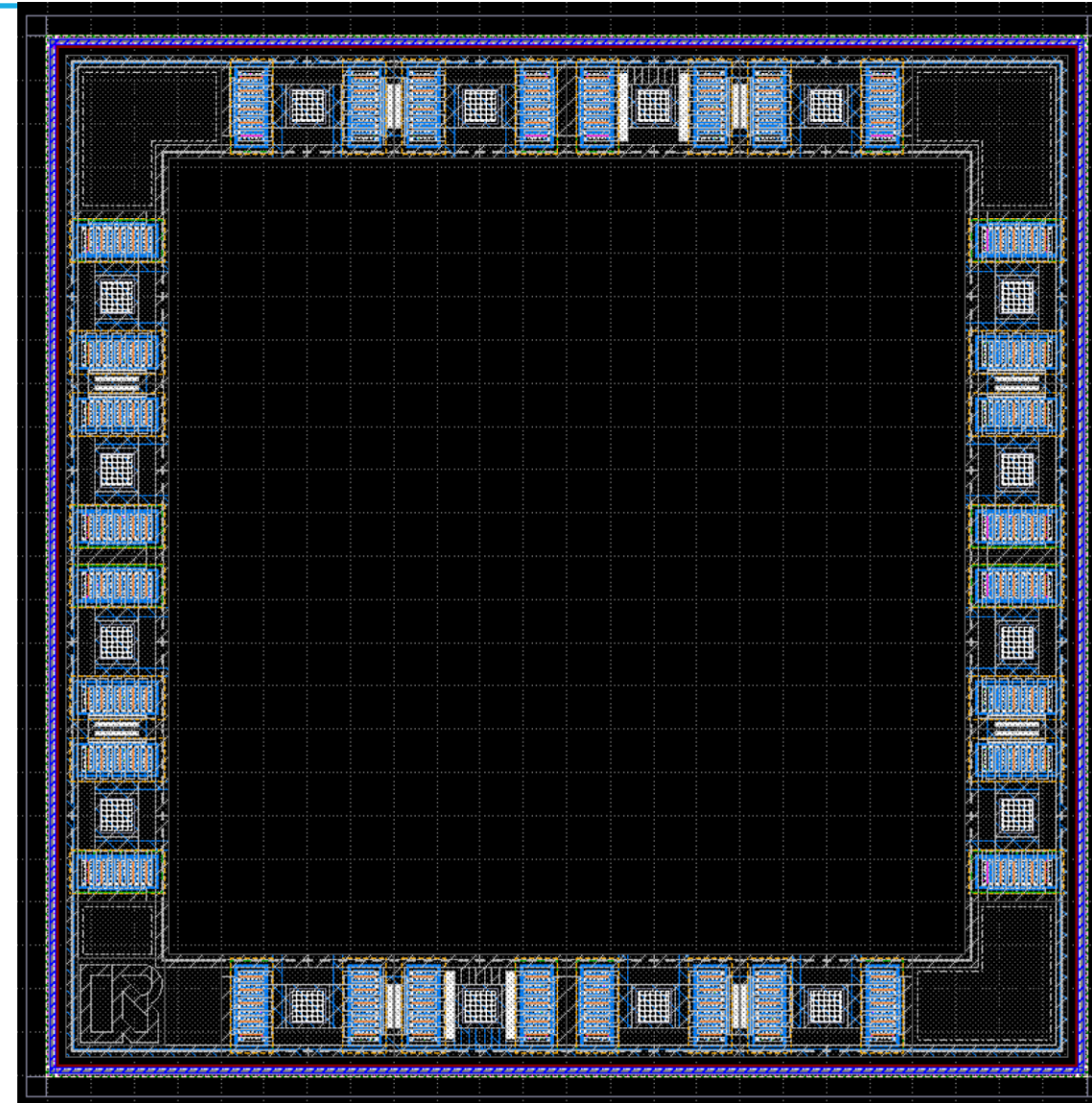
LVS



完成したら指定のフレームに回路を載せる（テープアウト対象者のみ）

指定のフレームにレイアウトを載せる

- 16ピン(ユーザが使用可能なピンは14ピン)
 - 自分が使用するピンについてはピンリストを参照



九州大学大学院システム情報科学府附属価値創造型半導体人材育成センター主催 2025年度実習シリーズで採用

2025_九州大学価値創造型半導体人材育成センター_実習シリーズ③



集積回路設計・評価ハンズオンセミナー

1

九州地域では、TSMC/JASMをはじめとする半導体製造メーカーの集積が進んでおり、「オール九州」での半導体エコシステム構築が始まっています。しかし、その中核を担う設計人材がとても少ないことが問題となっています。そこで九州大学価値創造型半導体人材育成センターでは、設計人材育成のため、半導体の設計から評価までの技術を一気通貫で習得できるセミナーを開催します。

内容: CMOS集積回路(Inverter)をOpen Toolにより設計し、レイアウトします。各自のレイアウトは東海理化のシャトルサービスにより試作します。また、出来上がったICチップを測定し、評価します。ICチップは持ち帰ることができます。

日時場所: 設計ハンズオンセミナー: 2025年9月24日～25日、九州大学 W2-325号室

ICチップお渡し会 & 評価実習: 2026年3月(未定)、九州大学内

※ 9月、3月の2回で1つのセミナーです。WindowsノートPCが必要です。

または、こちら



申込み方法: ・【件名】に参加したいセミナーの名称を記入ください。

・お名前、お名前(ローマ時)、ご所属、メールアドレス をお知らせください。

申し込み先: class_program-at-ecsvc.ed.kyushu-u.ac.jp -at- =@

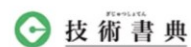
ご協力: OPEN SUSI, ISHI-KAI, AIST solutions、株式会社東海理化

締め切り: 7月23日
先着順で、定員になり次第しめきりとさせていただきます



書籍紹介コーナー

- タイトル : Open Source Silicon Magazine vol.1
 - サブタイトル : はじめてのIC 設計～オリジナルのインバータを作ってみよう～
- 内容
 - 本日の解説から、さらに一步『手前』や端折ってしまっている部分を解説した内容
 - 技術書典「第10回刺され！技術書アワード」のニュースタンド部門のファイナリストに選出されました！
- 価格
 - 電子セット
 - 無料

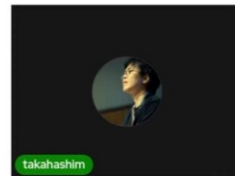


第10回刺され！技術書アワード



最終候補：7作品

- L3スイッチ学習ノート
- 超入門！サーバーサイドKotlin
- Open Source Silicon Magazine Vol.1 はじめてのIC設計
- りあクト！TypeScriptで始めるつらくないReact開発 第5版【③ React実践編】
- Mozilla Historica
- AWSの薄い本6 IAMのマニアックな話 2025
- ランレングスコード法による高速プロップ解析



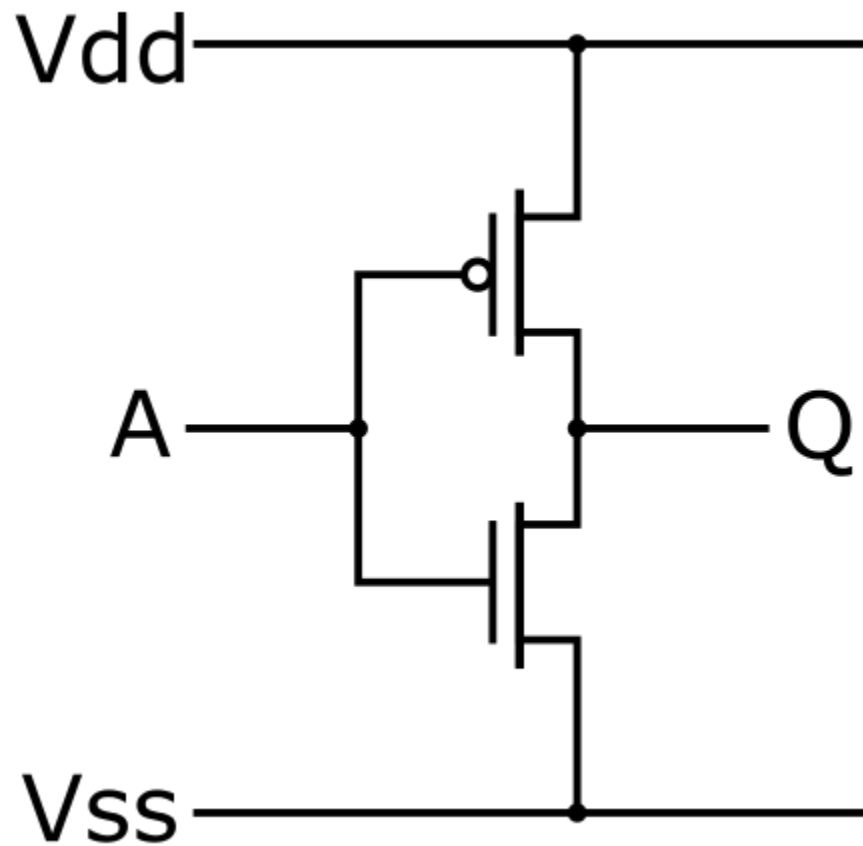
takahashim



アナログ半導体設計デモンストレーション

CMOSインバータ（NOT論理回路）作成

CMOSインバータ（NOT論理回路）



入力A	出力Q
L	H
H	L



入力A	出力Q
Vss	Vdd
Vdd	Vss

アナログ半導体の設計フロー

1. 回路図を描く

2. シミュレーションをする

3. 回路図を基にレイアウトを描く

4. レイアウトを検証する

5. レイアウトを基に寄生成分を考慮したシミュレーションをする

6. フレームに載せる

寄生成分データが
無いため割愛

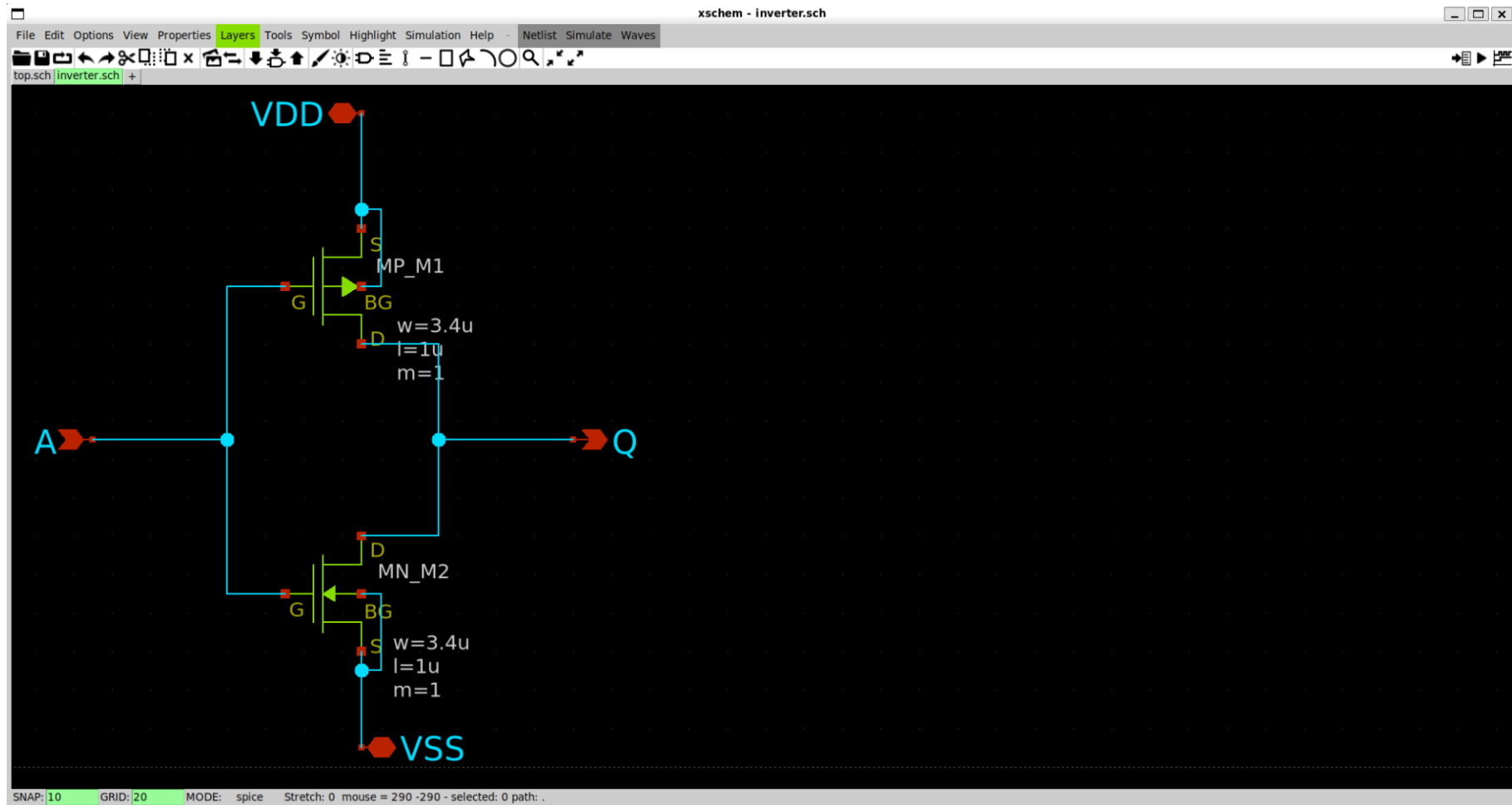
xschem使用上の留意点

- デバイスのシンボルは一部のみ
 - モデル名を変更する必要がある場合もある（P-FET,N-FETでは不要）

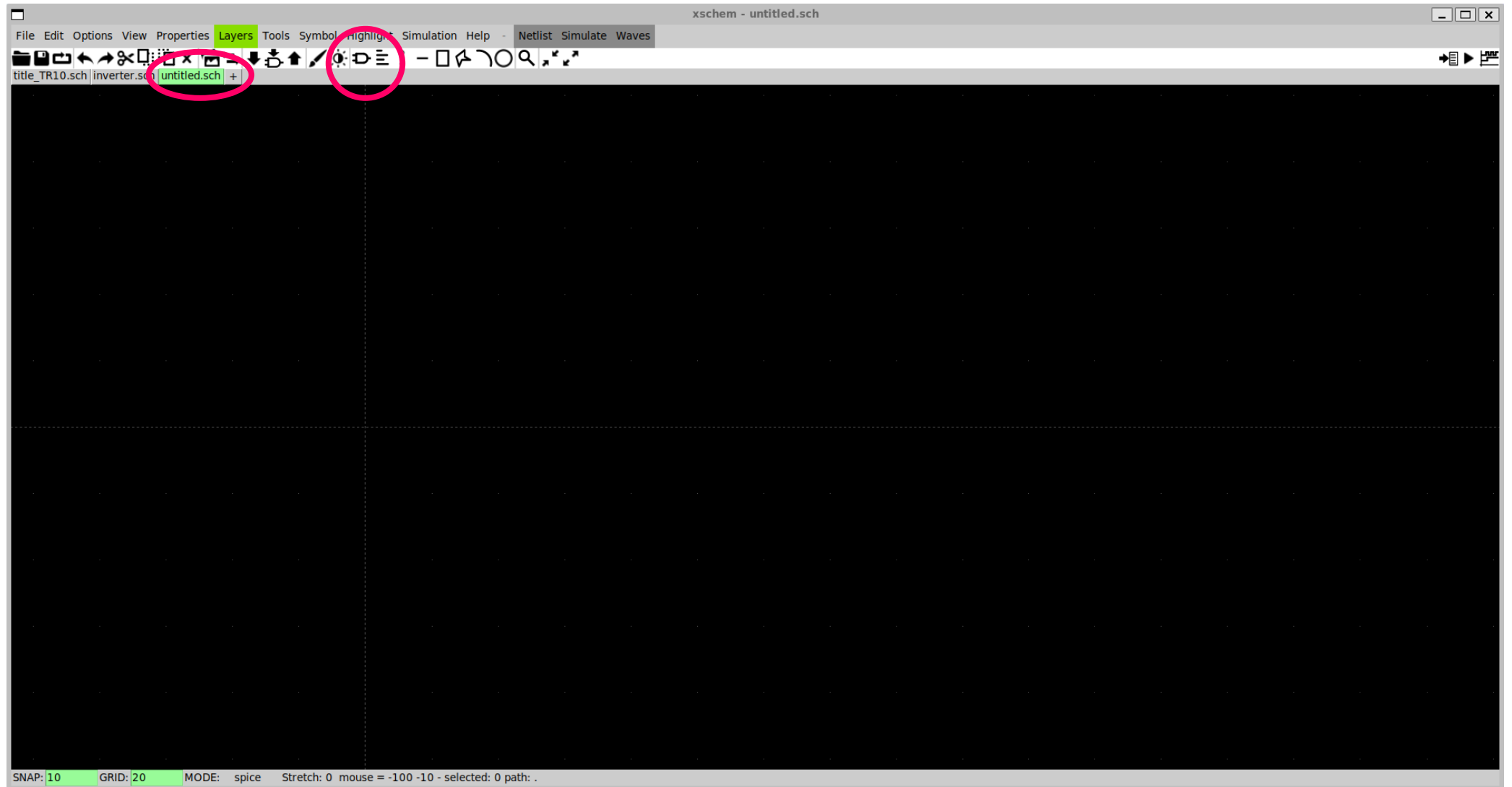
論理検証

- インバータでは入力に対して反転した出力が確認できれば良い
 - 入力 0 V $\rightarrow$ 出力 $V_{dd}\text{ V}$, 入力 $V_{dd}\text{ V}$ $\rightarrow$ 出力 0 V
 - DC解析でCMOSインバータの動作を試みる
- DC解析 電圧を変動させたときの定常応答
- tran解析 時間を変動させたときの過渡応答
- AC解析 周波数を変動させたときの定常応答

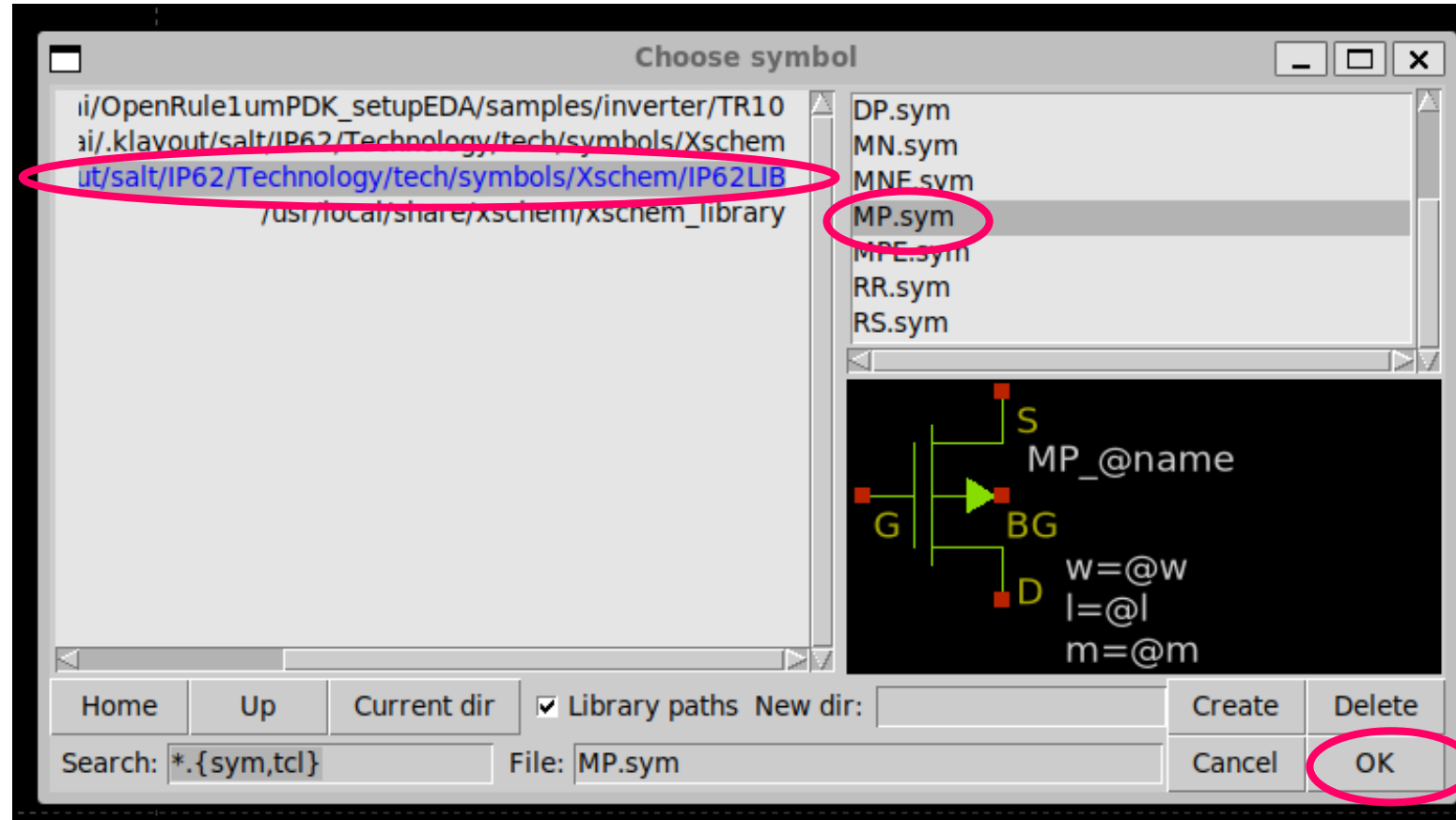
CMOSインバーター回路の例



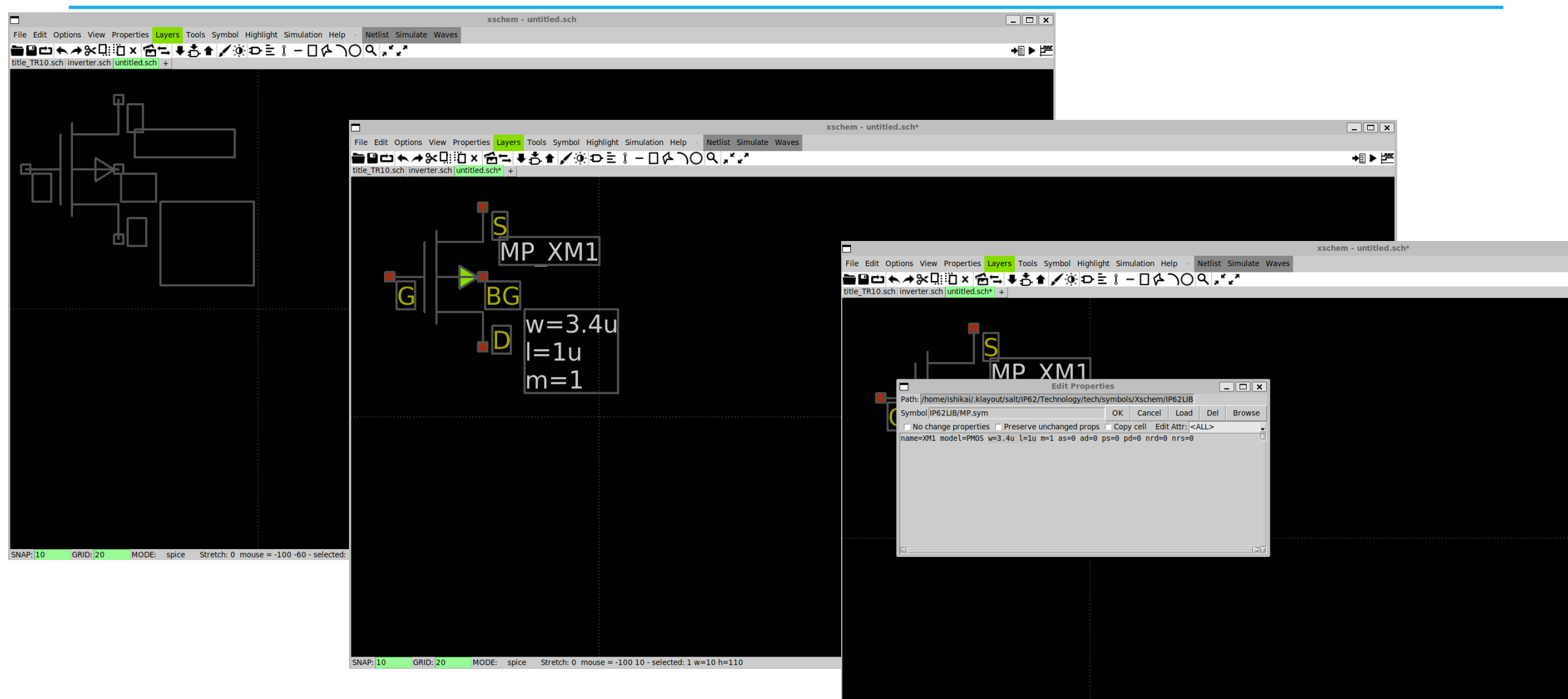
xschem使い方：新規回路図とMODULEウィンドウ



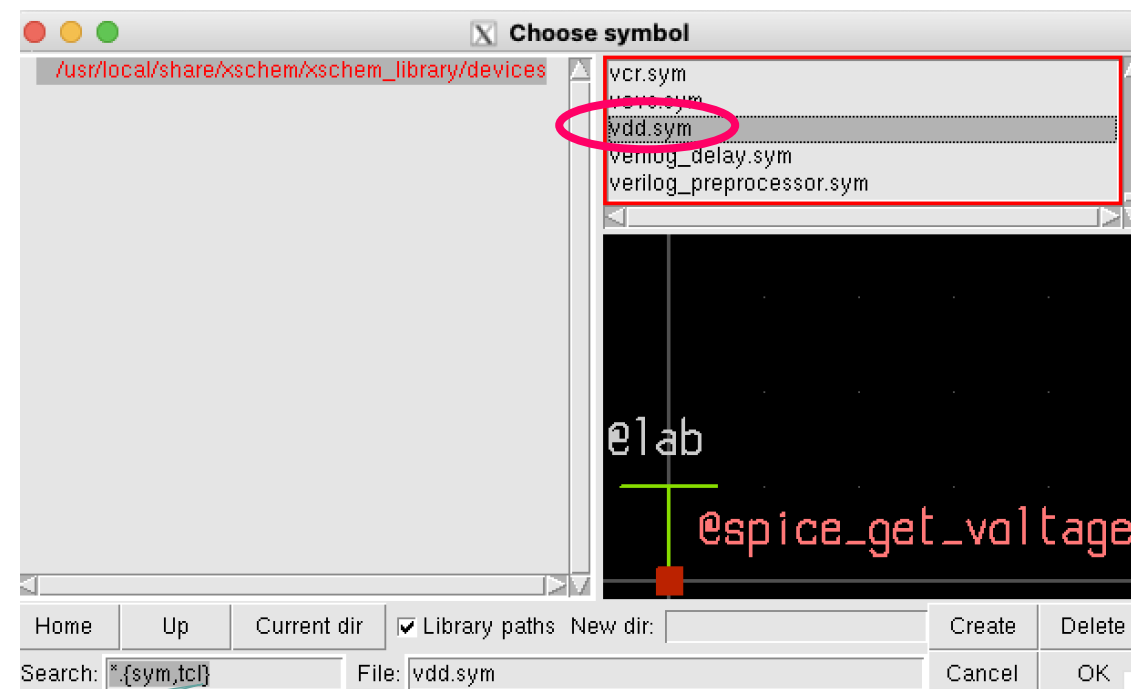
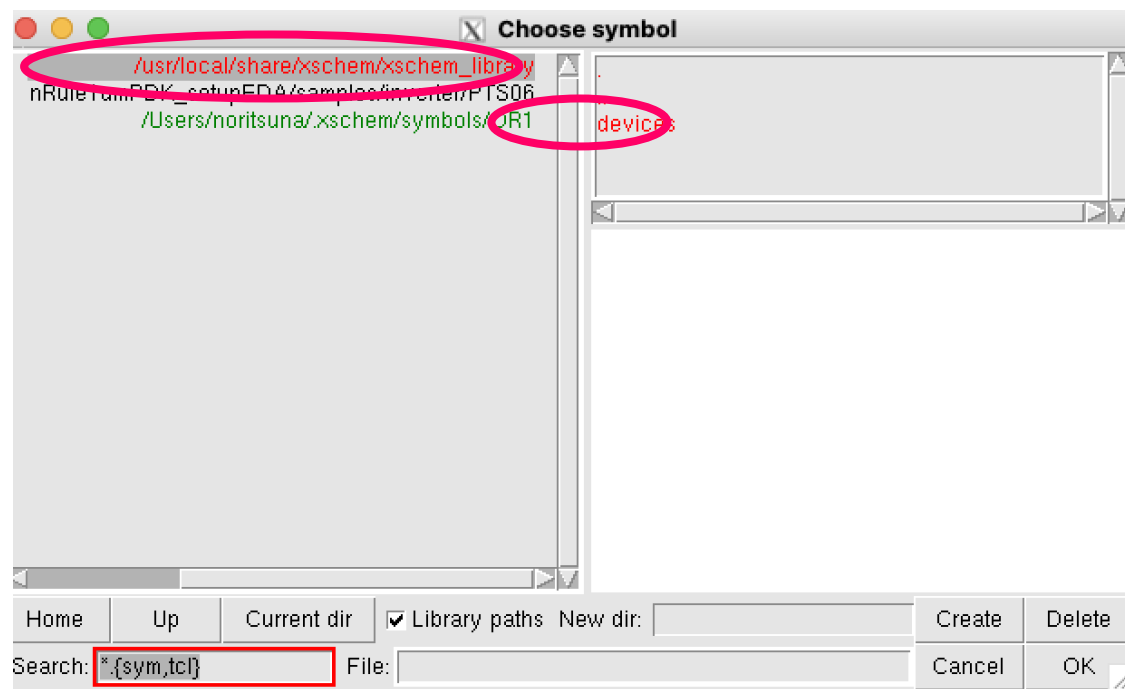
xschem使い方：IP62LIBのMODULEの選択：P-FET



xschem使い方：IP62LIBのMODULEの配置とプロパティ



xschem使い方：標準のMODULEの選択：VDD



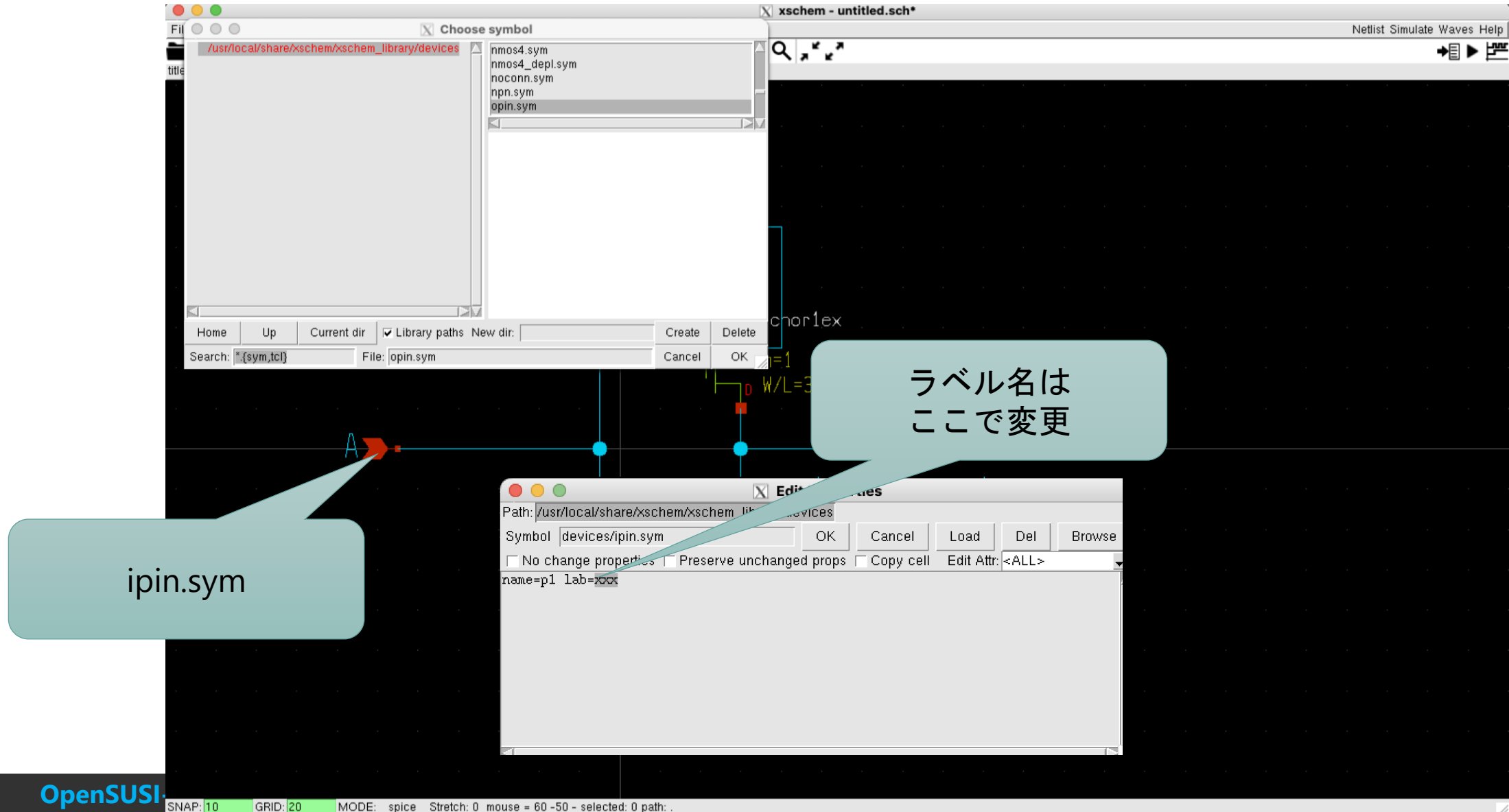
検索も可能

xschem使い方：標準のMODULEの配置とプロパティ：VDD

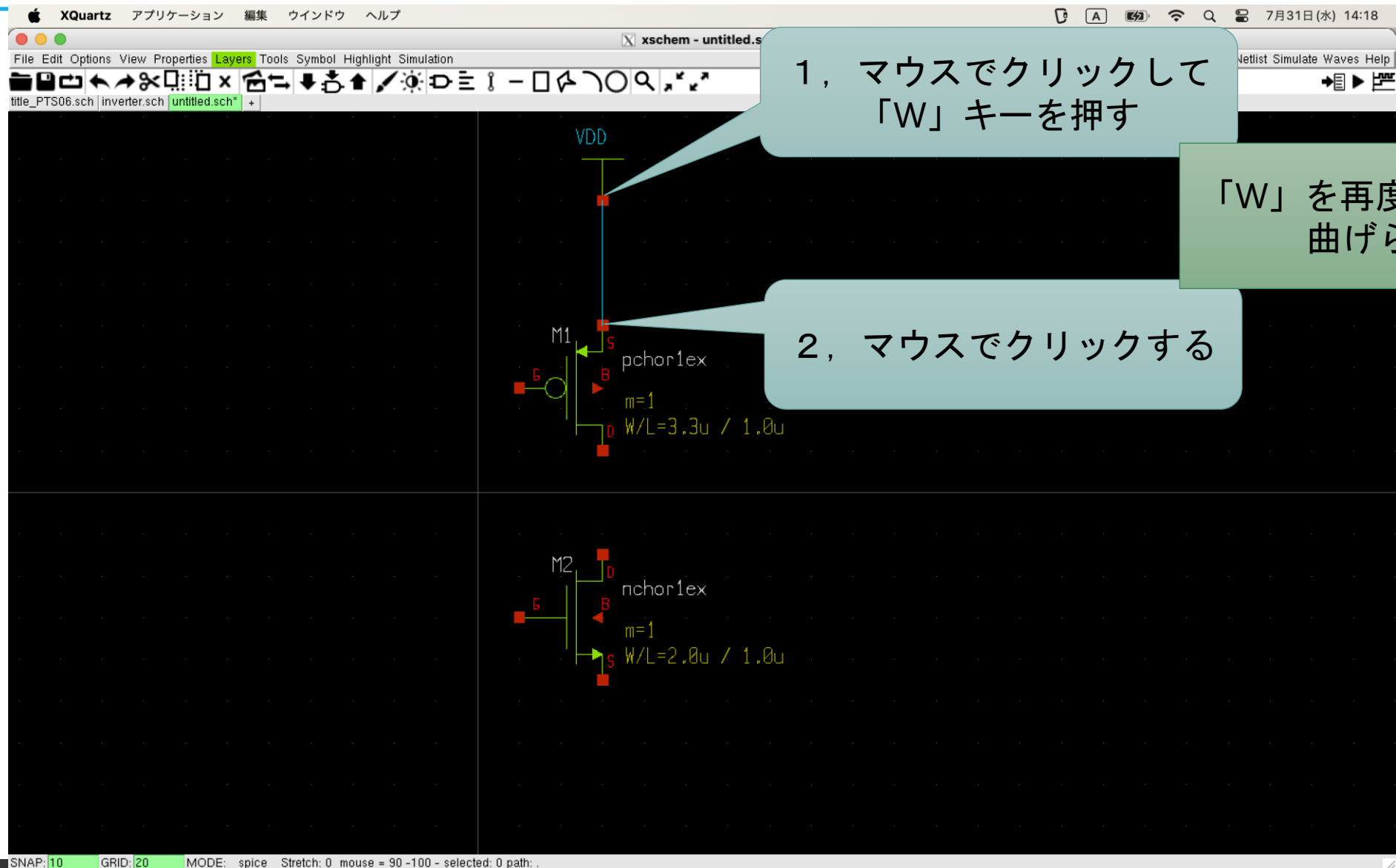
The screenshot displays the xschem interface with a circuit diagram. Two MOSFETs, M1 and M2, are shown. M1 is a PMOS transistor with its source connected to VDD and its gate to ground. M2 is an NMOS transistor with its source connected to ground and its gate to VDD. The VDD supply is represented by a symbol with a label 'VDD'. An 'Edit Properties' dialog box is open for the VDD component, showing the path '/usr/local/share/xschem/xschem_library/device' and the symbol 'devices/vdd.sym'. The 'lab' field is highlighted, and a callout bubble points to it with the text 'ラベル名'.

ラベル名

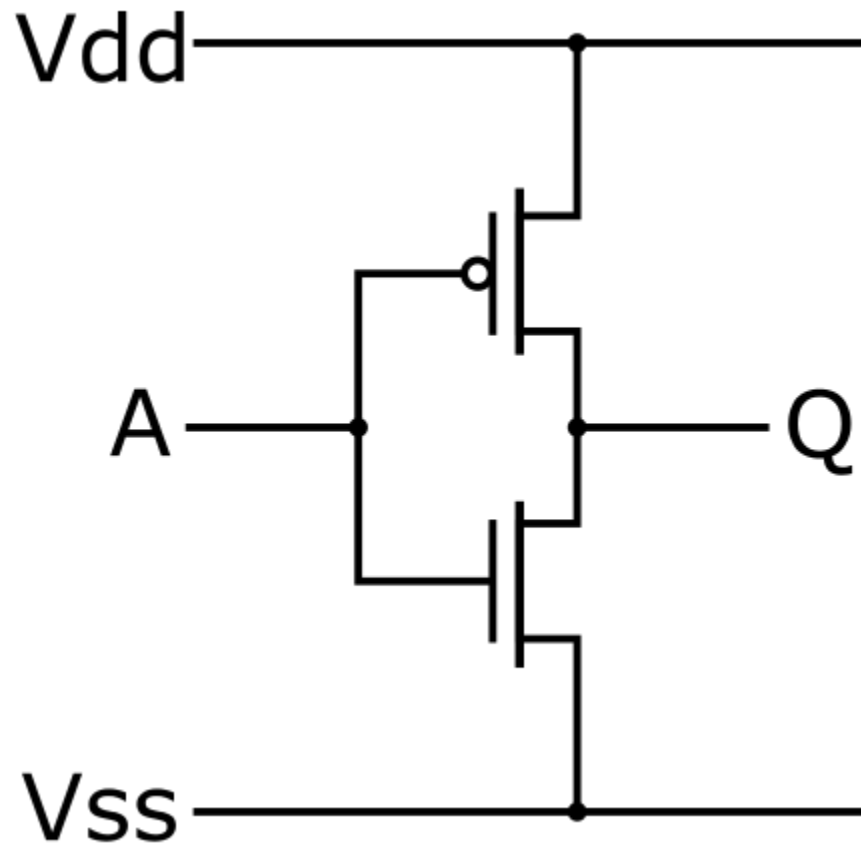
xschem使い方：標準のMODULEの選択：ipin, opin, iopin



xschem使い方： wireの引き方



ハンズオン：CMOSインバータを作る

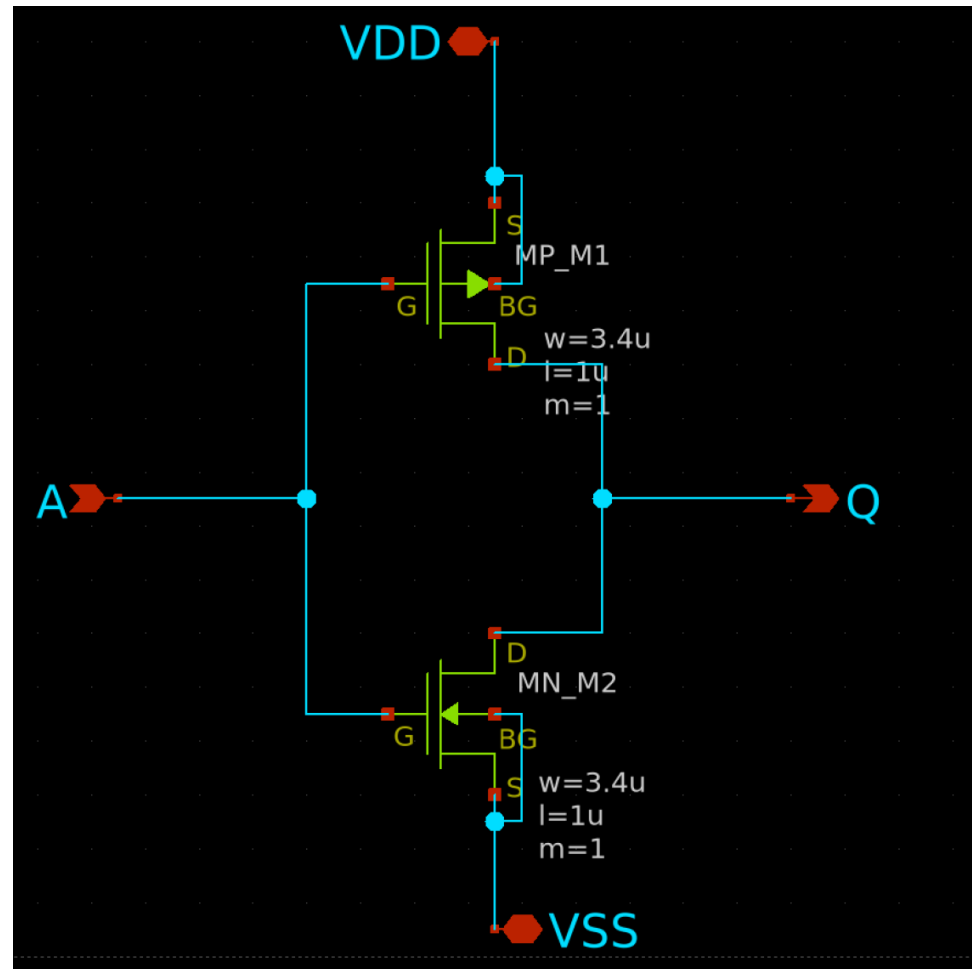


入力A	出力Q
L	H
H	L



入力A	出力Q
V_{ss}	V_{dd}
V_{dd}	V_{ss}

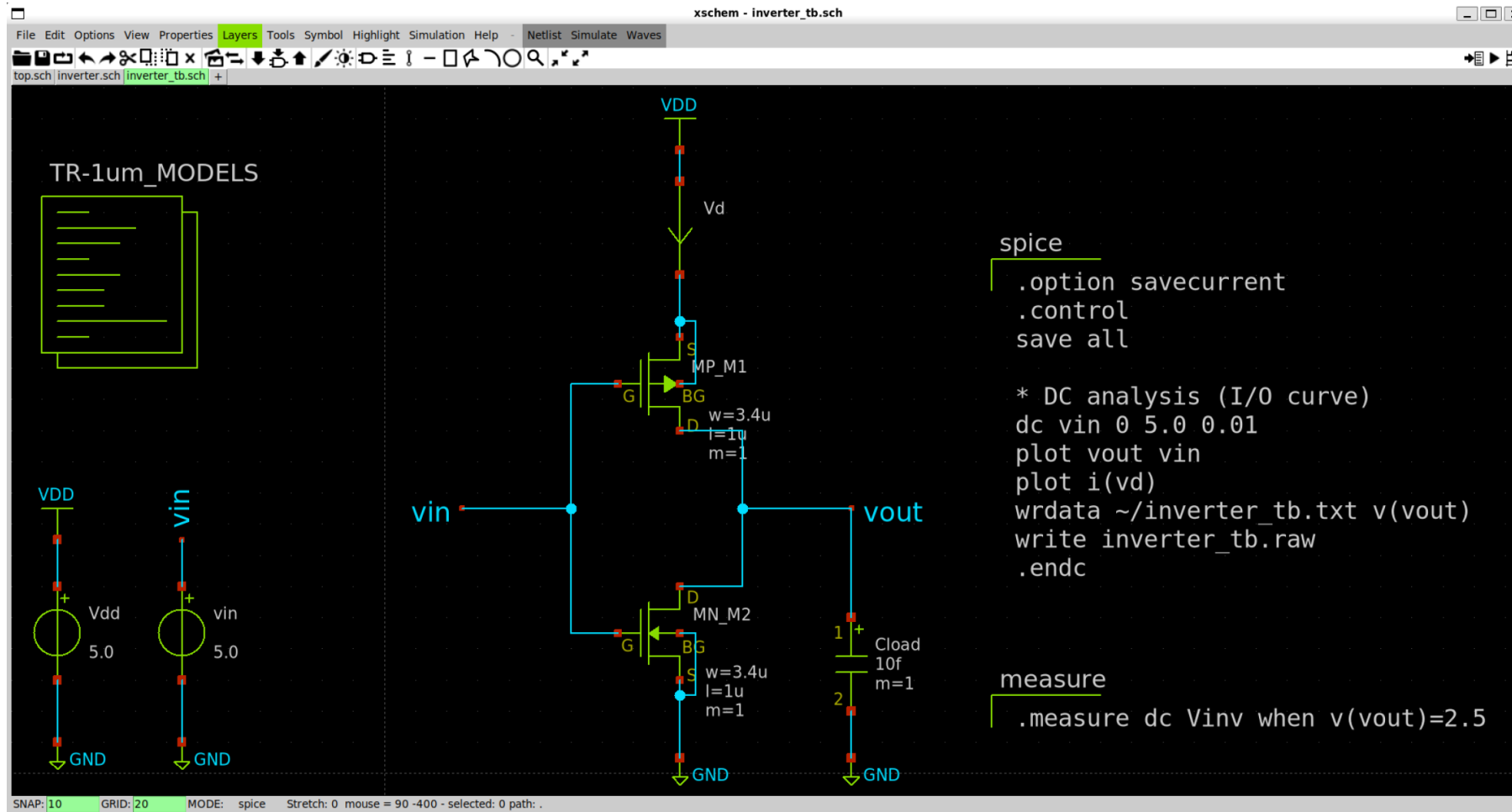
ハンズオン：CMOSインバータのサンプル



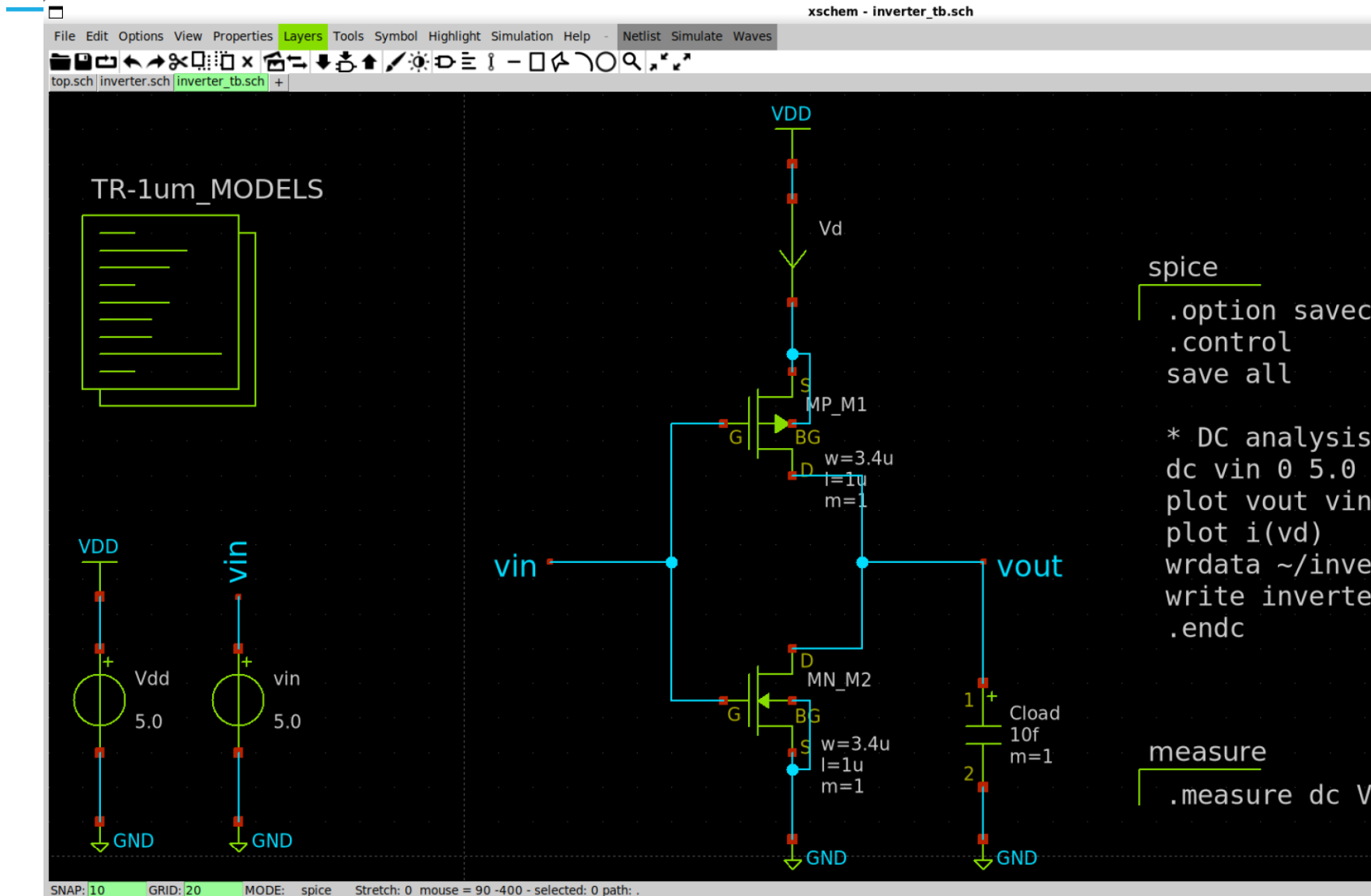
アナログ半導体の設計フロー

1. 回路図を描く
2. シミュレーションをする
3. 回路図を基にレイアウトを描く
4. レイアウトを検証する
5. レイアウトを基に寄生成分を考慮したシミュレーションをする
6. (フレームに載せる)

xschem : ベンチマーク例



xschem : ベンチマーク



- キーワード
 - code.sym
 - code_shown.sym
 - vsource.sym
 - gnd.sym
 - ammeter.sym
 - lab_pin.sym
 - capa.sym
 - vdd.sym

spice

```
.option savecurrent  
.control  
save all
```

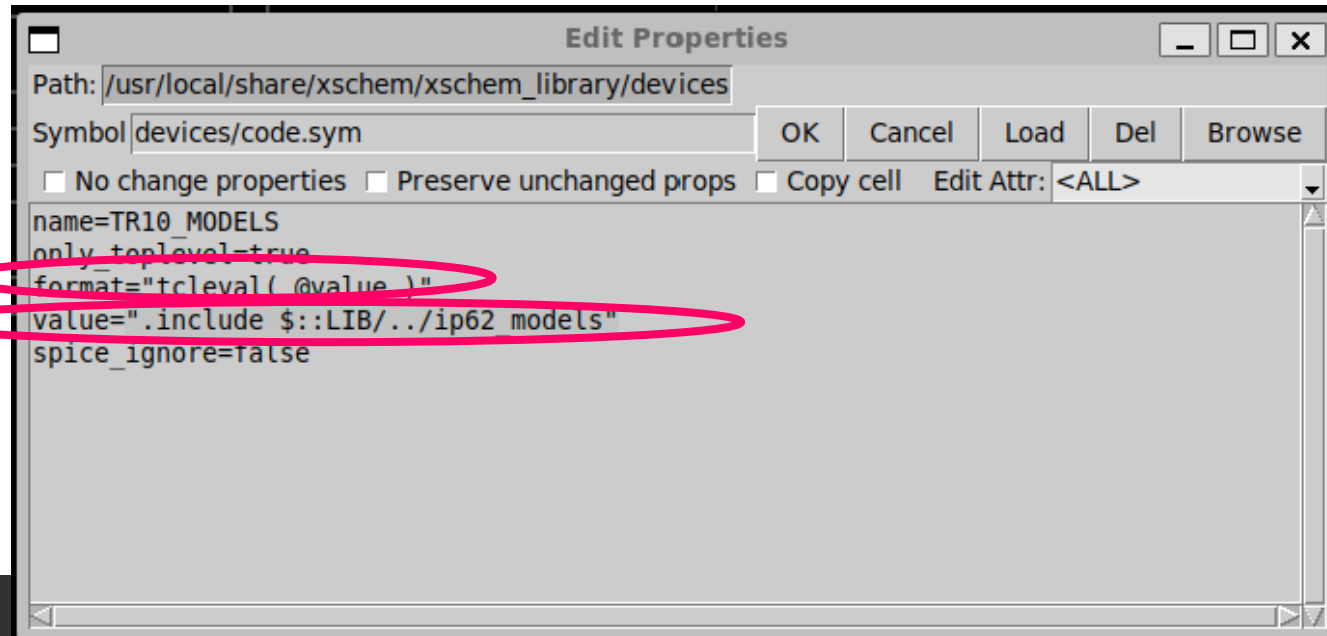
```
* DC analysis (I/O curve)  
dc vin 0 5.0 0.01  
plot vout vin  
plot i(vd)  
wrdata ~/inverter_tb.txt v(vout)  
write inverter_tb.raw  
.endc
```

measure

```
.measure dc Vinv when v(vout)=2.5
```

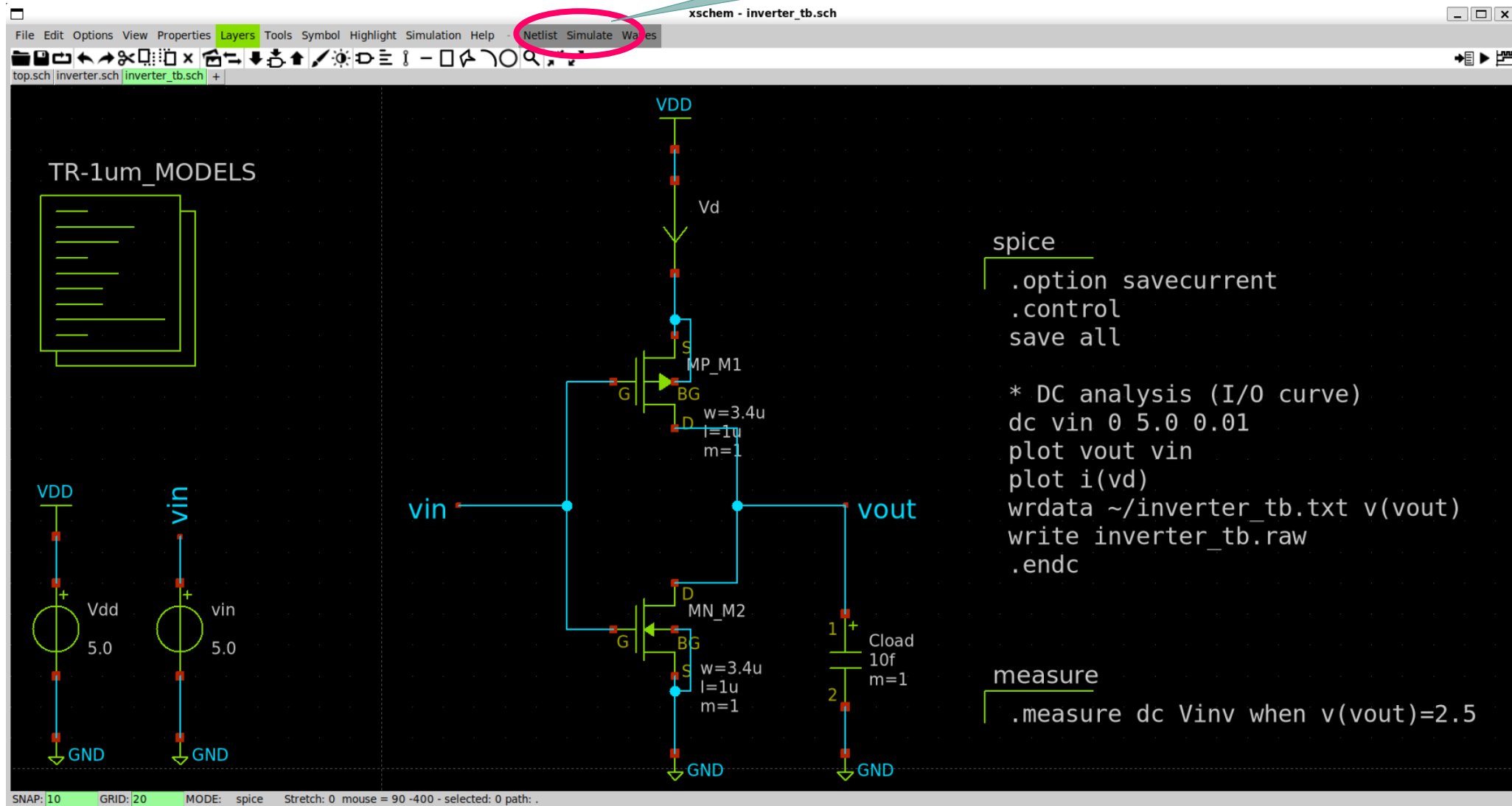
xschem : TR10_MODELSを記述する際の注意点x

- code.symを使用する際、以下の行を追記する必要がある
 - format="tcleval(@value)"
- includeは絶対パス（フルパス）である必要がある。相対パスは使えない。
- 「\$::LIB」は特殊パスのため利用可能

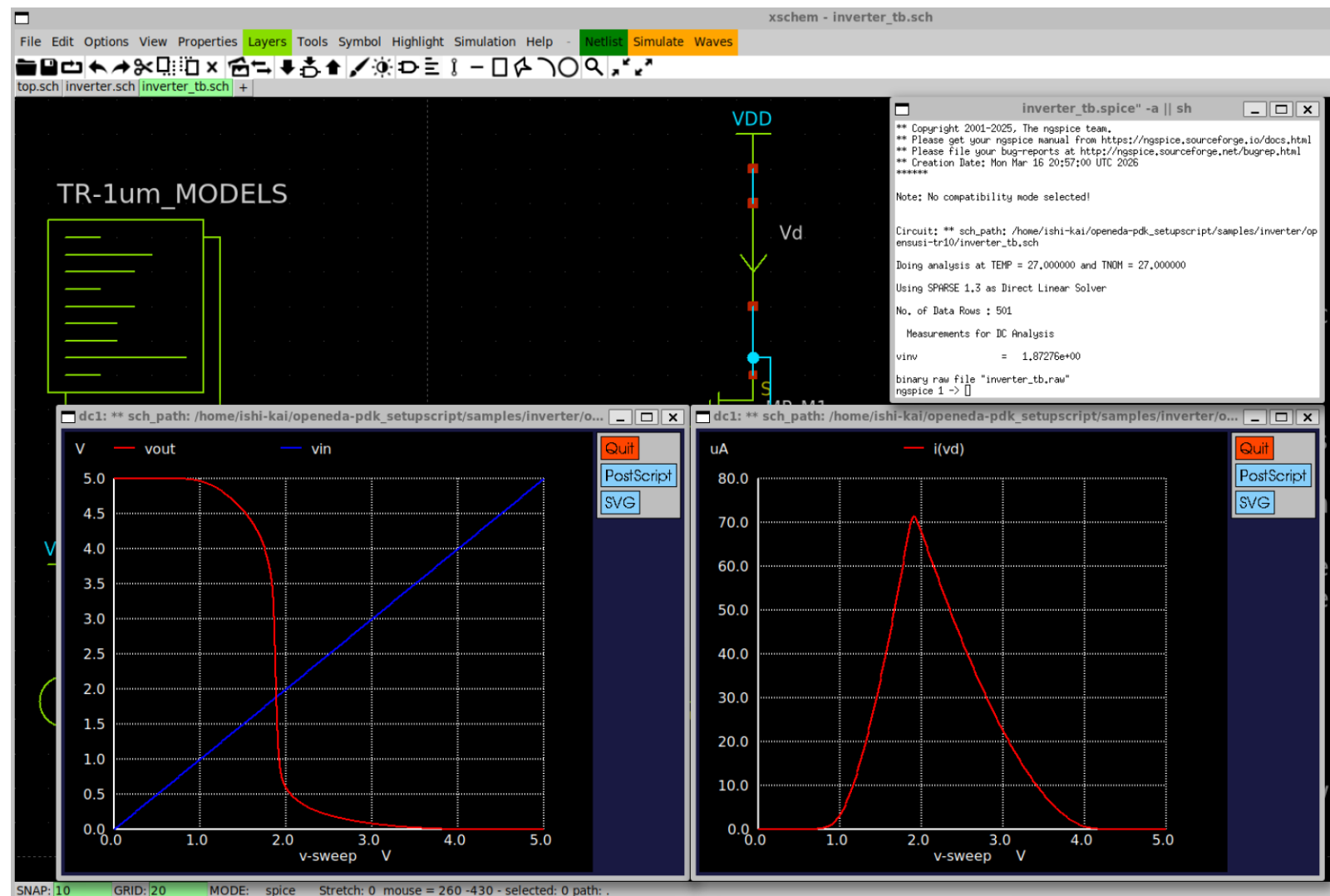


ngspice : シミュレーションの実行

netlist→simulateの
順にクリック



ngspice : シミュレーション結果



入力A = 0 V → 出力Q = 5.0 V

入力A = 5.0 V → 出力Q = 0 V

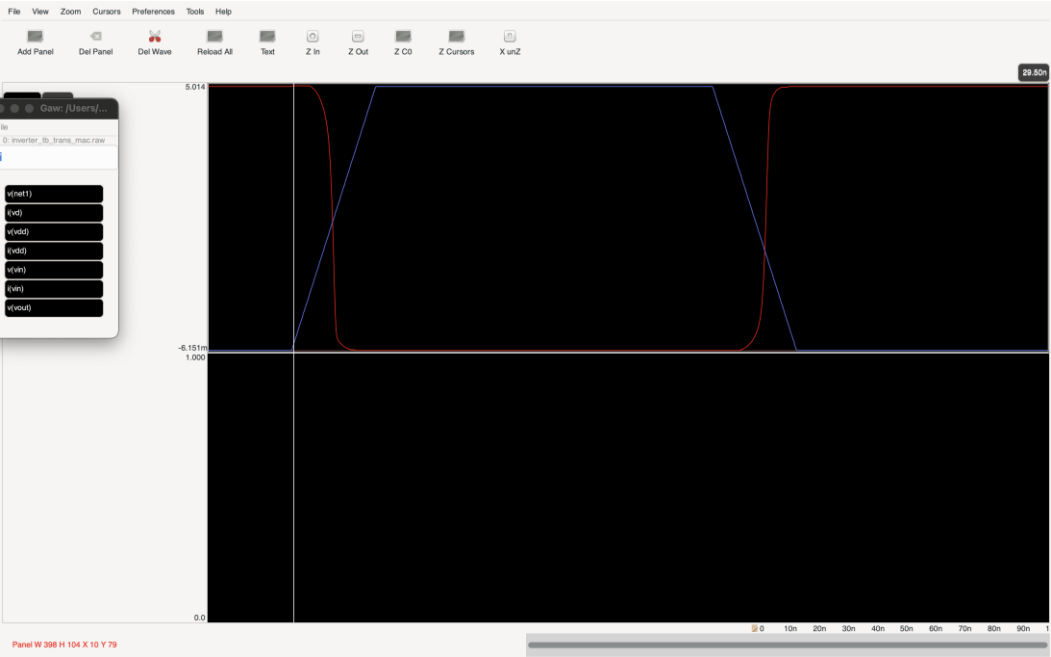
- インバータとして機能している

ngspice : wrdata と gawによる波形表示

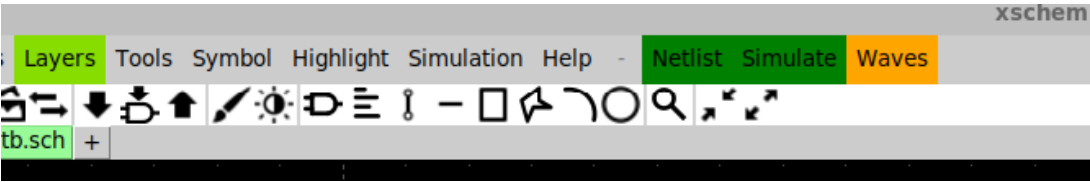
wrdataを使用するとテキスト出力ができる

0.00000000e+00	3.29999999e+00
5.00000000e-02	3.29999999e+00
1.00000000e-01	3.29999997e+00
1.50000000e-01	3.29999990e+00
2.00000000e-01	3.29999960e+00
2.50000000e-01	3.29999842e+00
3.00000000e-01	3.29999375e+00
3.50000000e-01	3.29997564e+00
4.00000000e-01	3.29990878e+00
4.50000000e-01	3.29968137e+00
5.00000000e-01	3.29900073e+00
5.50000000e-01	3.29728006e+00

writeでrawファイルを出力すると gaw で波形表示ができる



-
-
-



xschem 右上の
Wavesから起動できる

ngspice : 過渡応答、遅延時間

- 過渡応答から遅延時間を見たい場合はtran解析を用いる
- 出力段に負荷を設定しないと正しい過渡応答が見れない
今回は入力段・出力段共に何も接続しない状態を見る

ngspice : 過渡解析ベンチマーク例

The screenshot shows the xschem - inverter_tb_trans.sch window. The circuit diagram on the left includes a VDD source, a pulse input source (pwl) for vin, and an inverter circuit consisting of an NMOS transistor (MN_M2) and a PMOS transistor (MP_M1). The output vout is connected to a load capacitor (Cload) and a voltage source (Vd). The SPICE netlist on the right is as follows:

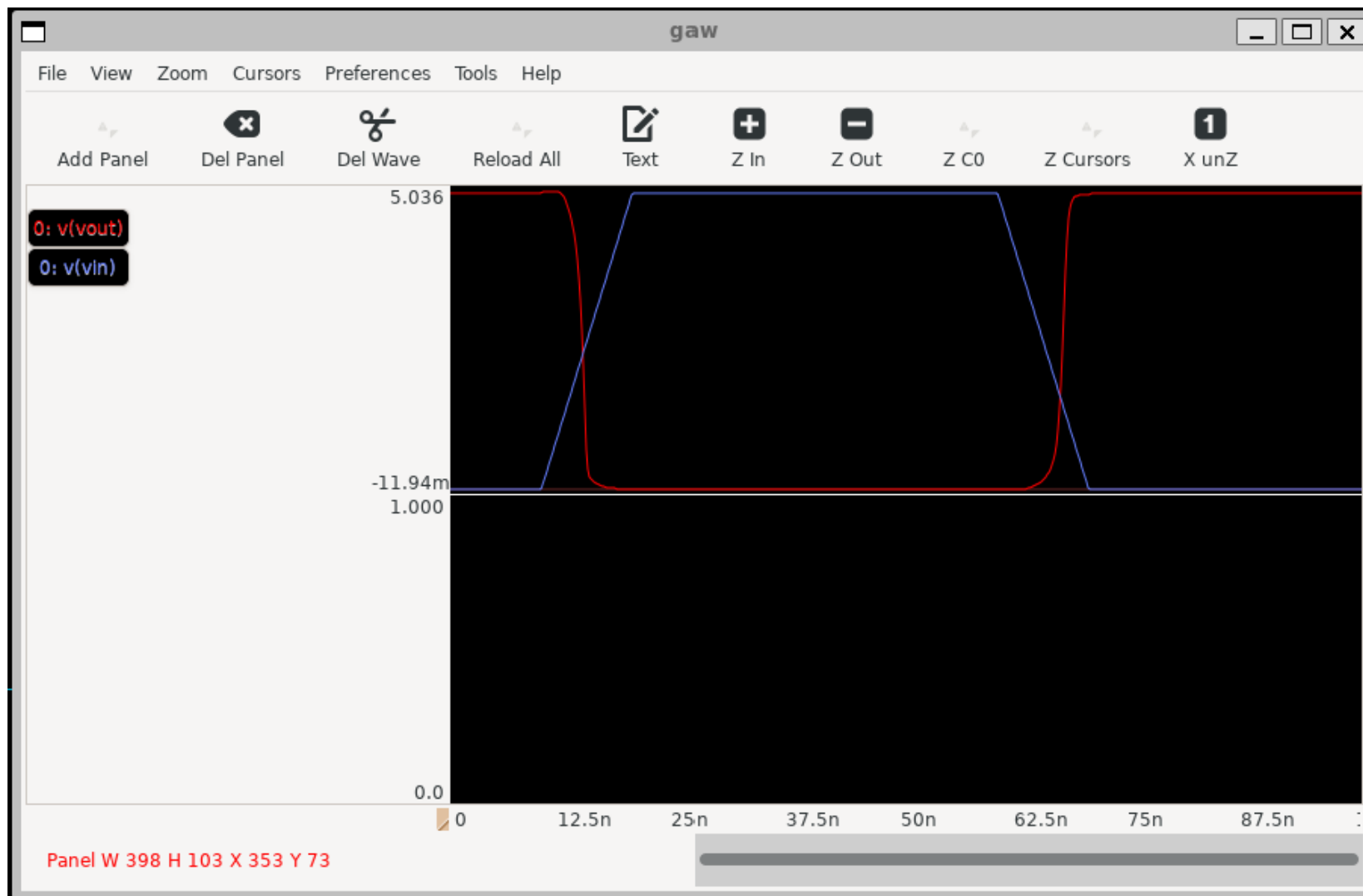
```
spice
.option savecurrent
.control
save all

* Tran analysis
tran 0.1n 100n
plot vout vin
plot i(vd)
wrdata ~/inverter_tb_tran.txt v(vout)
write inverter_tb_trans.raw
.endc

measure
.measure tran td_r trig v(vin) val=2.5 fall=1 targ v(vout) val=2.5 rise=1
.measure tran td_f trig v(vin) val=2.5 rise=1 targ v(vout) val=2.5 fall=1
.measure tran trise trig v(vout) val=0.83 rise=1 targ v(vout) val=4.17 rise=1
.measure tran tfall trig v(vout) val=4.17 fall=1 targ v(vout) val=0.83 fall=1
```

At the bottom of the window, the status bar shows: SNAP: 10 GRID: 20 MODE: spice Stretch: 0 mouse = 210 -650 - selected: 0 path: .

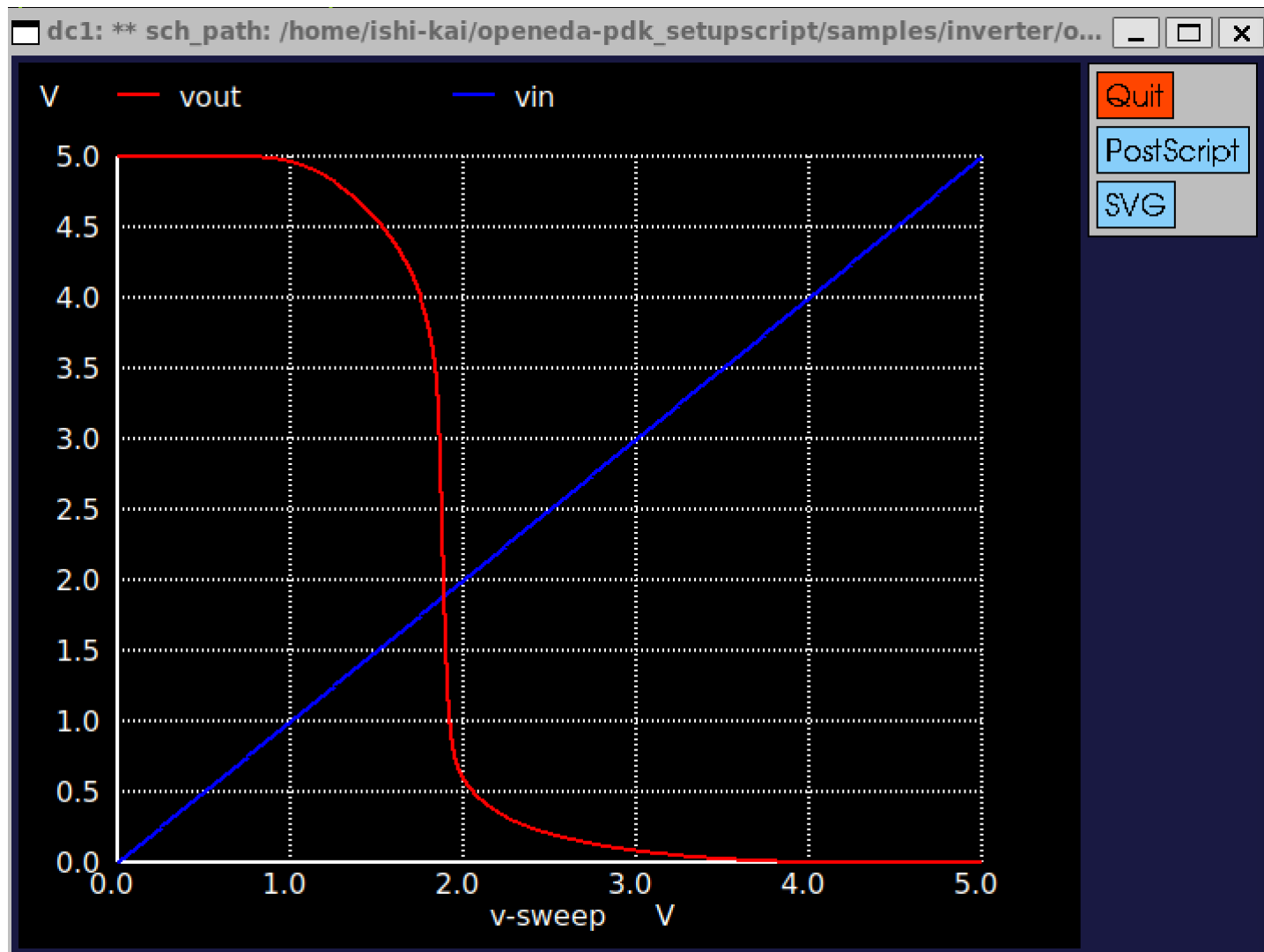
ngspice : 過渡解析シミュレーション結果



半導体の基礎知識

回路の設計

インバータ回路の設計ポイント

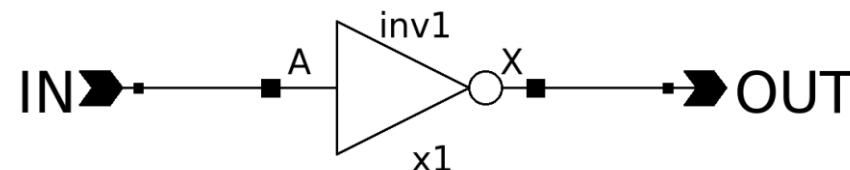
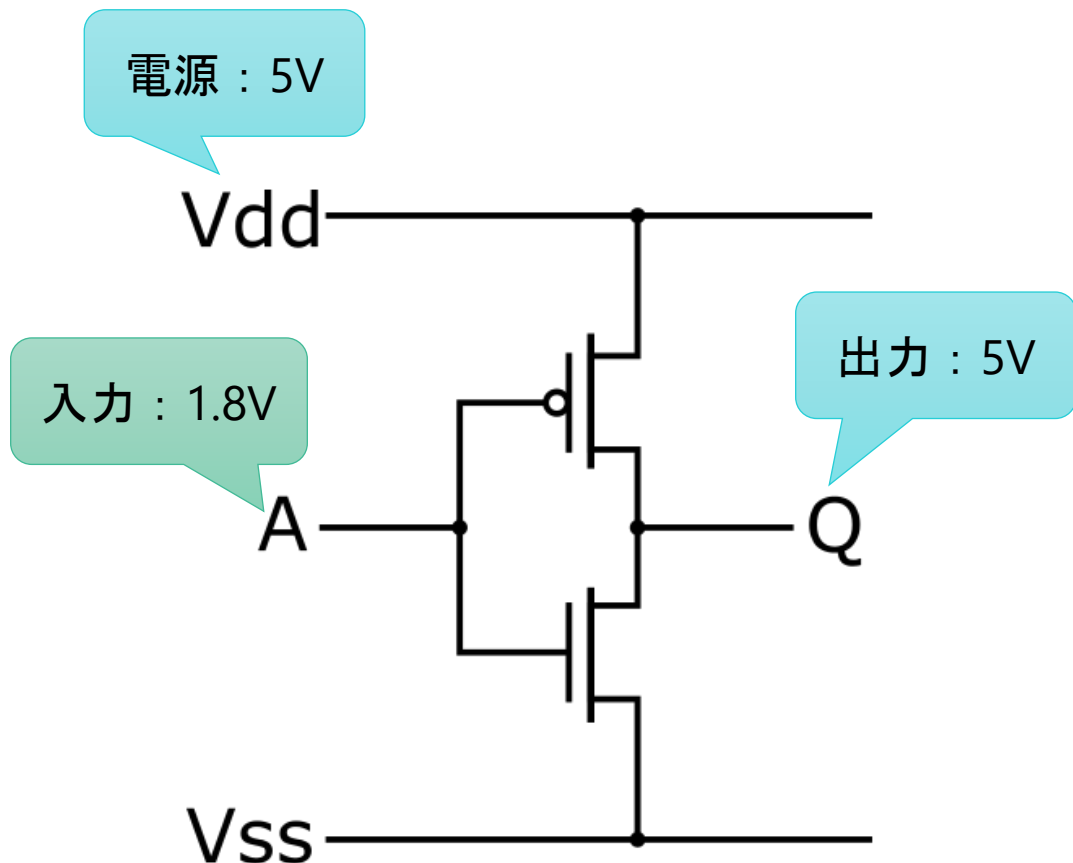


入力A = 1.8V → 出力Q = 0 V

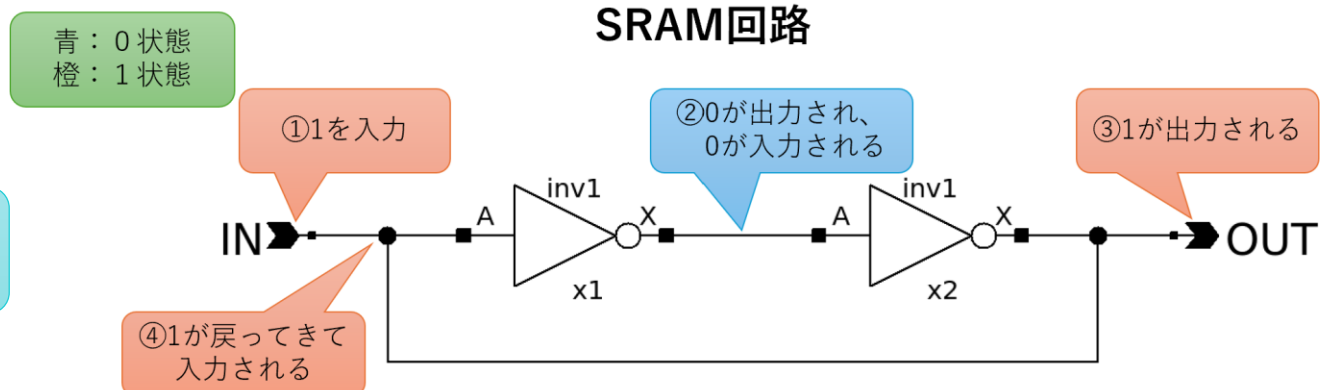
- 本来は、VDD=5.0Vの半分である入力A = 2.5Vで動作すべき
- 入力A = 2.5Vより低い方向のため、PMOSの性能が低いことが原因
- 対策
 - PMOSの性能を上げる
 - NMOSの性能を下げる

インバータ回路ってなにもの？

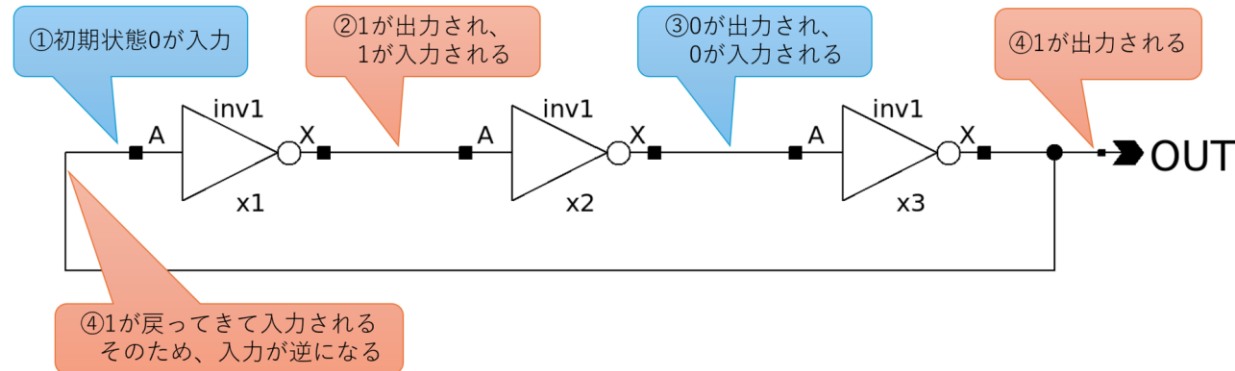
- 本来は、 $V_{DD}=5.0V$ の半分である入力 $A = 2.5V$ で動作すべき



SRAM回路

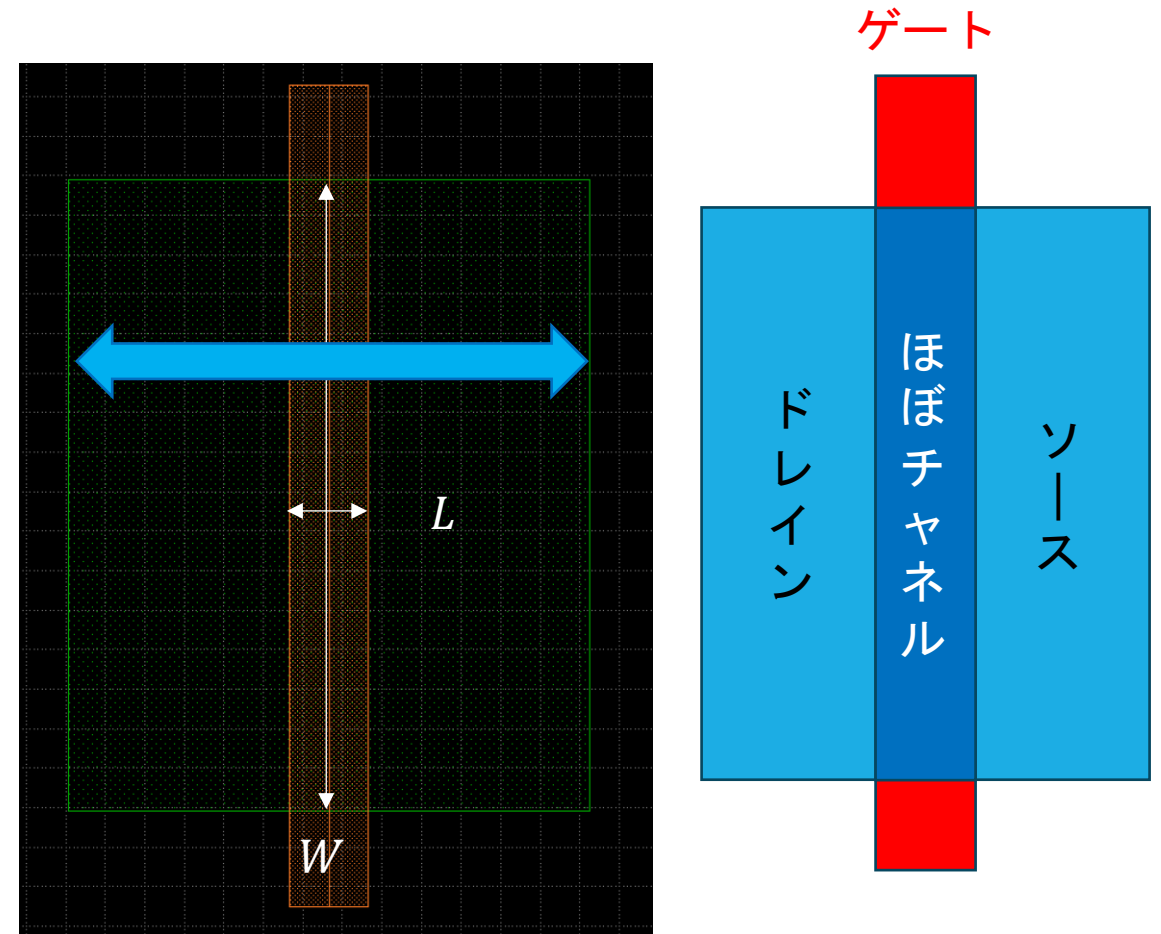


リングオシレータ回路



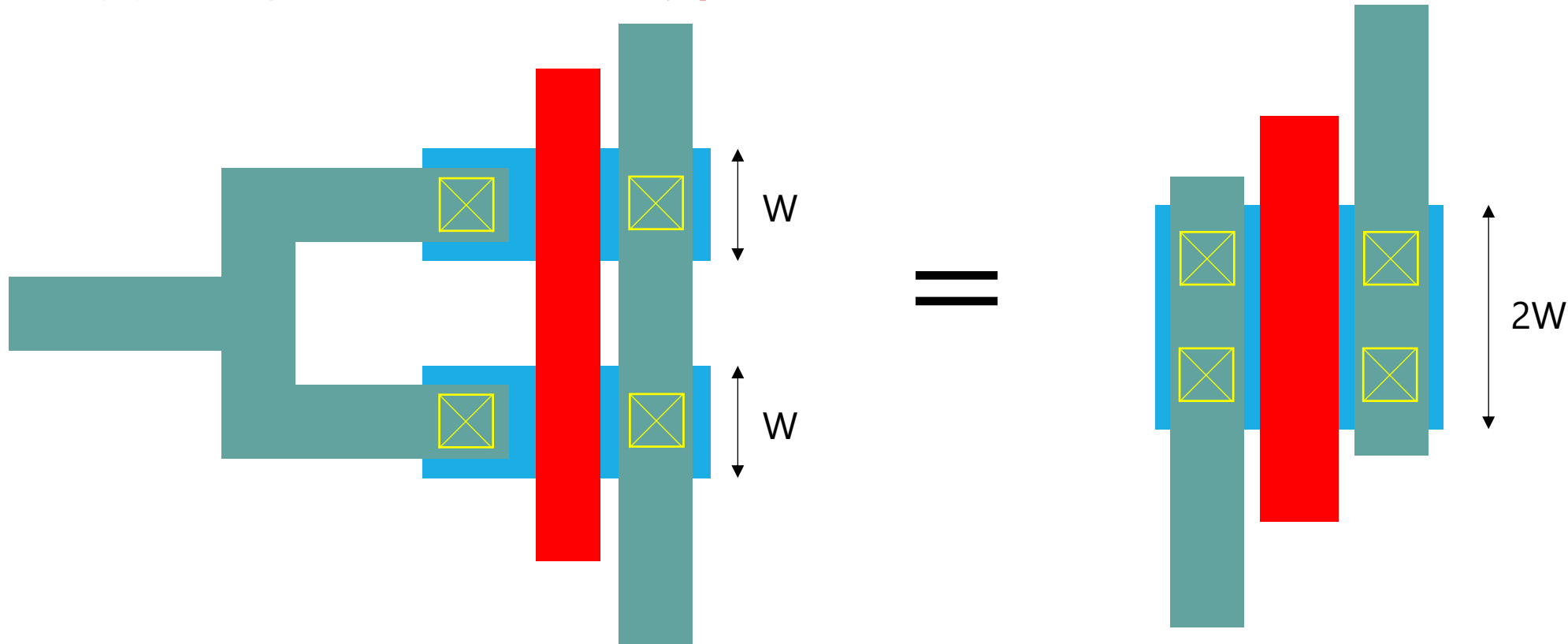
トランジスタのパラメータ

- 基本は W と L を変える
 - W : チャネル幅
 - L : チャネル長 (※実際のチャネル長はちょっと短い)
- **ゲートと拡散層の重なった部分がトランジスタ本体**
ゲートに十分な電位を与えると**チャネル**が形成
- m , nf , mf などはゲートの本数 (マルチフィンガー)



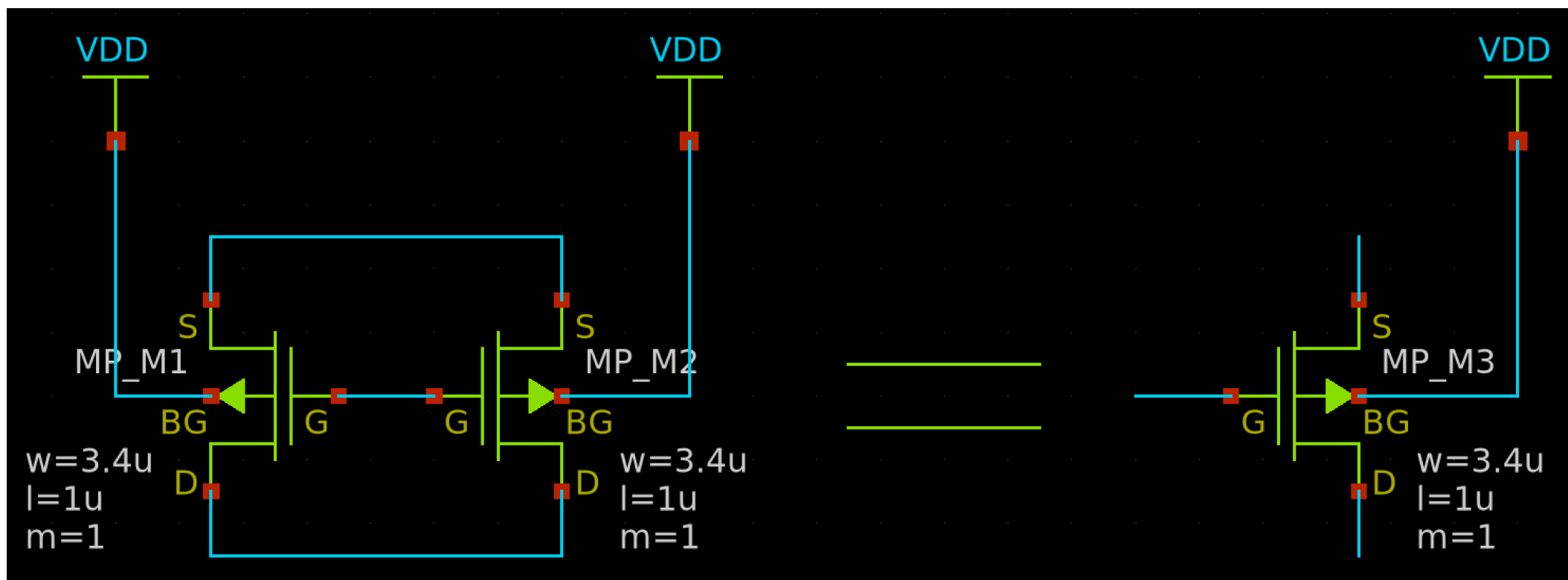
チャンネル幅 W を変えるとどうなる？

- チャンネル幅を大きくすると電流の通り道が広がるので、ドレインソース間の電流は増える
- チャンネル長が同じトランジスタでは、**チャンネル幅とドレインソース間電流は比例の関係**



ゲートの本数

ドレイン・ソース・ゲート・ボディが同じノードである同じサイズのトランジスタは、 W が2倍のトランジスタに等しい
これを「 m , nf , mf 」で表現し、**マルチフィンガー**という



チャンネル長 L を変えるとどうなる？

- **短チャンネル効果**による影響で、特性が全く変化する。
チャンネル長 L が短くなると基本は I_{ds} が増加するが、**短チャンネル効果**も発生
- DIBL (Drain Induced Barrier Lowering)により、 V_{ds} の増加に対して V_{th} が低下
- Subthreshold swingの増加により、弱反転領域におけるパンチスルーが増加
 - オフ状態でのリーク電流が増え、 V_{gs} の増加幅に対する I_{ds} の増加幅が鈍くなる →スイッチング特性劣化
- チャンネル長変調効果により、 V_{ds} 変動幅に対する I_{ds} 変動幅が増え、**飽和状態にならない**

チャンネル長 L を変えるとどうなる？

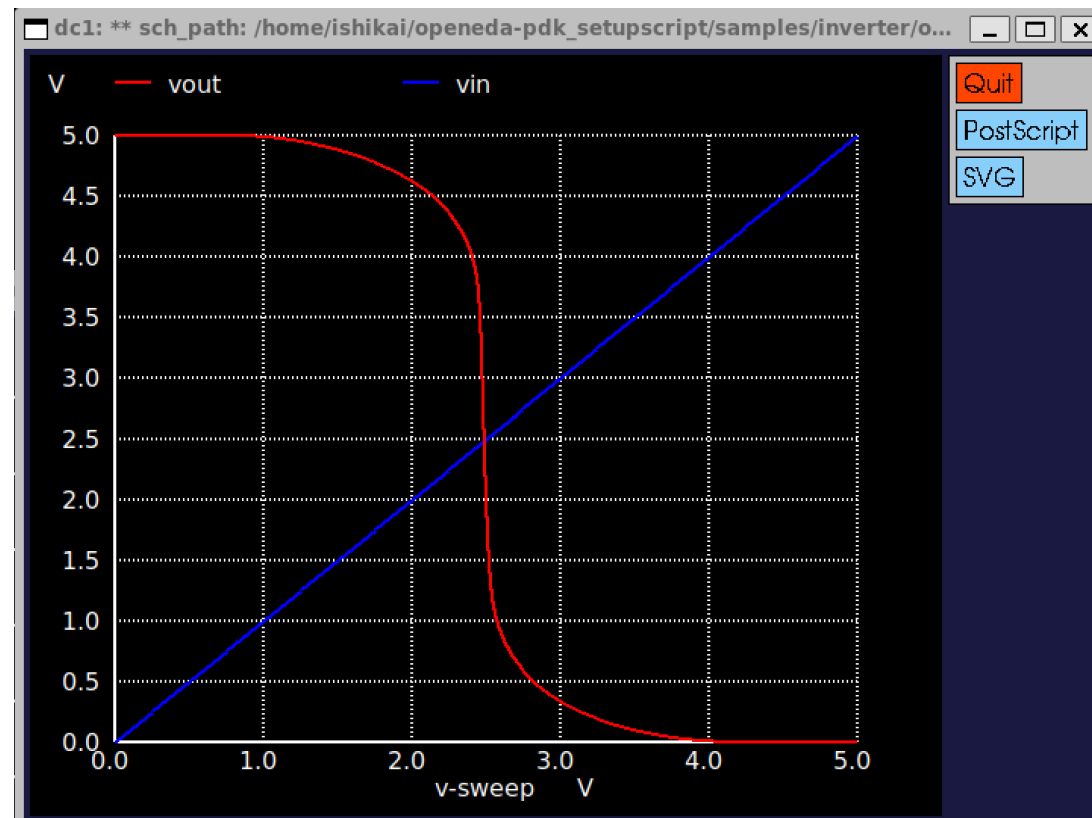
- **短チャンネル効果**による影響で、特性が全く変化する。
チャンネル長 L が短くなると基本は I_{ds} が増加するが、**短チャンネル効果**も発生

ロジックゲートは通常、高集積密度、低 V_{th} 、高 I_{ds} のため、**チャンネル長(L)は最短**で設計される。

短チャンネル効果やちゃんとした設計手法についてはオペアンプ編にて詳しく解説します。

設計方針

- PMOSの性能を上げる
 - PMOSのWを大きくする
 - PMOSのLを小さくする
- NMOSの性能を下げる
 - NMOSのWを小さくする
 - NMOSのLを大きくする



計算による設計手法についてはオペアンプ編にて詳しく解説します。

アナログ半導体の設計フロー

1. 回路図（テストベンチ）を描く
2. シミュレーションをする
- 3. 回路図を基にレイアウトを描く**
4. レイアウトを検証する
5. レイアウトを基に寄生成分を考慮したシミュレーションをする
6. フレームに載せる

半導体の基礎知識

プレーナ型FETプロセス レイアウト編

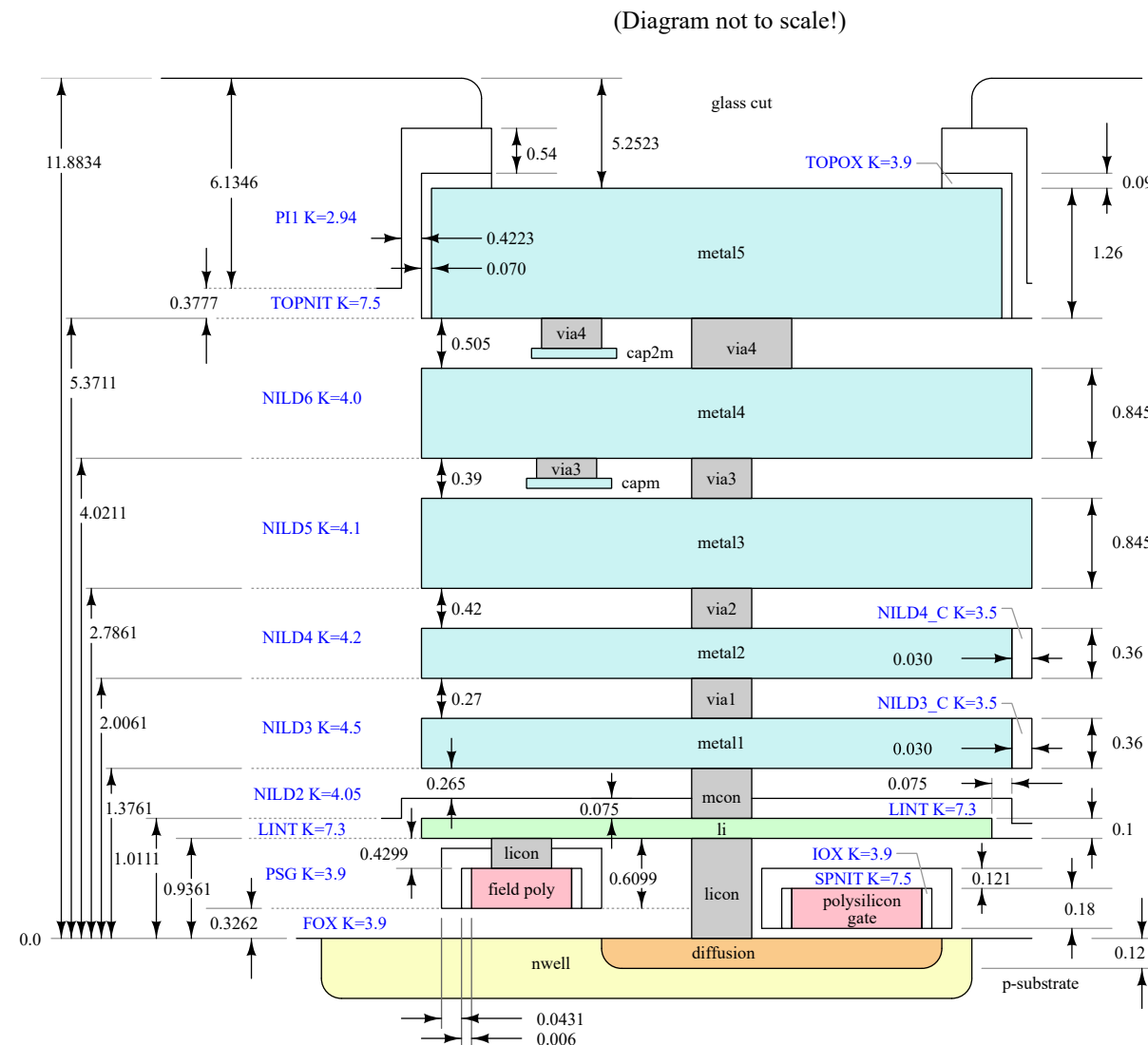
プレーナ（平面）型MOSトランジスタ（SKY130の例）

substrate（基板）を最下層として積層していく。

基板上にMOSトランジスタ(well, diffusion, poly)

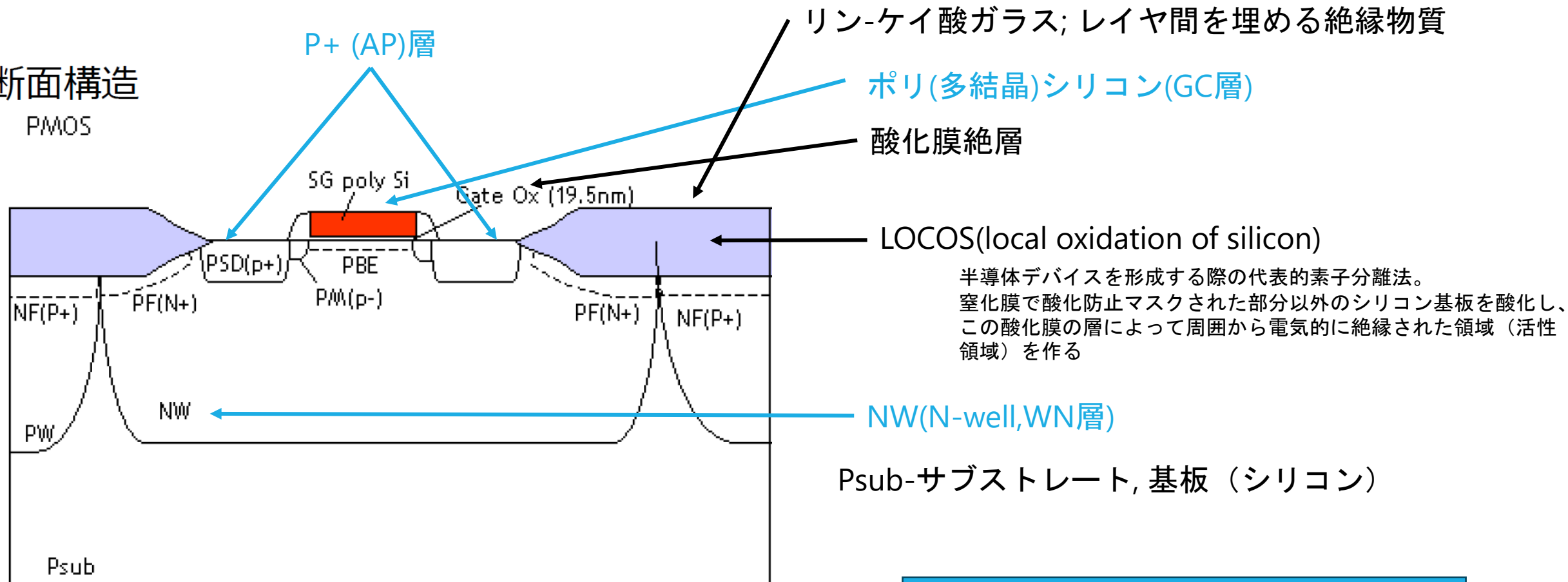
トランジスタより上は配線層(metal*, via*)

metal間に絶縁体(insulator)を挟んで
MIMキャパシタが作成可能(capm, cap2m)



プレーナ型PMOS

断面構造
PMOS



青字はレイアウトする必要がある層

プレーナ型PMOS

WN

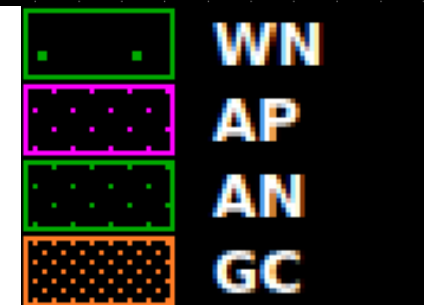
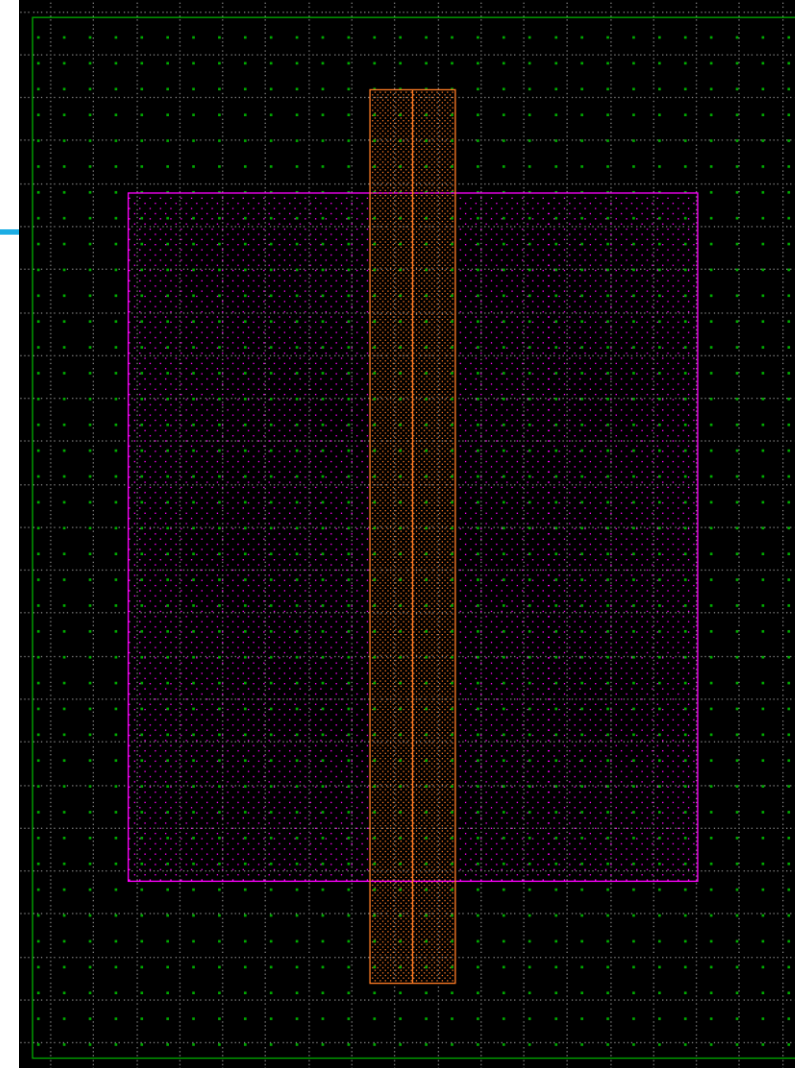
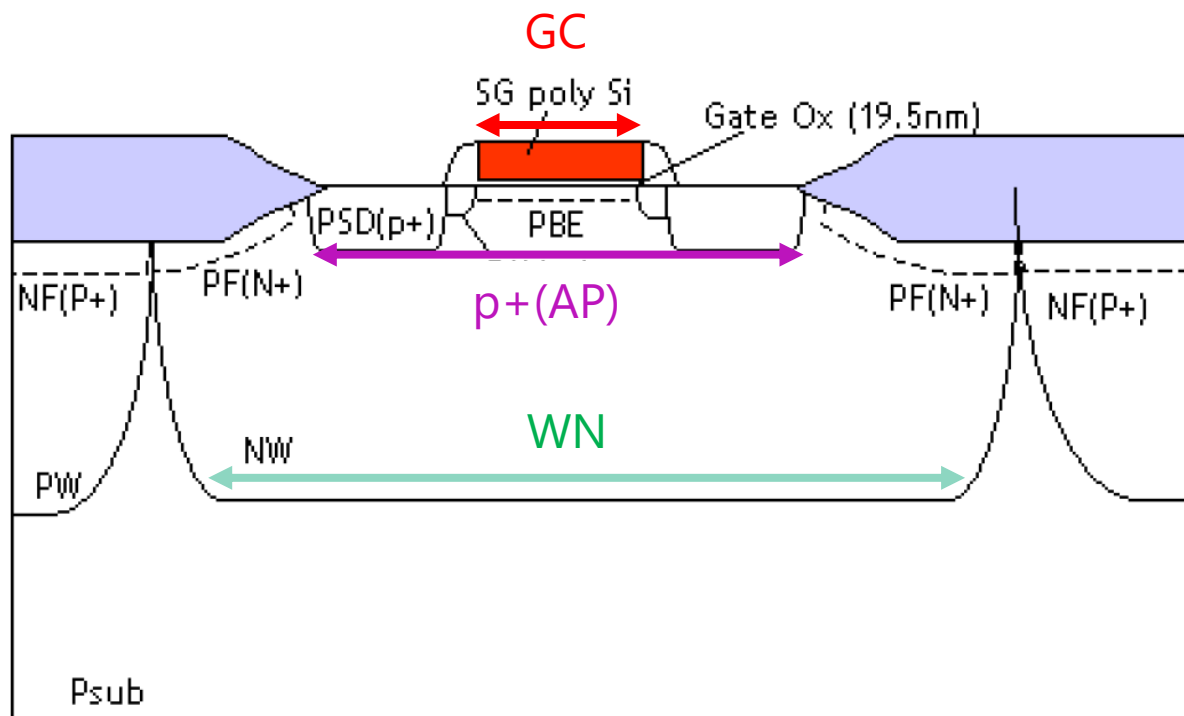
AP

GC

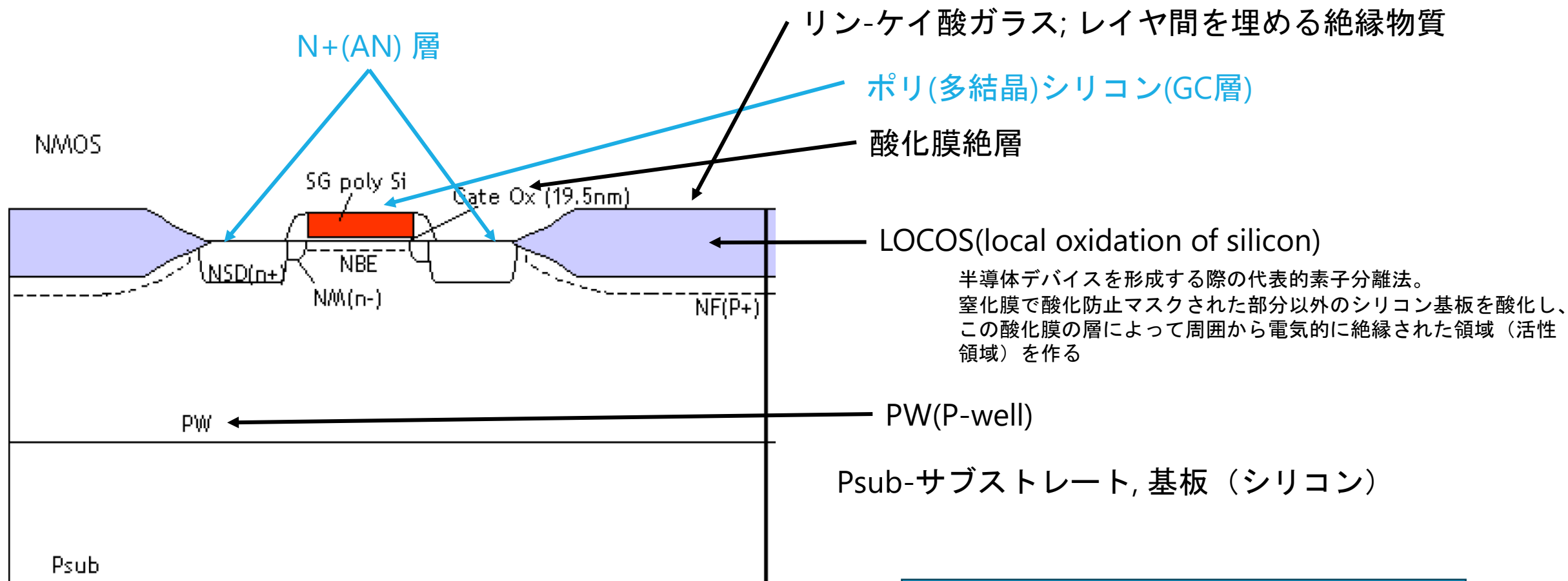
N-well

不純物注入によるP+イオン領域と拡散層

ポリシリコン



プレーナ型NMOS

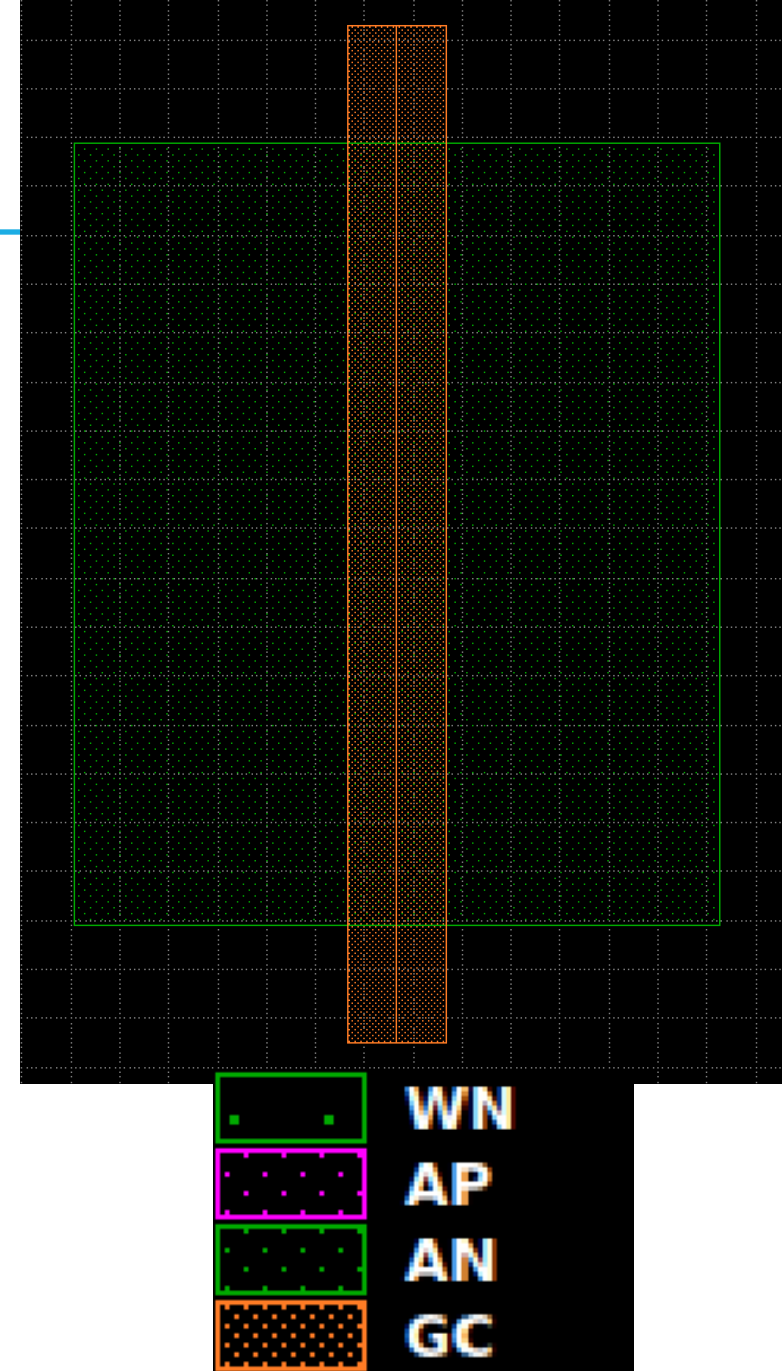
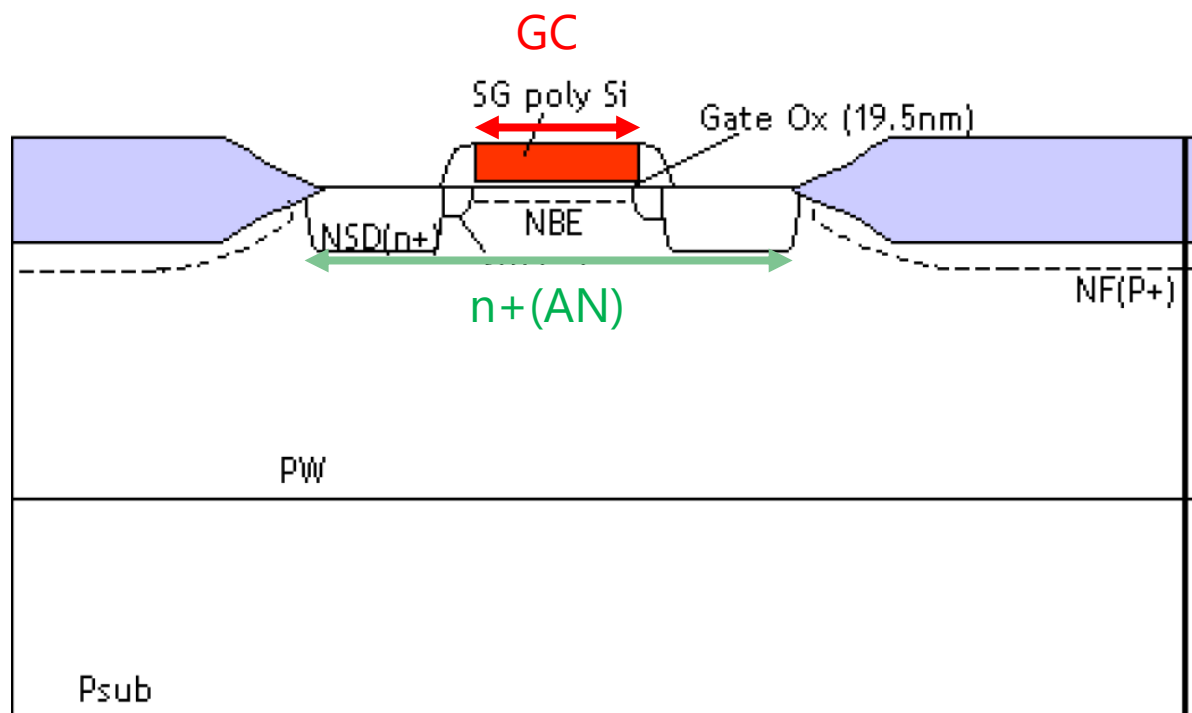


青字はレイアウトする必要がある層

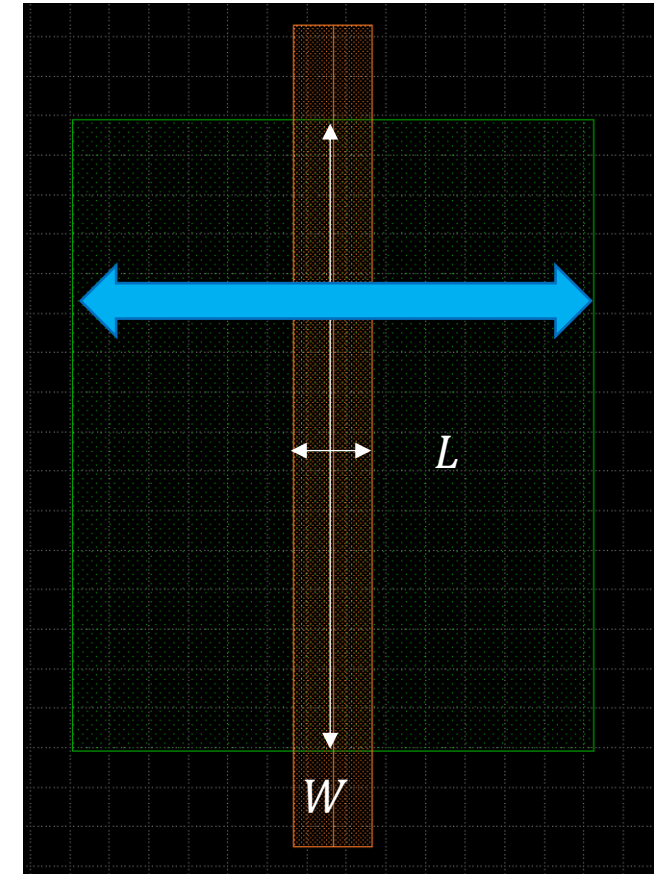
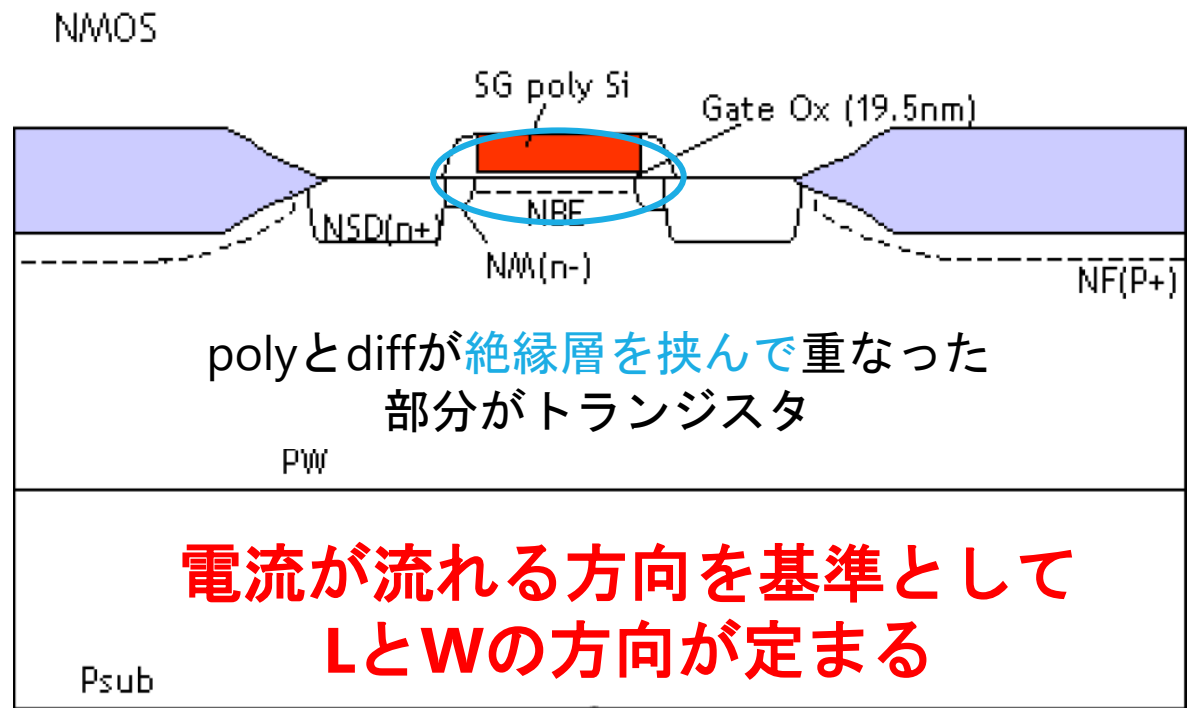
プレーナ型NMOS

AN 不純物注入によるN+イオン領域と拡散層
GC ポリシリコン

レイアウトでは、PW(P-well)は描かれない



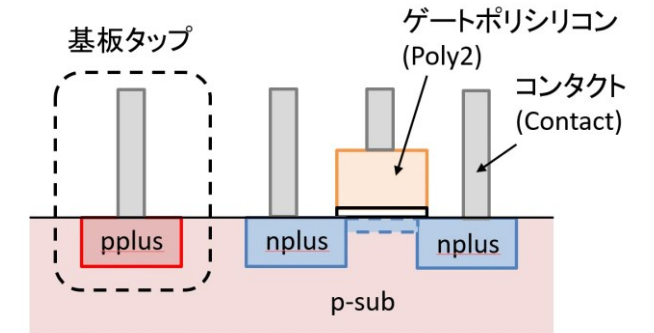
プレーナ型NMOS



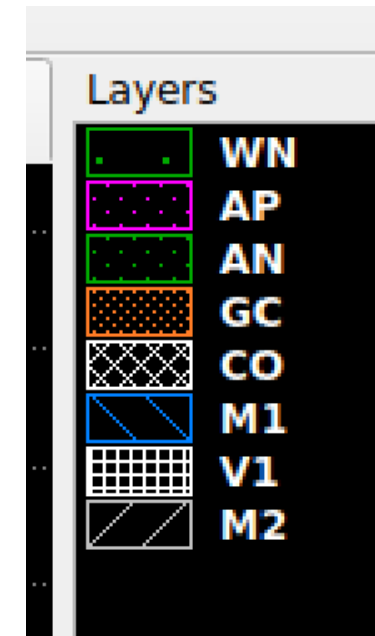
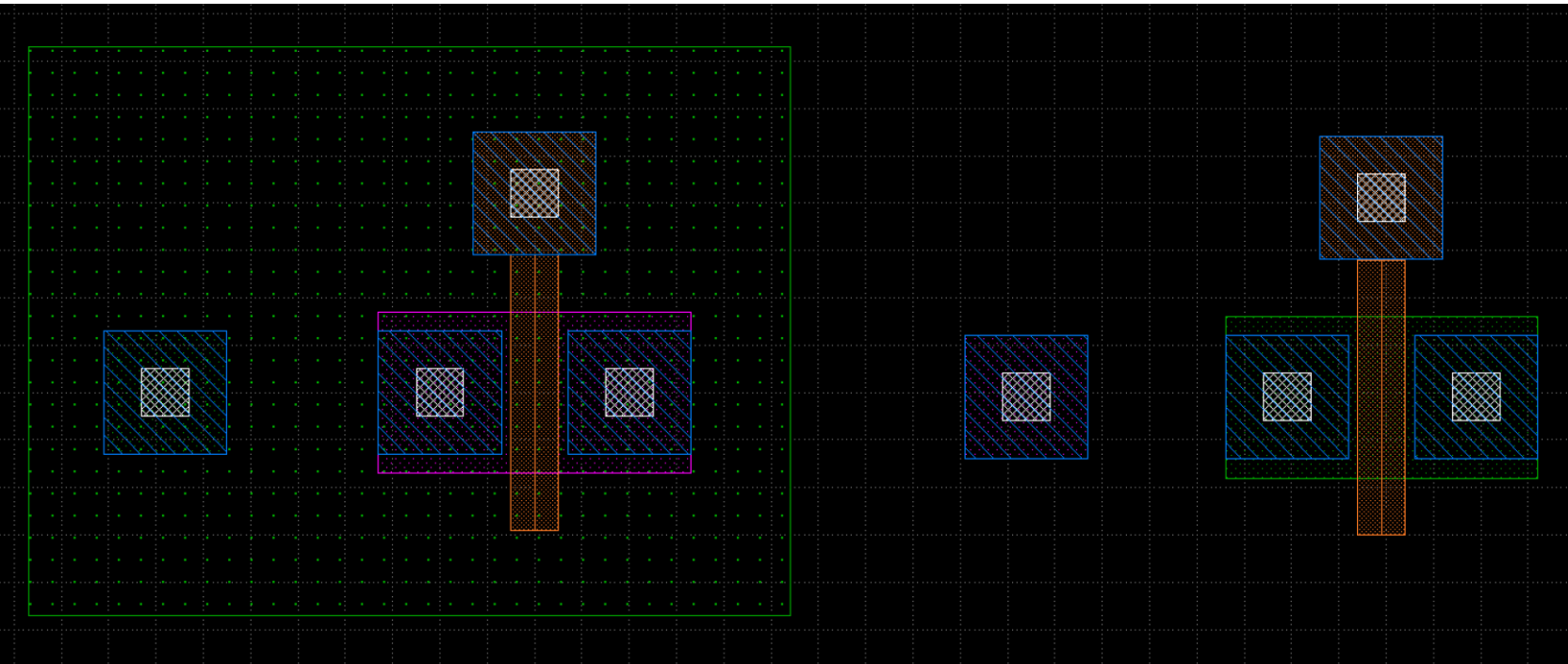
PCELLでのボディの扱い

PMOSはN+を介してN-well に繋がっている

NMOSはP+を介してP-substrateに繋がっている

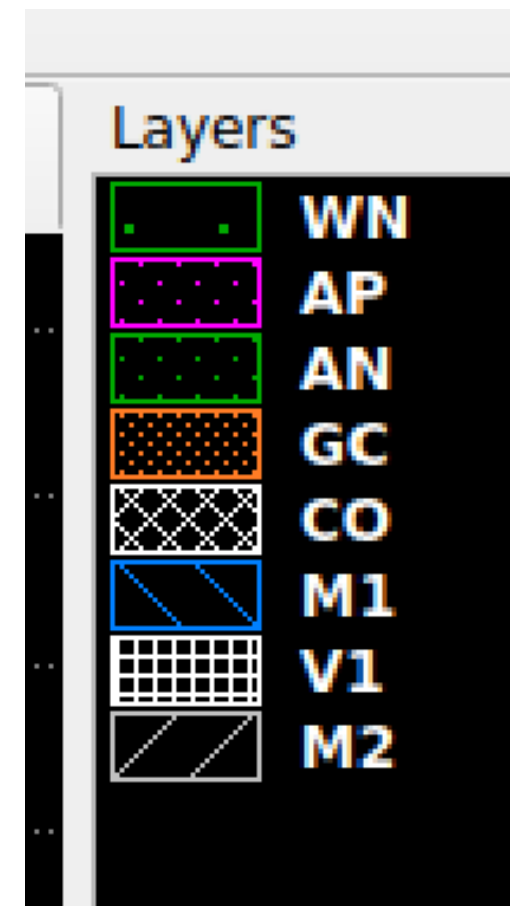


https://note.com/akira_tsuchiya/n/nd4444304f013



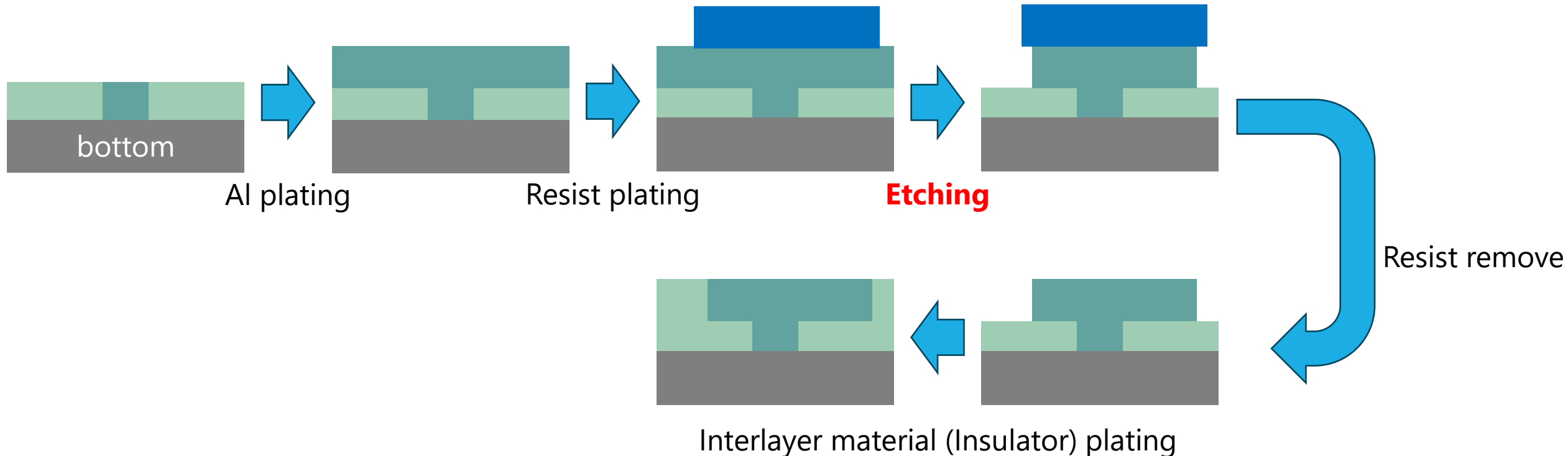
レイヤーの名前と意味

WN	N-well
AP	不純物注入によるP+イオン領域と拡散層の複合レイヤー
AN	不純物注入によるN+イオン領域と拡散層の複合レイヤー
GC	Polysilicon
CO	GCとM1接続用のVIA（コンタクトと呼ぶことが多い）
M1	メタル1層目
V1	M1,M2接続用のVIA
M2	メタル1層目



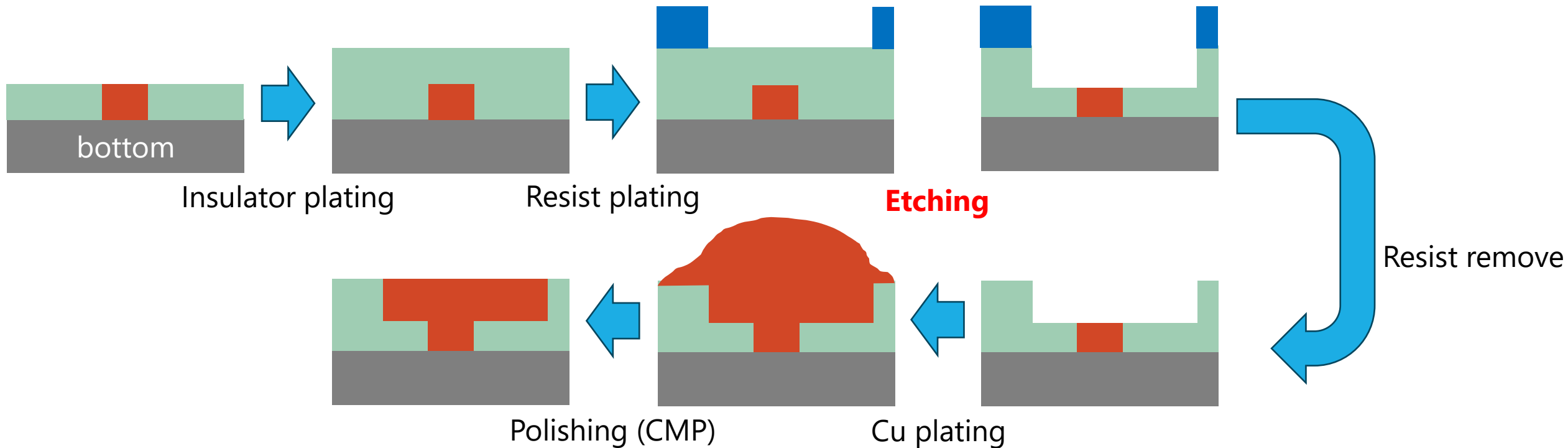
配線層ができるまで

- AlとCuでデザインルールが異なるのは、（一般論として）作り方が違うため。
- **サブトラクティブプロセス：金属めっきにレジストを堆積してエッチング（Al配線層）**
- **ダマシンプロセス：絶縁物で型を作って金属を流し込む（Cu配線層）**



配線層ができるまで

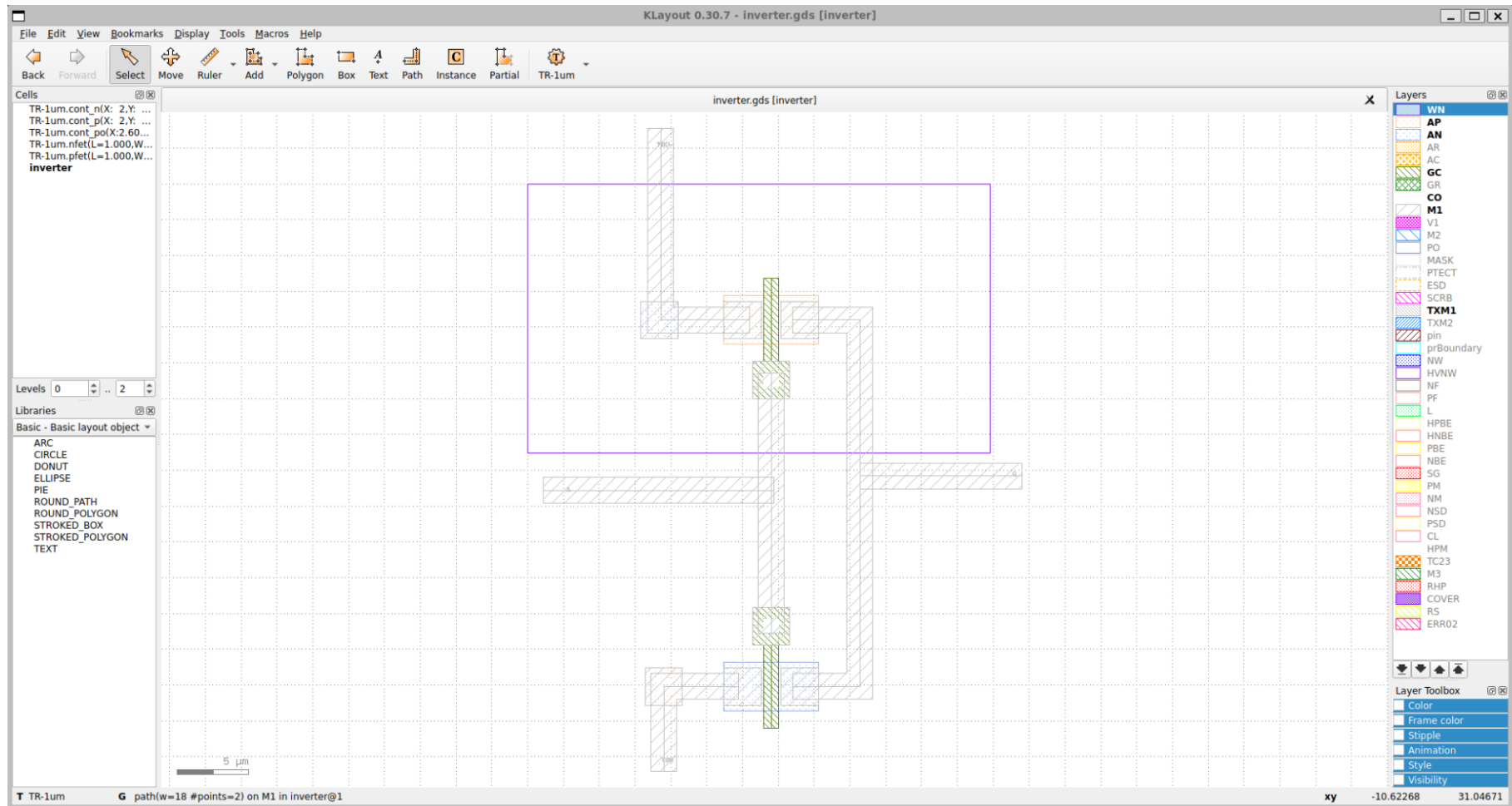
- AlとCuでデザインルールが異なるのは、（一般論として）作り方が違うため。
- サブトラクティブプロセス：金属めっきにレジストを堆積してエッチング（Al配線層）
- ダマシンプロセス：絶縁物で型を作って金属を流し込む（Cu配線層）



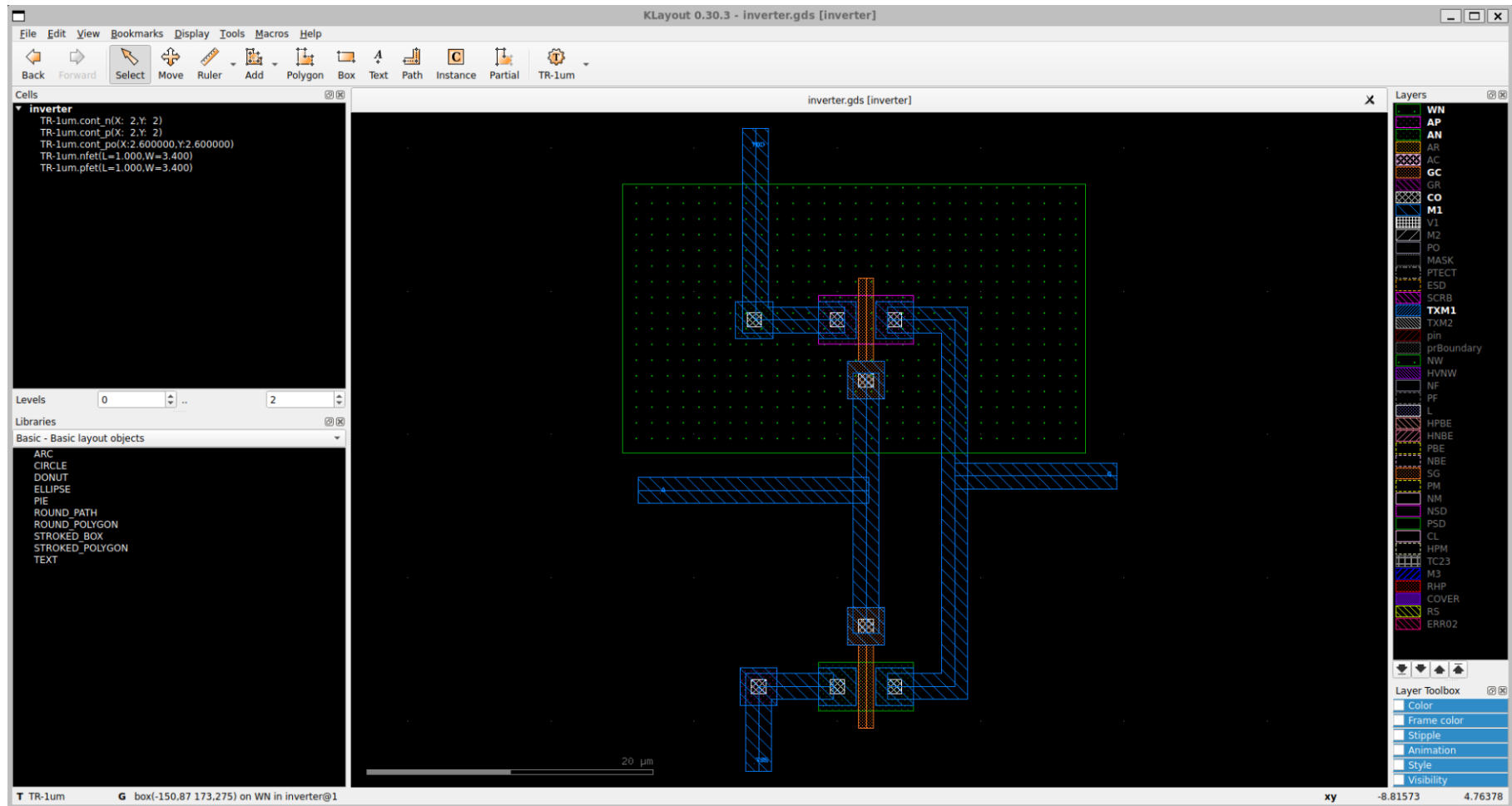
アナログ半導体の設計フロー

1. 回路図を描く
2. シミュレーションをする
- 3. 回路図を基にレイアウトを描く**
4. レイアウトを検証する
5. レイアウトを基に寄生成分を考慮したシミュレーションをする
6. フレームに載せる

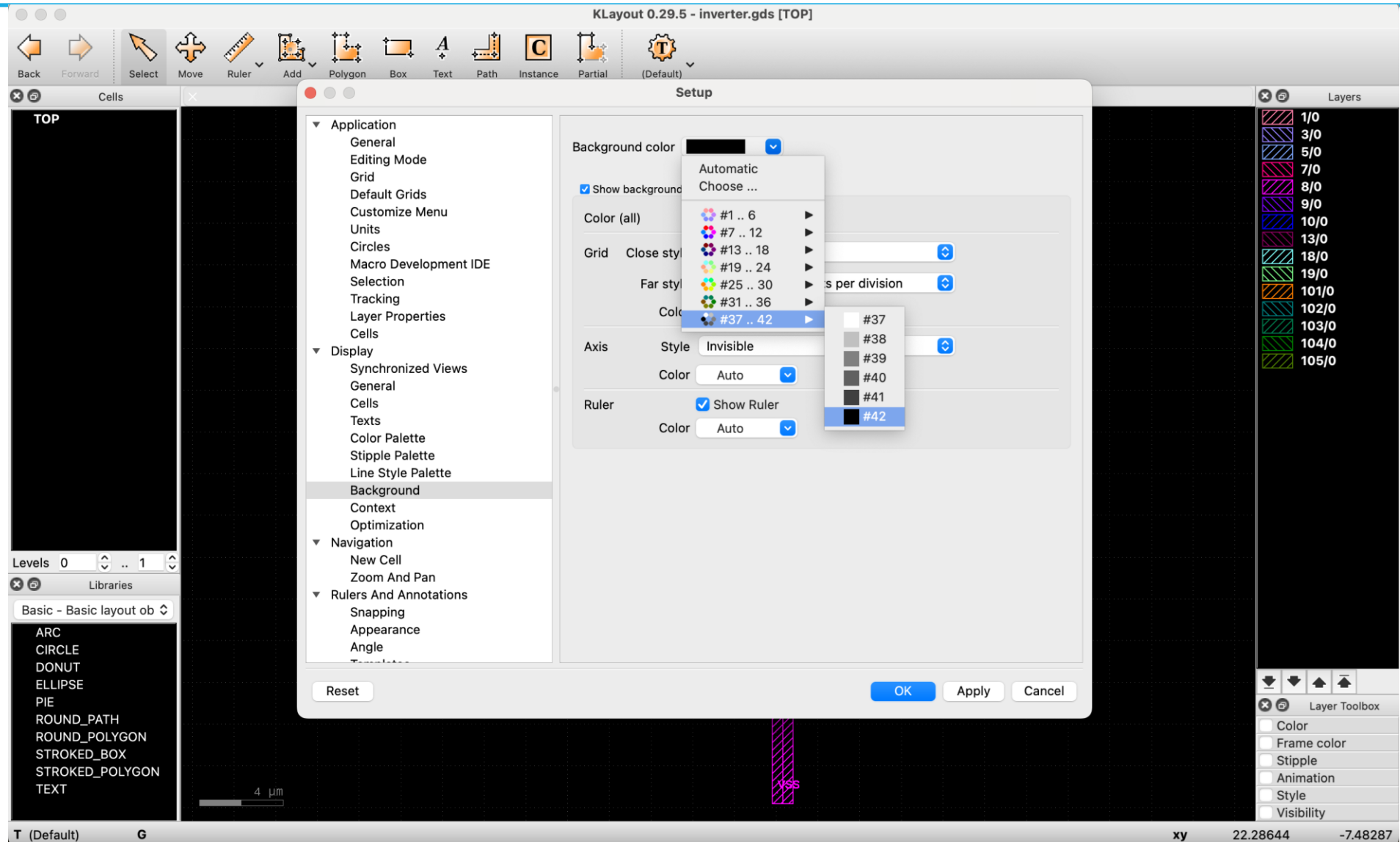
Klayout : 公式PDKのレイアウト画面



Klayout : バッググラウンドが「黒」のレイアウト画面

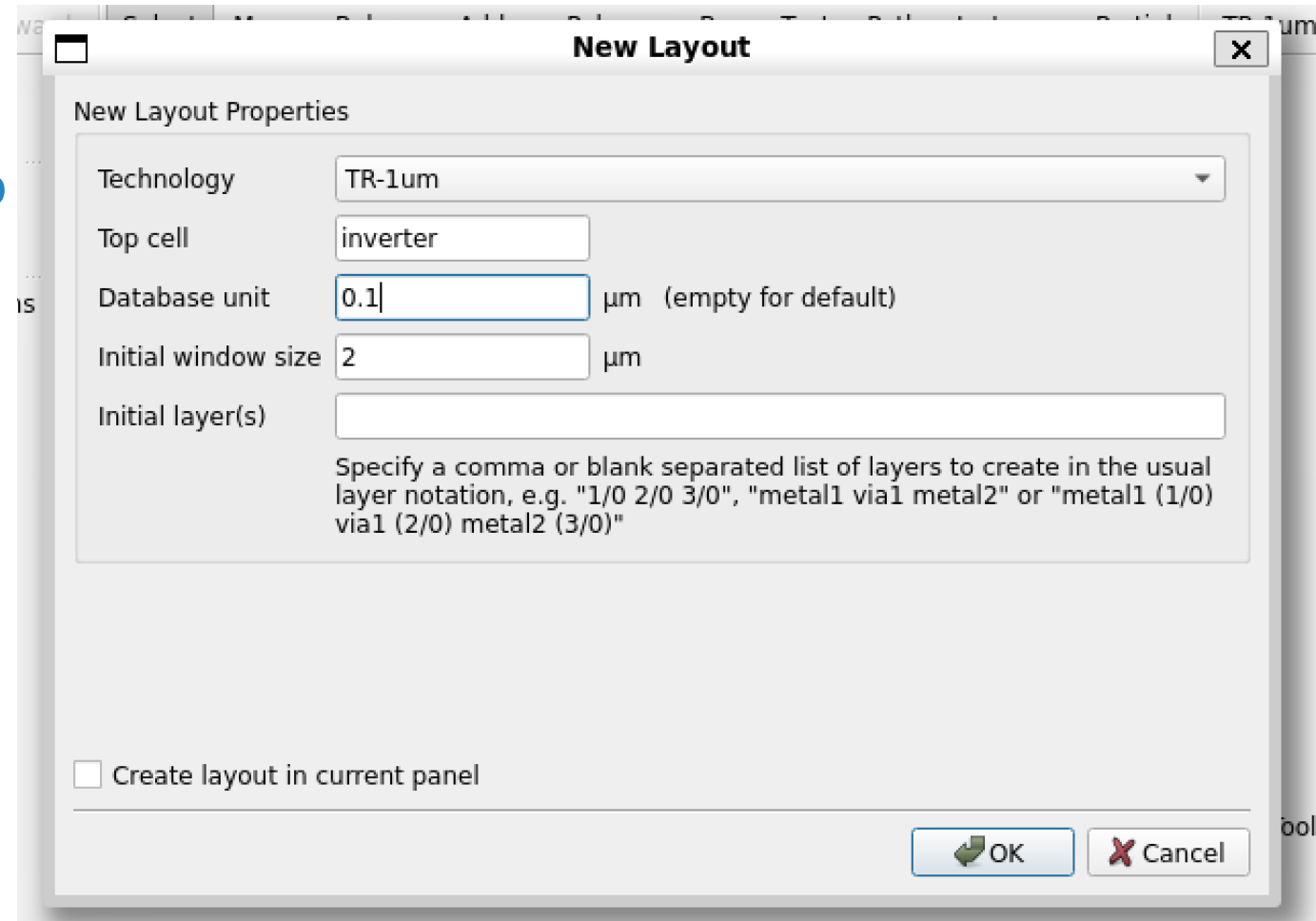


Klayout : バックグラウンドを「黒」にする



Klayout : レイアウトを描く上での注意

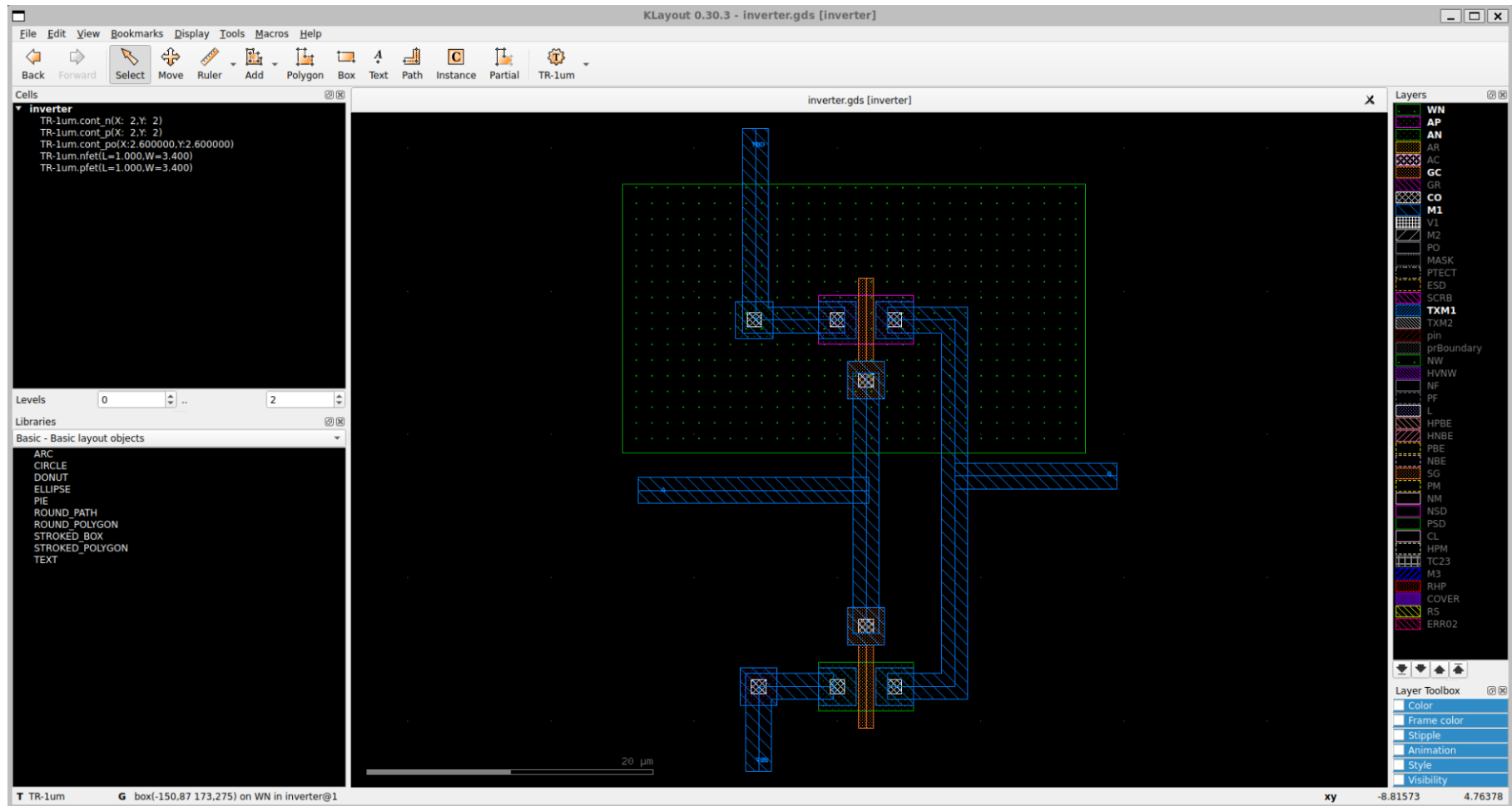
- Technologyを TR-1umにする
- Top cellを 回路図と同じ名前にする
- Database unit を 0.1にする



Klayout : デザインルールのある場所

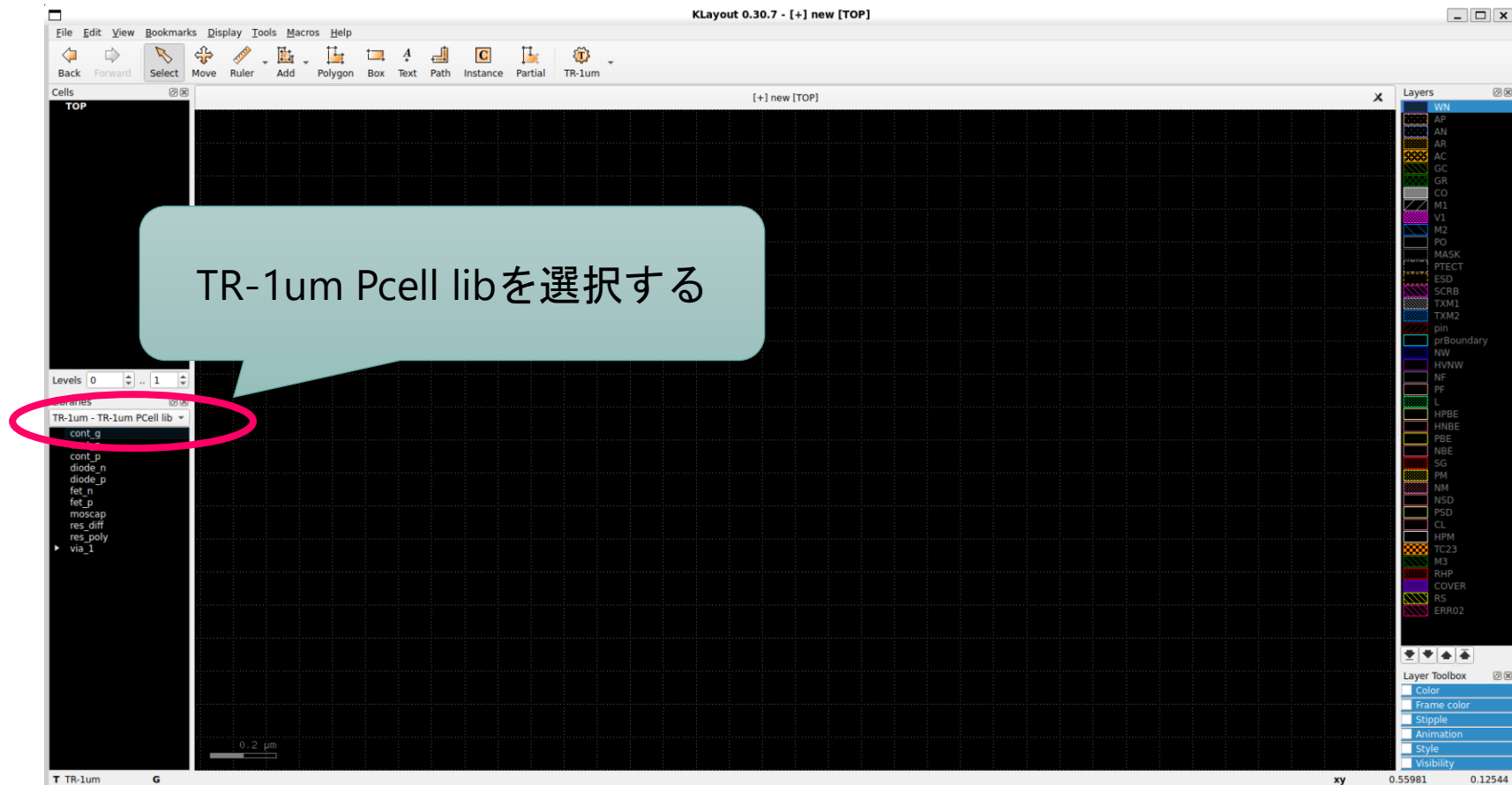
- ~/pdk/TR-1um/Document/
 - デザインルールマニュアル
 - TR-1um_DRC_summary.pdf
 - レイヤーマニュアル
 - TR-1um_Drawing_Layer_DR_Table.pdf
 - TR-1um_GDSII_Table.xlsx.pdf
- ~/pdk/TR-1um/openIP62/IP62/Technology/doc
 - 必要に応じて参照すること
 - OS04_ESD保護素子ガイドライン.pdf
 - OS05_スタンダードセルラインナップ.pdf
 - OS06_素子接続ガイドライン.pdf
- ~/pdk/TR-1um/openIP62/IP62/Tools
 - 抵抗・容量計算ツール(オープン版).xlsx
 - 抵抗やコンデンサを利用する場合に容量計算する

Klayout : 完成サンプル



Klayout : P-Cellの使い方

- P-Cell（パラメータ化セル）とは、EDAで使用する再利用可能で構成可能なブロックのことです。P-Cellは、集積回路のレイアウトパターンを特定の設計要件に応じて調整可能なパラメータで作成することができます。これにより、各プロジェクトごとにレイアウト全体を再設計する必要がなくなり、設計プロセスの効率と柔軟性が大幅に向上します。



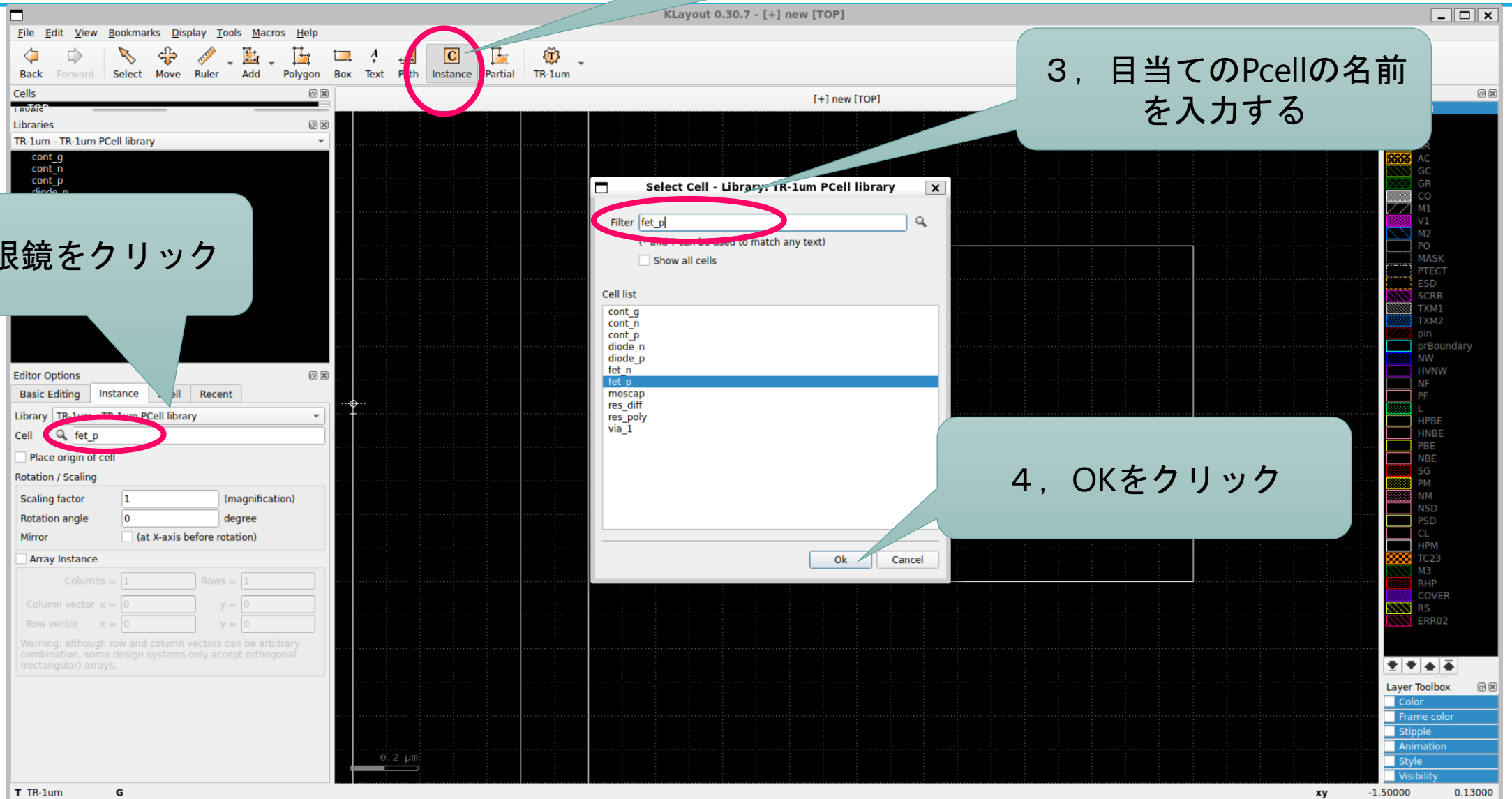
Klayout : P-Cellの使い方

1, Instanceをクリック

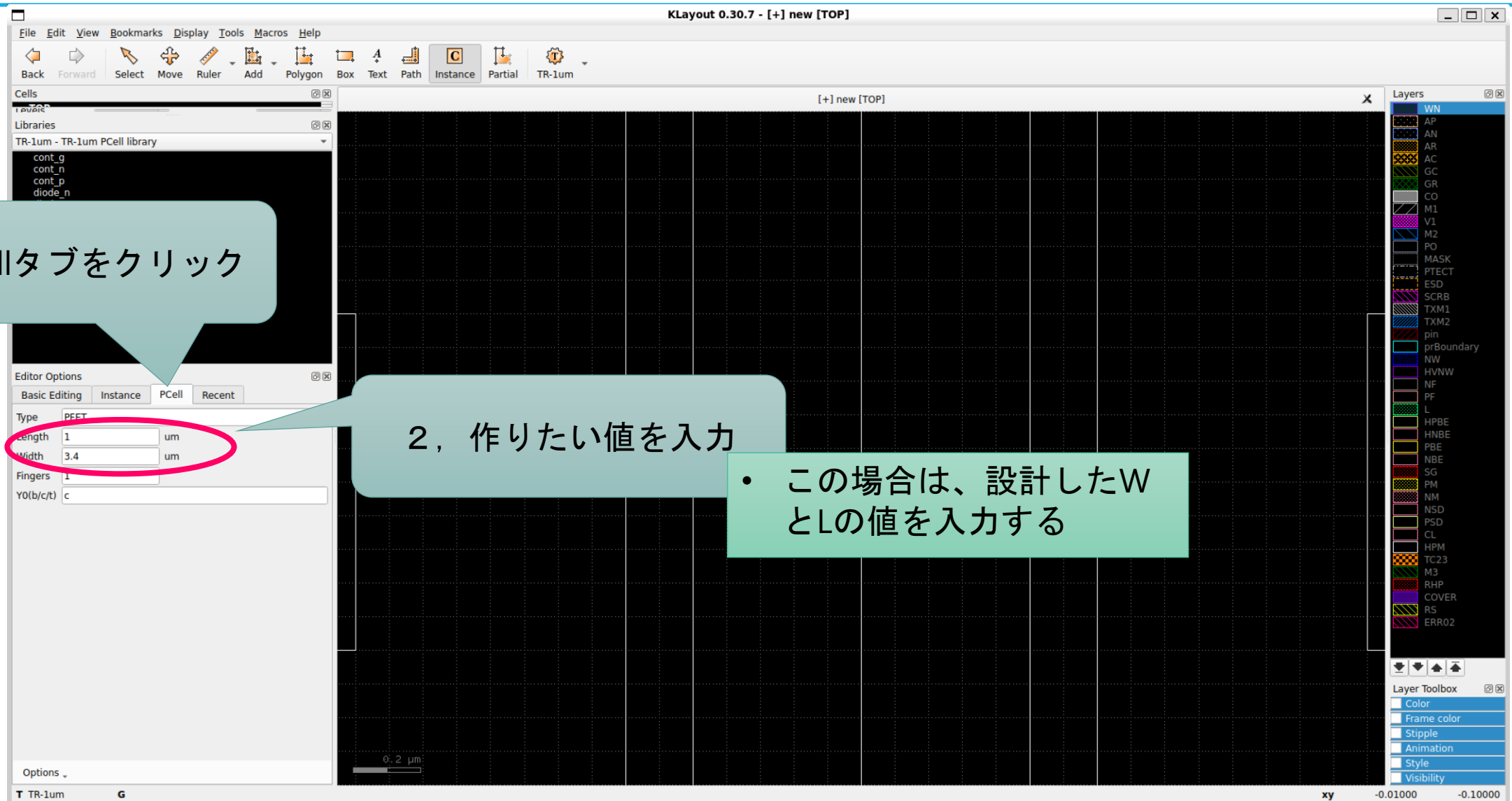
2, 虫眼鏡をクリック

3, 目当てのPcellの名前を入力する

4, OKをクリック



Klayout : P-Cellの使い方

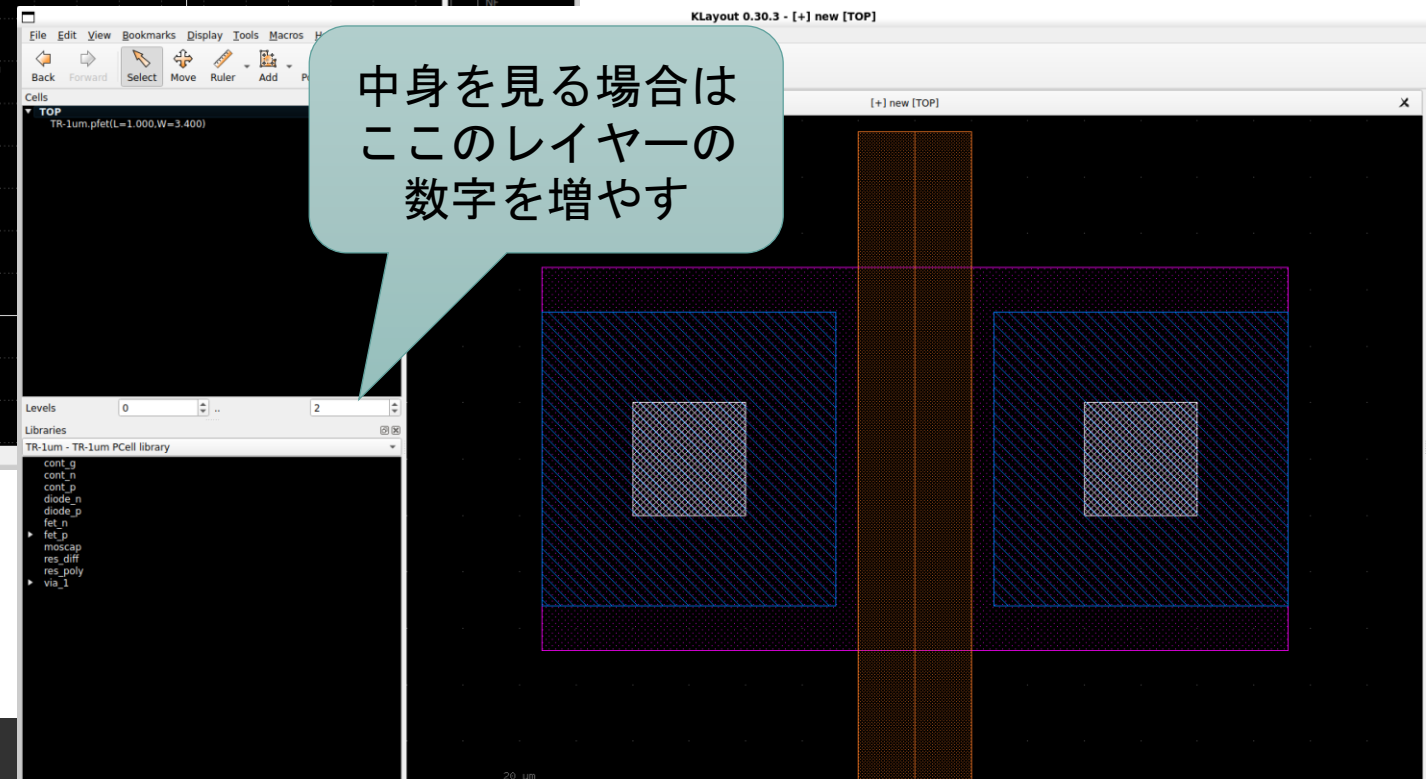
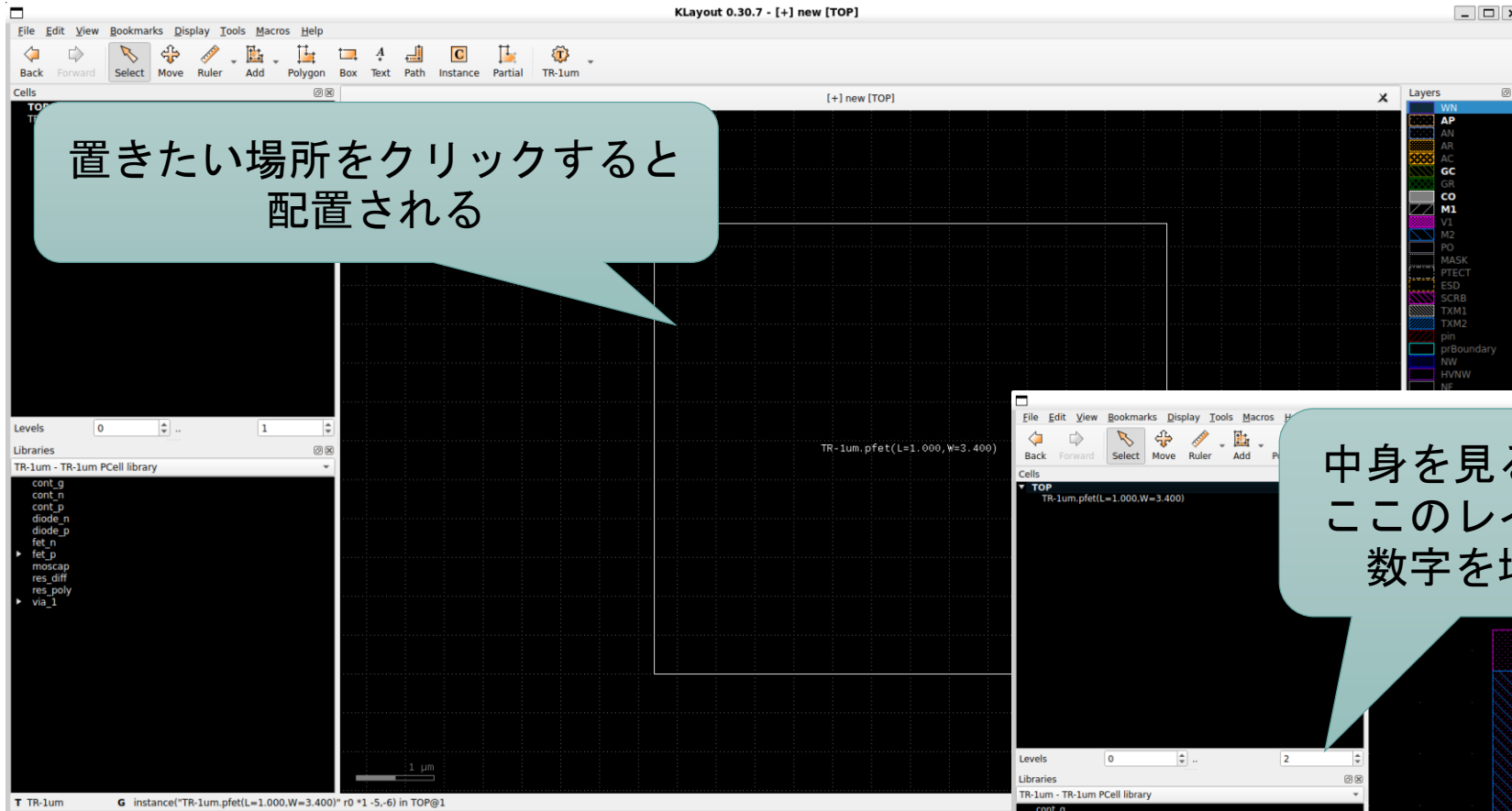


1, PCellタブをクリック

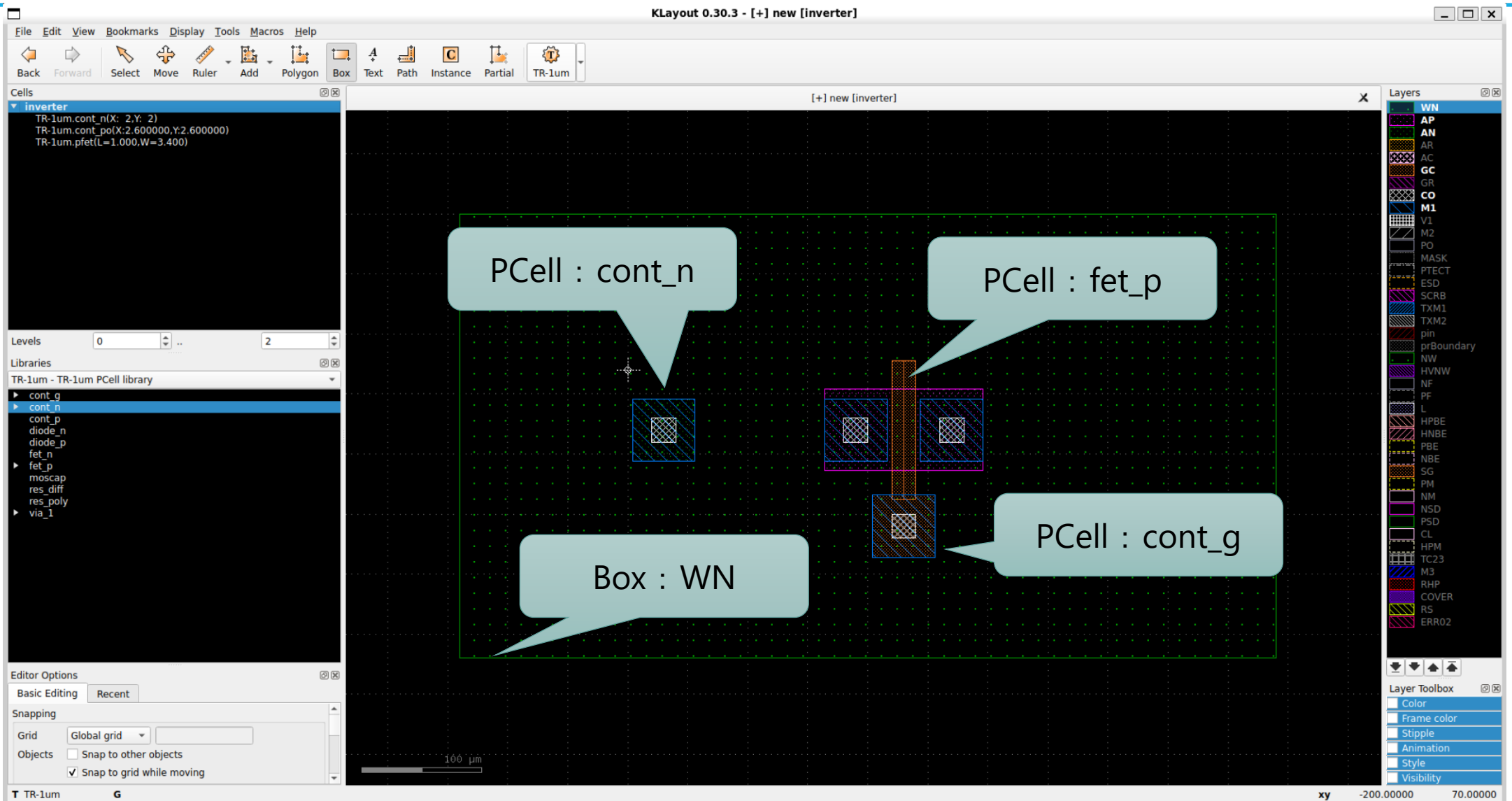
2, 作りたい値を入力

- この場合は、設計したWとLの値を入力する

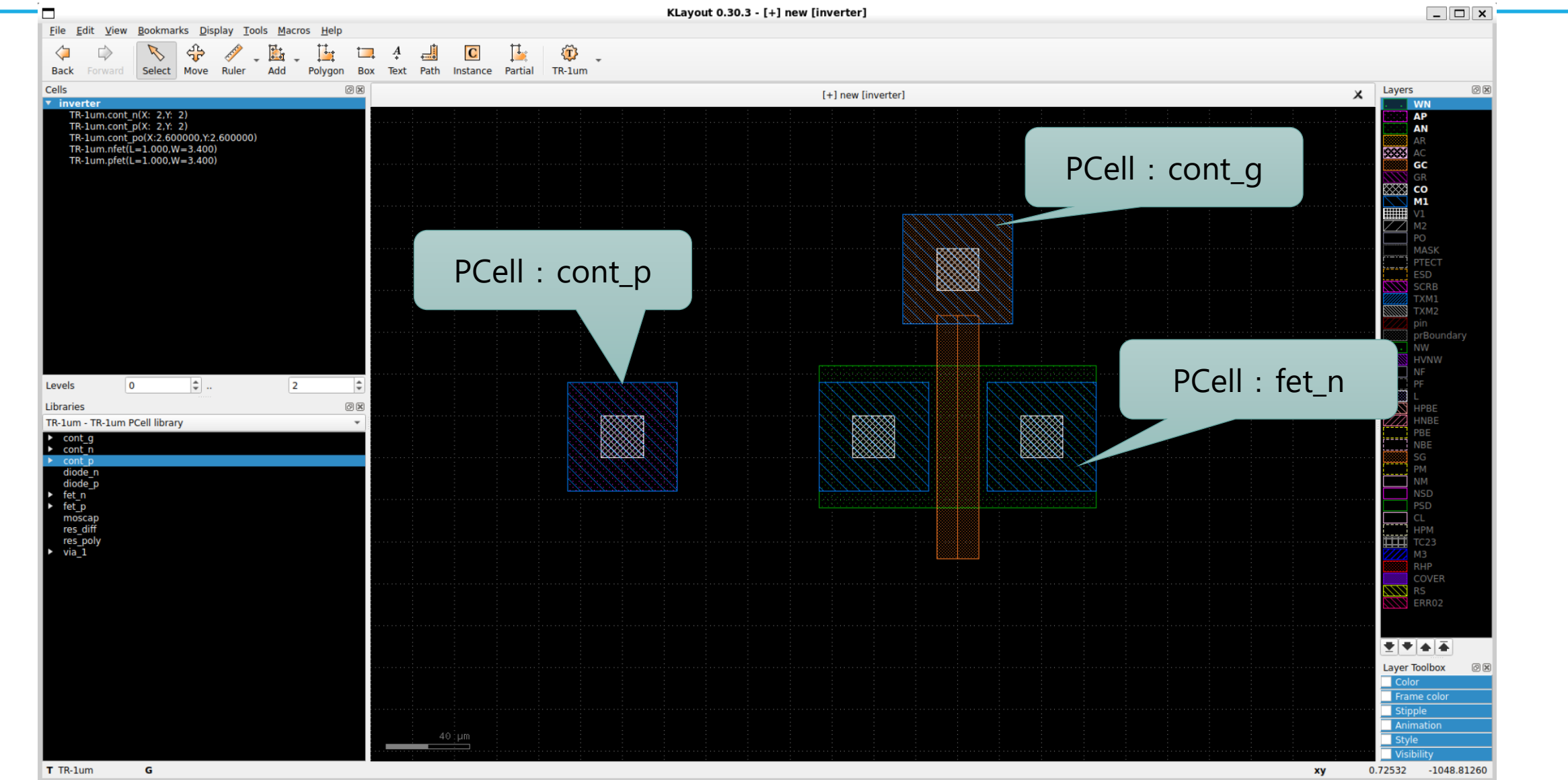
Klayout : P-Cellの使い方



P-FET

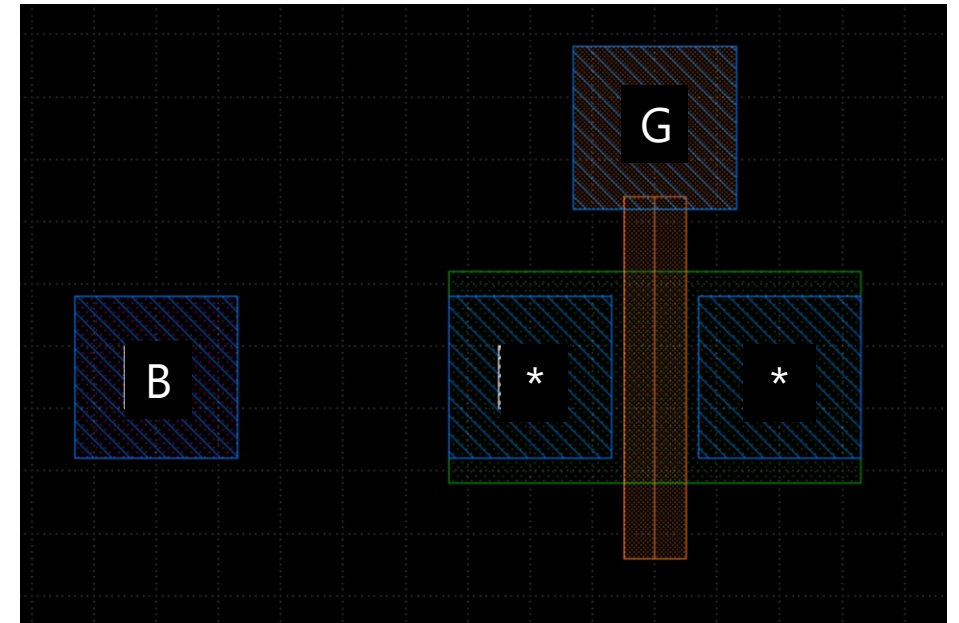
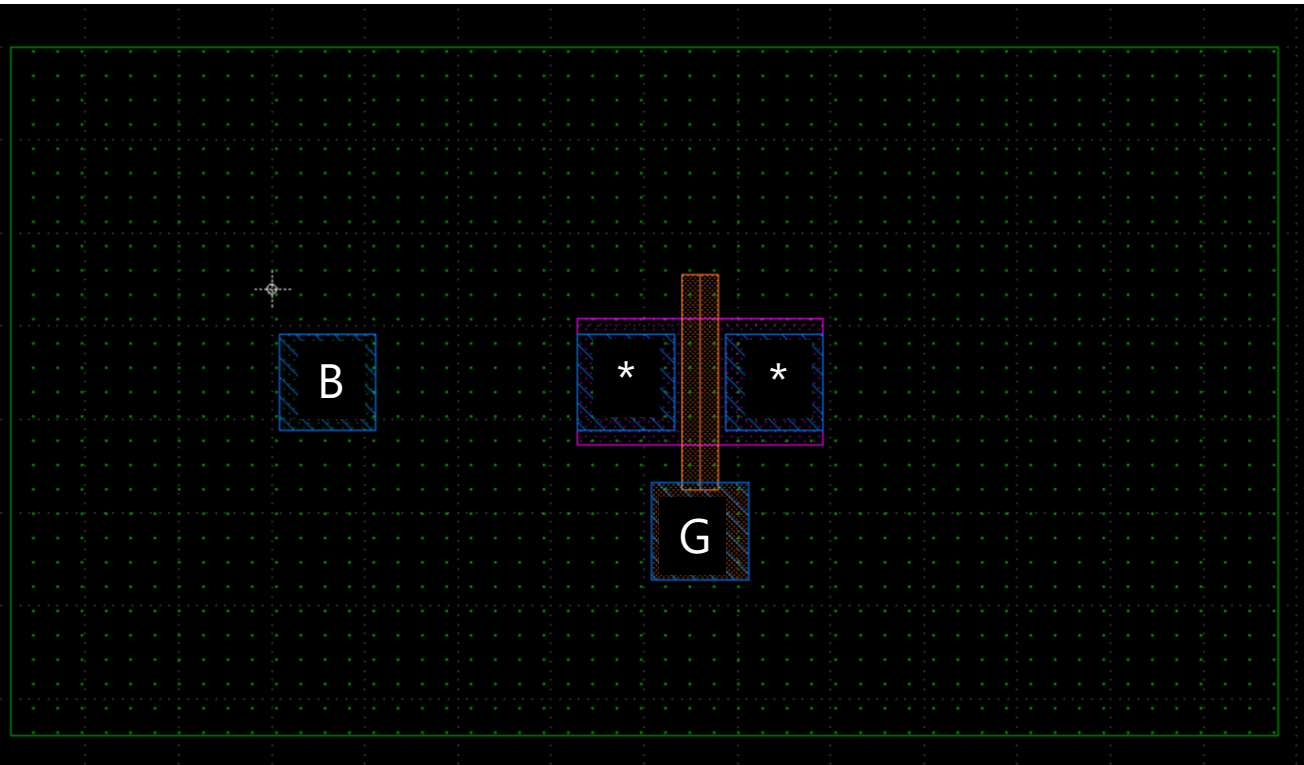


N-FET



Klayout : タップ付 PFET と NFET

- [*] どちらをドレインにしてどちらをソースにするかは任意。



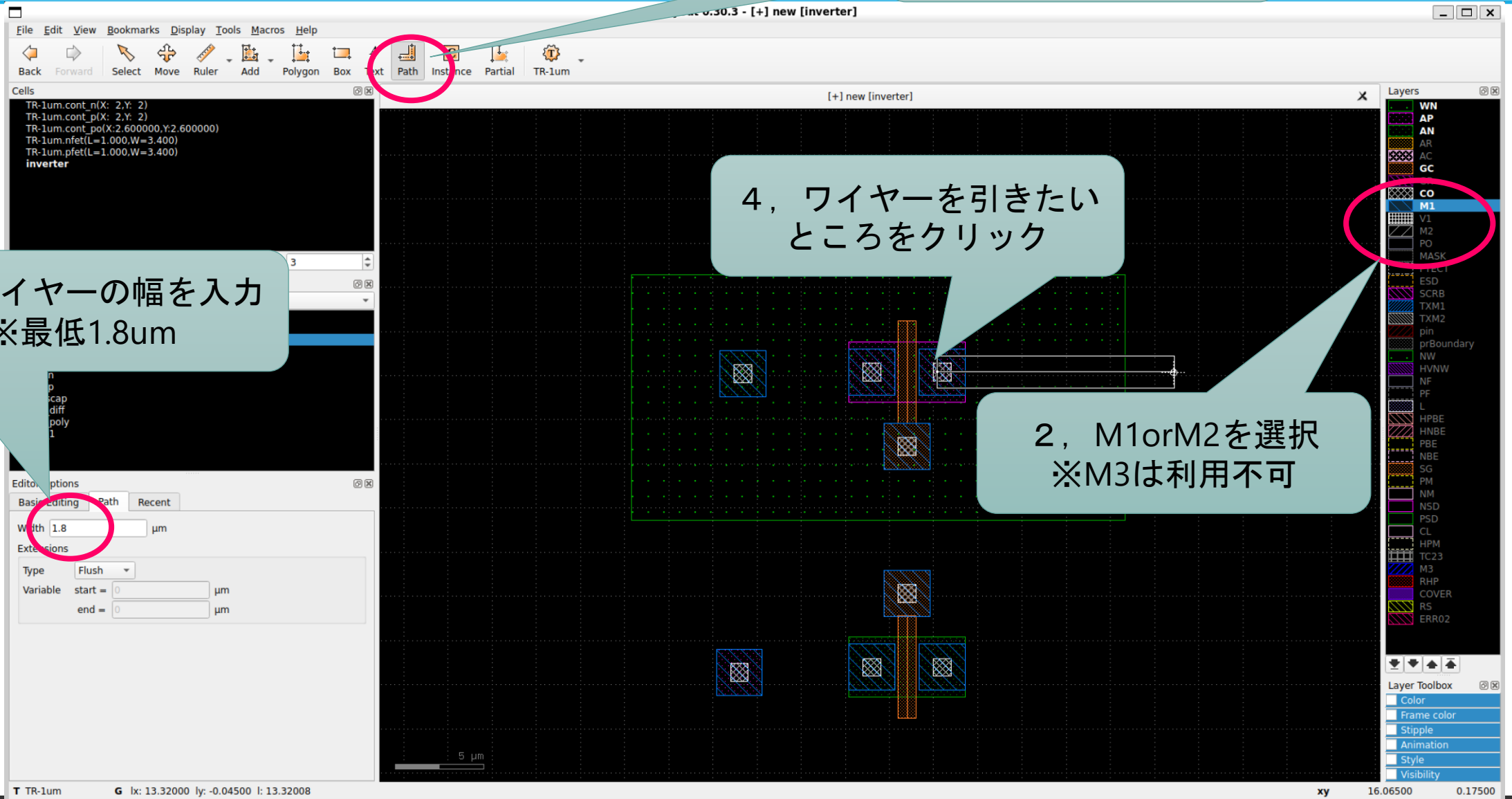
Klayout : ネットリスト (ワイヤー) の引き方

1, Pathをクリック

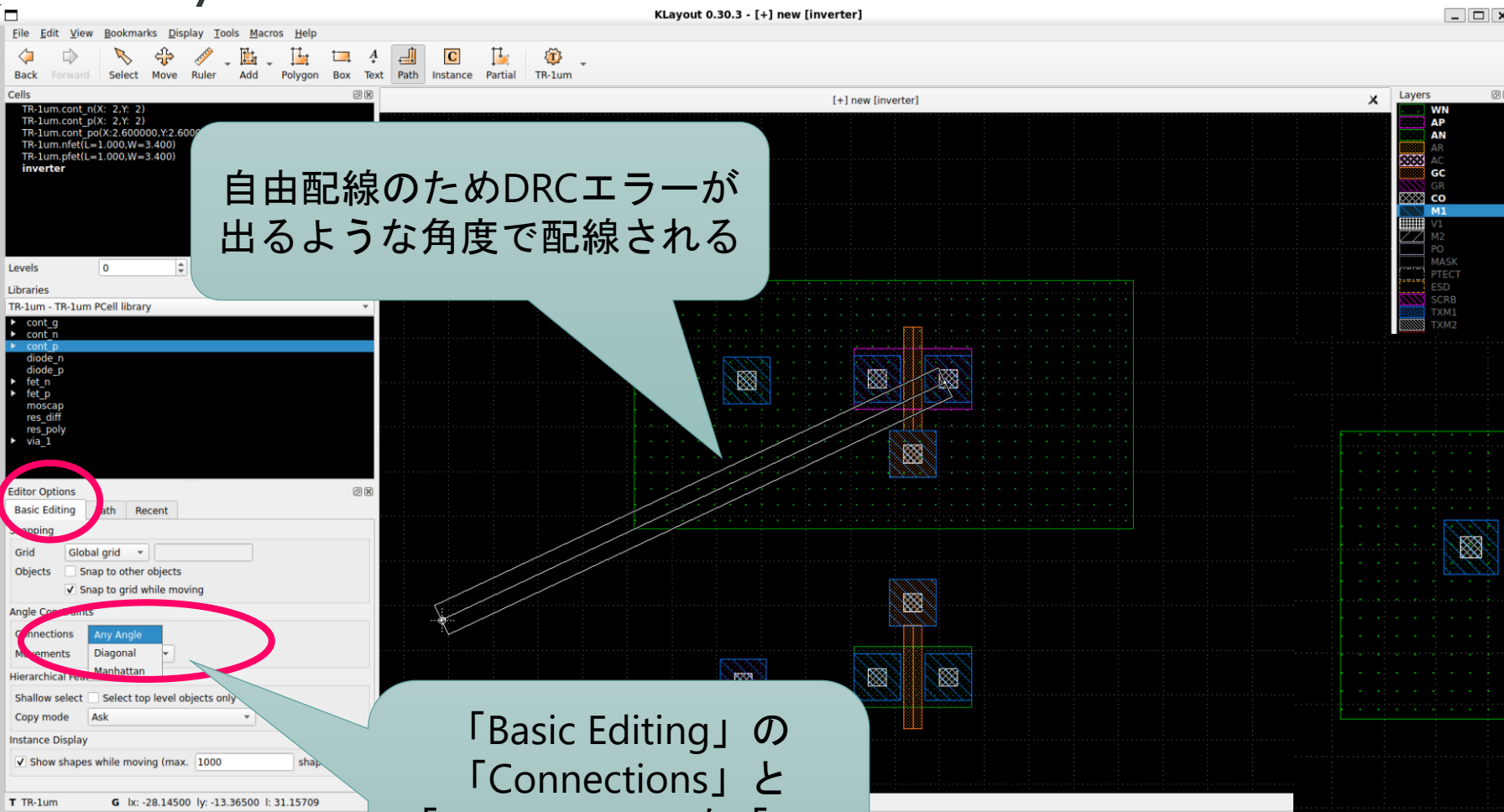
3, ワイヤーの幅を入力
※最低1.8um

4, ワイヤーを引きたい
ところをクリック

2, M1orM2を選択
※M3は利用不可



Klayout : ネットリスト (ワイヤー) の設定

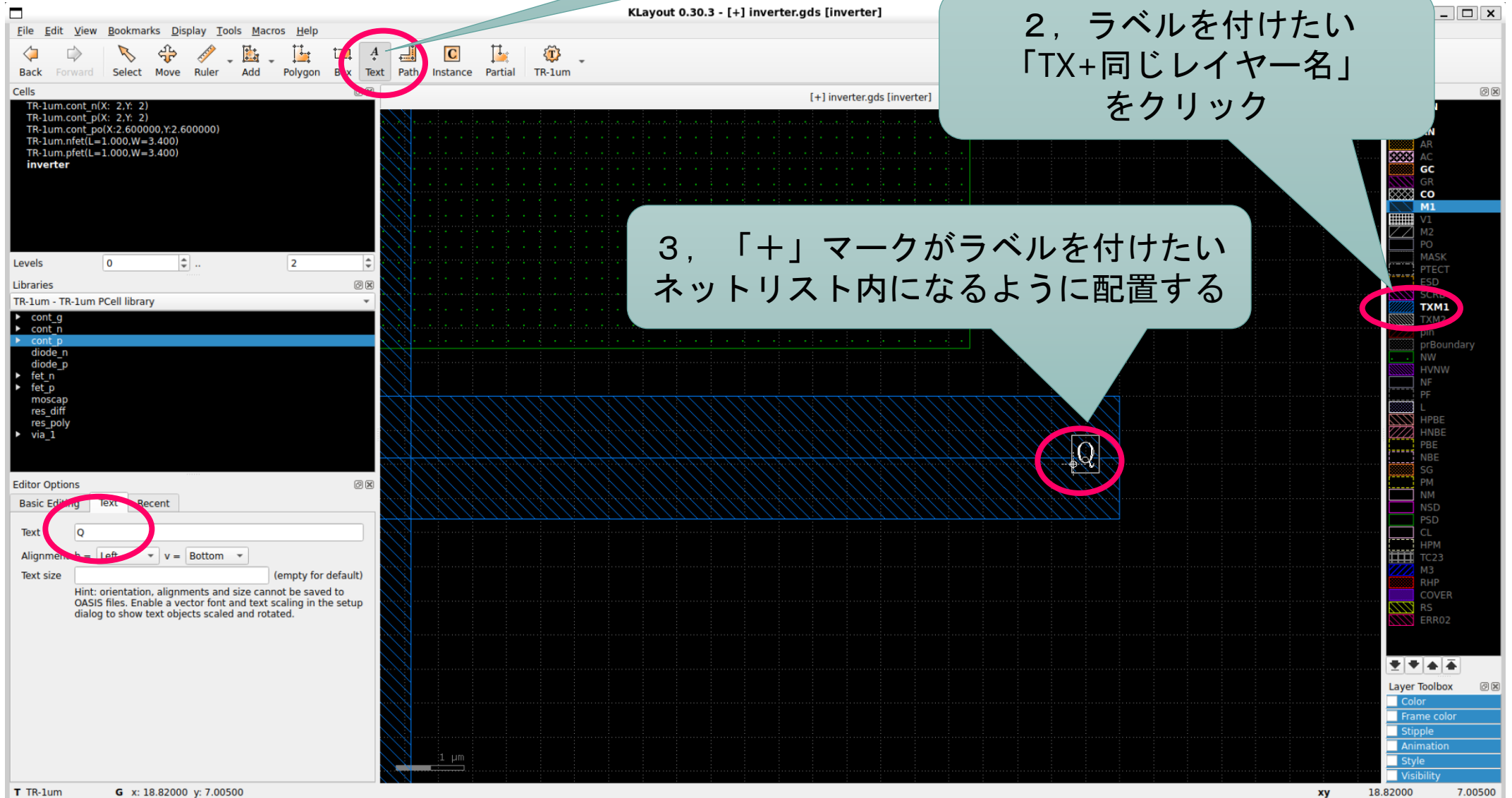


Klayout : ラベルの付け方

1, TEXTをクリック

2, ラベルを付けたい
「TX+同じレイヤー名」
をクリック

3, 「+」マークがラベルを付けたい
ネットリスト内になるように配置する



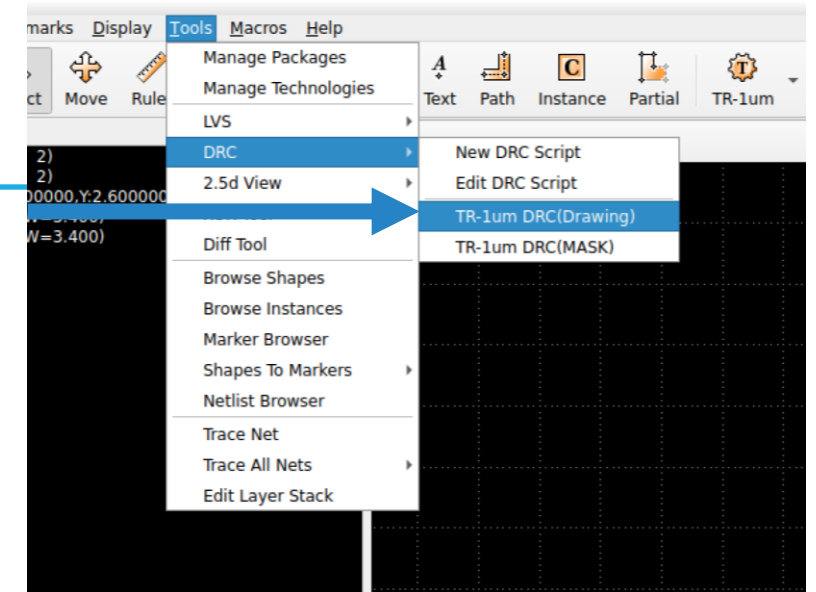
アナログ半導体の設計フロー

1. 回路図を描く
2. シミュレーションをする
3. 回路図を基にレイアウトを描く
- 4. レイアウトを検証する**
5. レイアウトを基に寄生成分を考慮したシミュレーションをする
6. フレームに載せる

Klayout : レイアウト検証

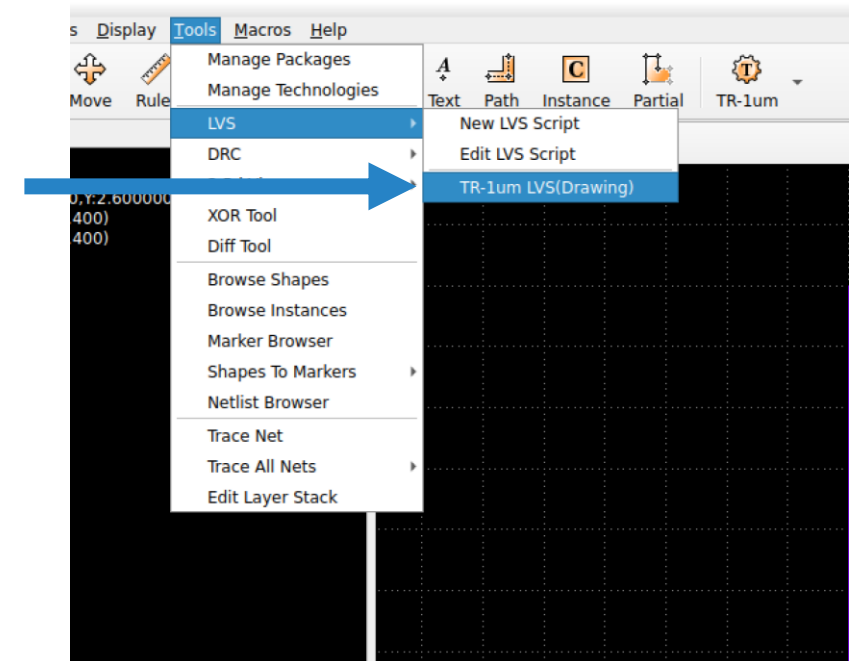
Design Rules Check (DRC)

- 指定されたデザインルールから違反していないか検証



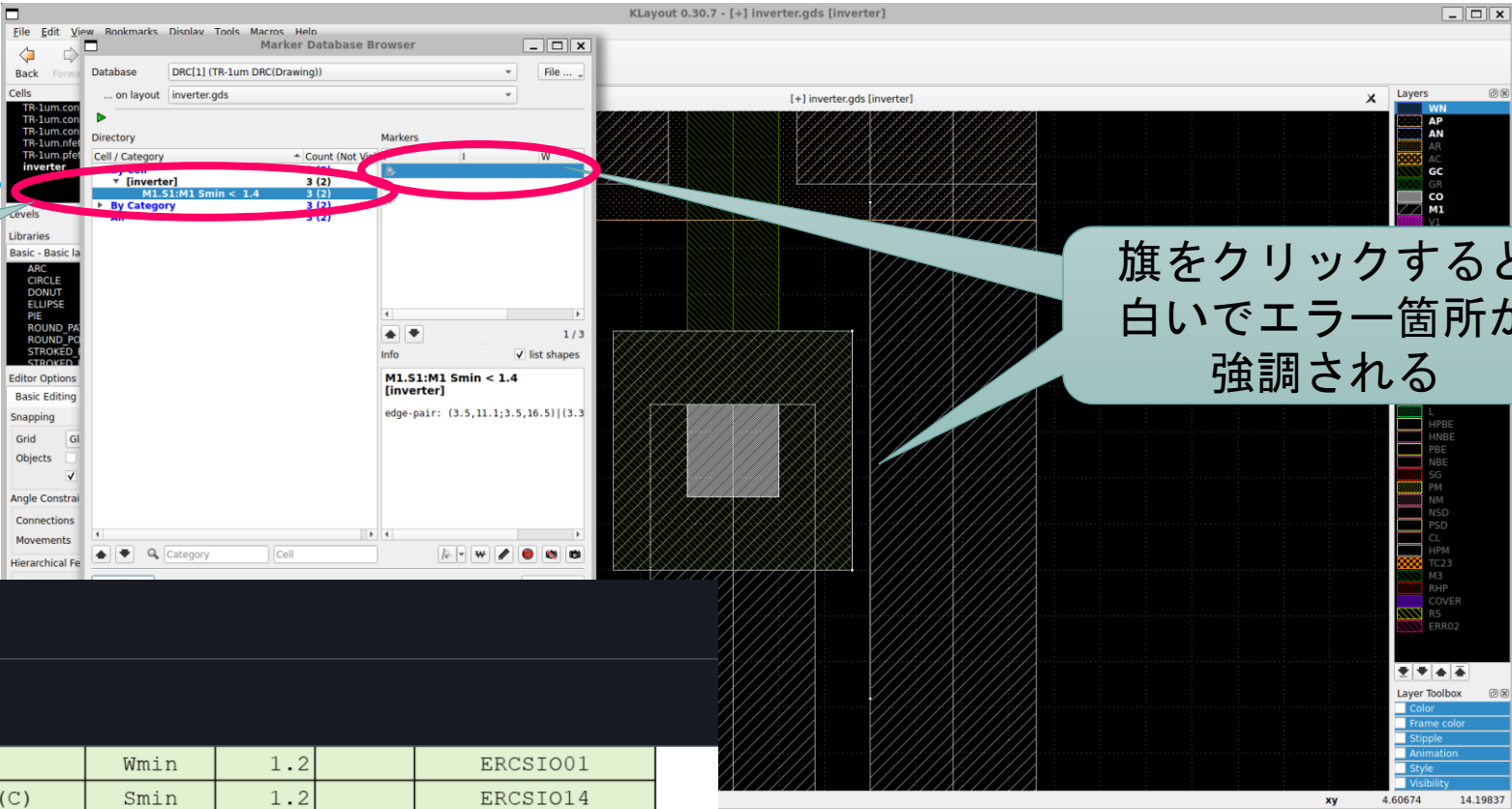
Layout Versus Schematic (LVS)

- レイアウトが回路図通りに描けているか検証



Klayout : DRCエラー

エラー番号をレファレンスマニュアルで検索するとエラー内容が解説されている



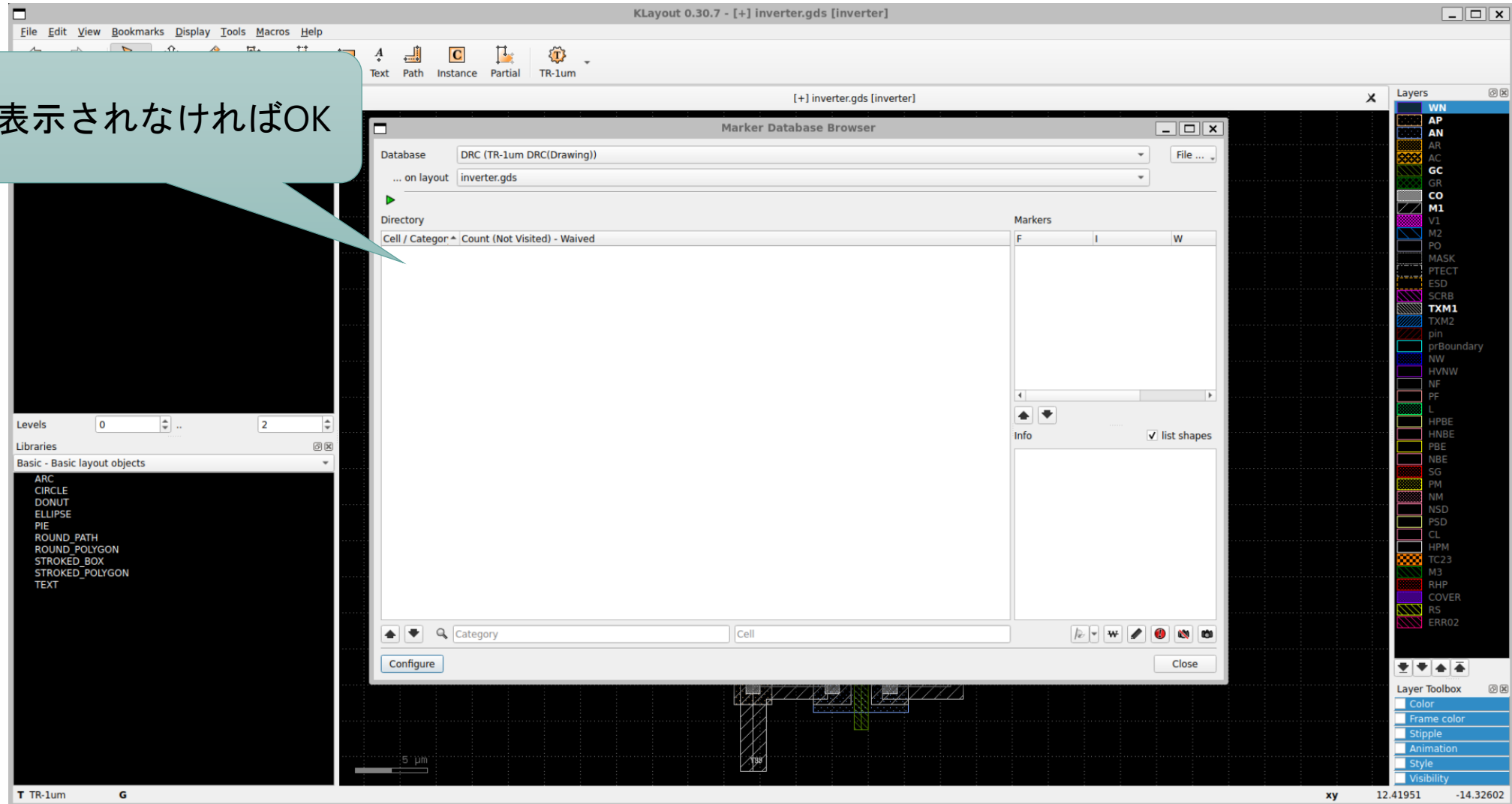
TR-1um / [Document](#) / TR-1um_Draw...r_DR_Table.pdf

150 KB

	CO (C)		Wmin	1.2		ERCSIO01
CO	CO (C)	CO (C)	Smin	1.2		ERCSIO14
CC	CO (C)	AC	Emin	2.9		ERCSIO03
CC..	CO (C)	AN	Emin	1.2		ERCSIO02
#	BEOL related					
Rule	L1	L2	Func	MIN	MAX	IP62 Rule Name
M1.W1	M1		Wmin	1.8		ER0021
M1.S1	M1	M1	Smin	1.4		ER0045
M1.SC	M1 (C)	M1 (C)	Smin	10.0		ERCSIO05
M1.CO	M1	CO (S)	Fmin	0.8		ER0051
M1.CC	M1	CO (C)	Fmin	1.2		ERCSIO02
M1.CL	M1	CO (L)	Fmin	1.2		ER1305
M1.SW	M1 (W)	M1	Smin	2.0		ER1402
V1.W1	V1		Wfix	1.4		ER0022
V1.S1	V1	V1	Smin	1.5		ER0046
V1.M1	V1	M1	Emin	1.0		ER0052

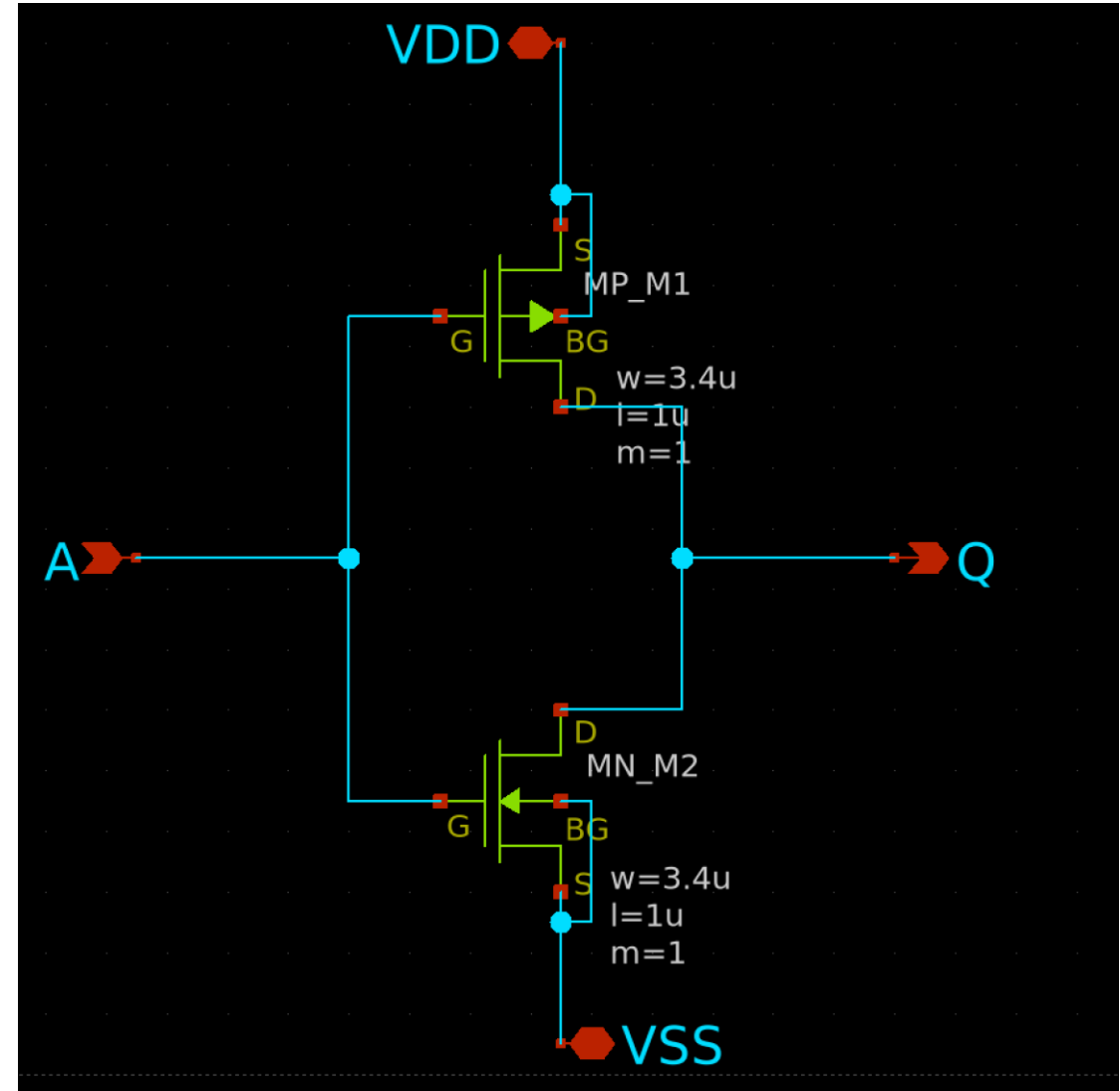
Klayout : DRC OK

何も表示されなければOK



Klayout : LVS用ネットリストの出力方法 (1/2)

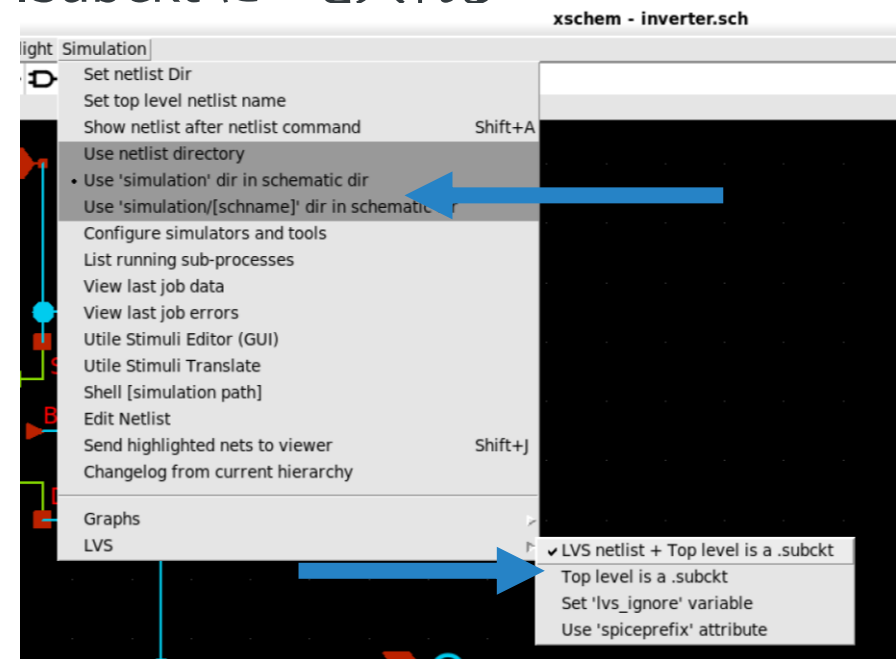
- xschem上であらかじめLVS用の回路図を作成しておく。
- ピンはipin, iopin, opin のみを使用する。
 - vdd.symやgnd.symを使用するとうまく通らない。
- ファイルを同じ名前にする 例: nand.sch, nand.gds



Klayout : LVS用ネットリストの出力方法 (2/2)

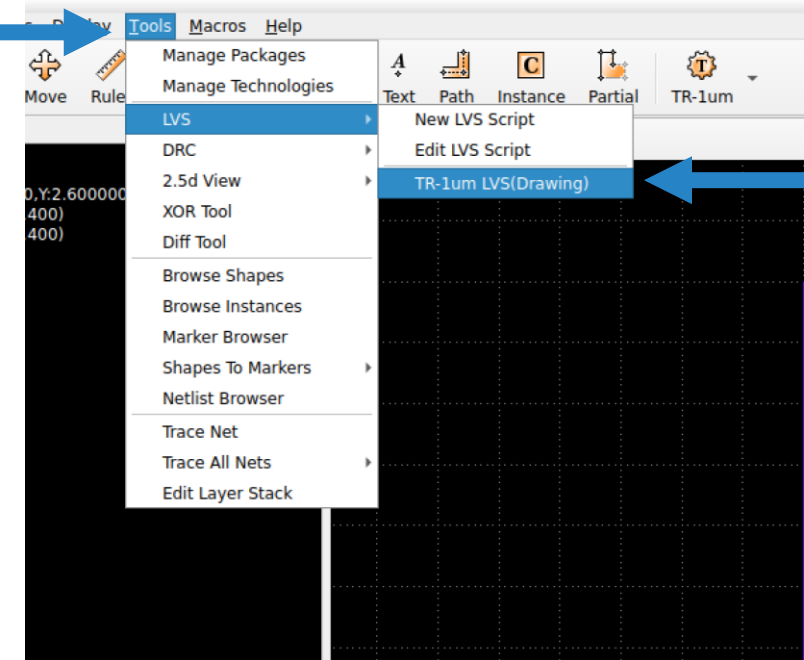
- xschem上で**LVS用**のネットリストを出力する。
 - メニュー Simulation > Use 'simulation' dir in schematic dir に✓を入れる
 - メニュー Simulation > LVS > Use 'spiceprefix' attribute の✓を外す
 - メニュー Simulation > LVS > LVS netlist + Top level is a .subckt に✓を入れる
 - Netlistを押す

- 終わったら閉じて良い
- デフォルトの生成場所は [xschem実行Dir]/simulation/



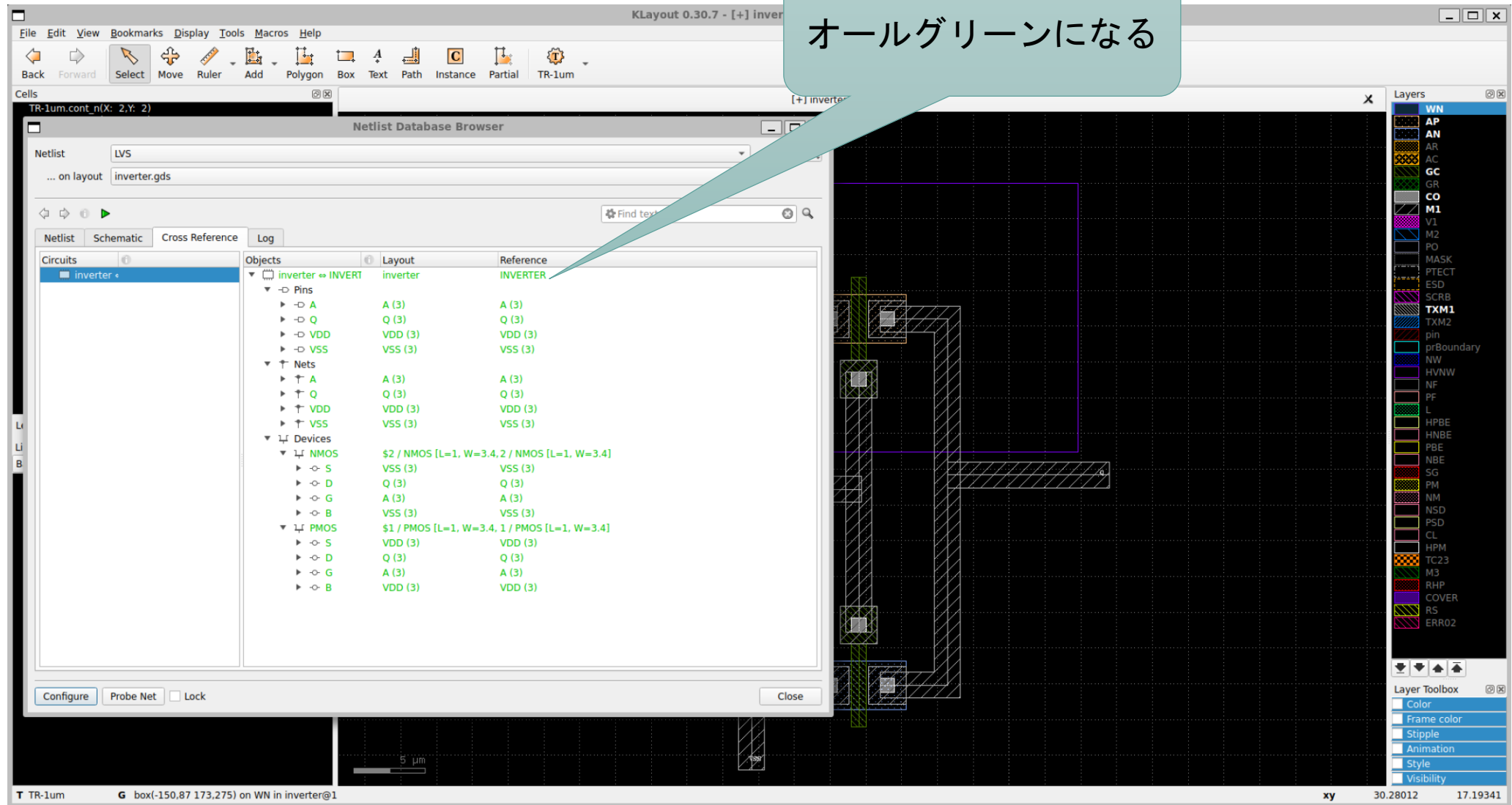
Klayout : LVSの実行

- klayout上でネットリスト比較を実行する。
1. メニュー Tools > LVS > TR-1um LVS(Drawing) を実行する



The screenshot shows the KLayout 0.30.7 interface. The main window displays a grid layout. Overlaid on this is the 'Netlist Database Browser' window, which shows a netlist for 'inverter.gds'. The netlist contains a table of objects with columns for 'Objects', 'Layout', and 'Reference'. Two entries in the 'Objects' column are circled in red: '2 / NMOS [L=1, W=3.6, (AS=10.08, AD=10.08, PS=2 / MOS [L=1, W=3.4]'. A green callout bubble points to the first entry with the text 'エラーマーク' (Error mark). A blue callout bubble points to the second entry with the text 'Lのサイズが違う' (L size is different). The background shows a grid layout with various layers and a layer toolbox on the right.

Klayout : LVS OK

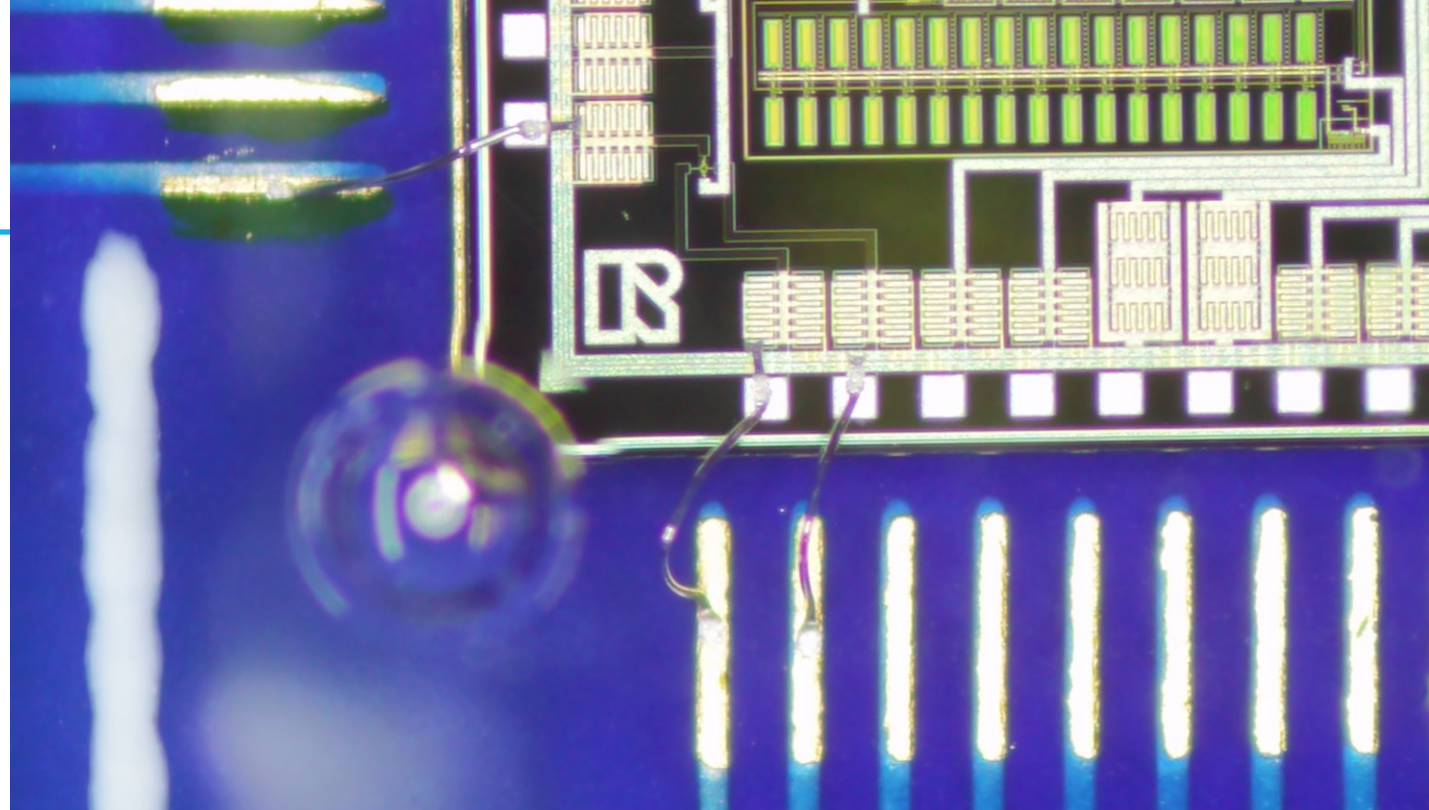
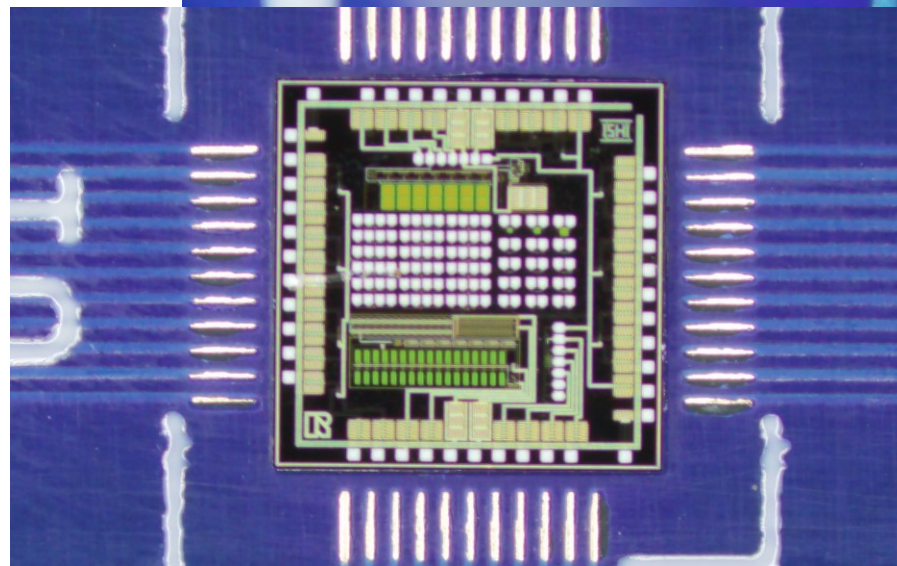
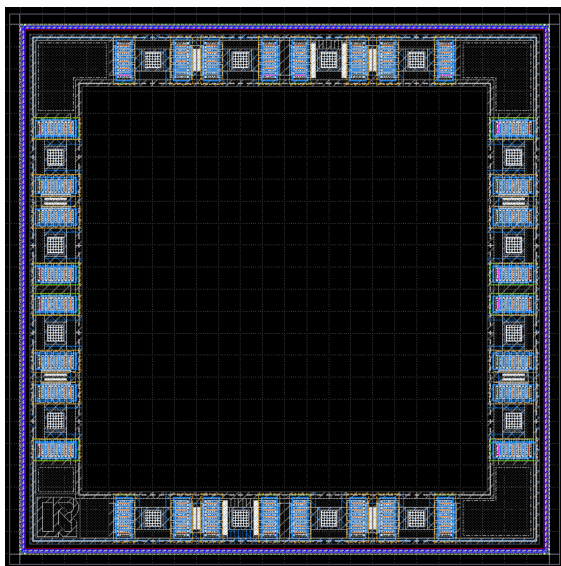


アナログ半導体の設計フロー

1. 回路図を描く
2. シミュレーションをする
3. 回路図を基にレイアウトを描く
4. レイアウトを検証する
5. レイアウトを基に寄生成分を考慮したシミュレーションをする
- 6. フレームに載せる**

フレームとパッド

- パッド
 - フレーム上に配置される配線用の電極
 - ボンディングでパッケージ基板と接続する
- フレーム
 - カットされる単位

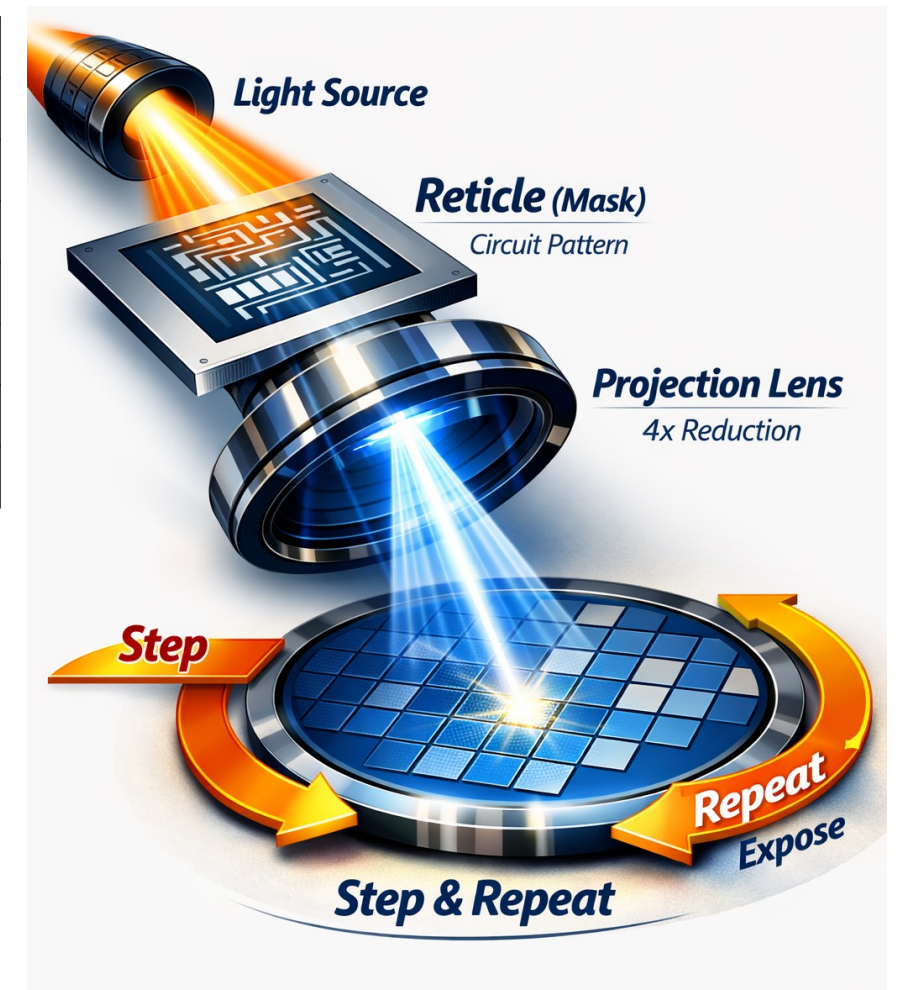


ウェハーとレチクル（マスク）とフレームの関係

- ウェハー
 - 一番大きな単位
 - シリコンウェハーそのもの
- レチクル（マスク）
 - ウェハー上に転写する単位
 - この転写数が1ウェハーから取れるチップ数となる
 - 1チップとしてはこれ以上大きくできない最大単位となる
- フレーム
 - 1チップとなるカットされる単位
 - 今回は、1レチクルに8x8の64フレームが入っている
 - 複数の形状を1レチクル内に入れることも可能

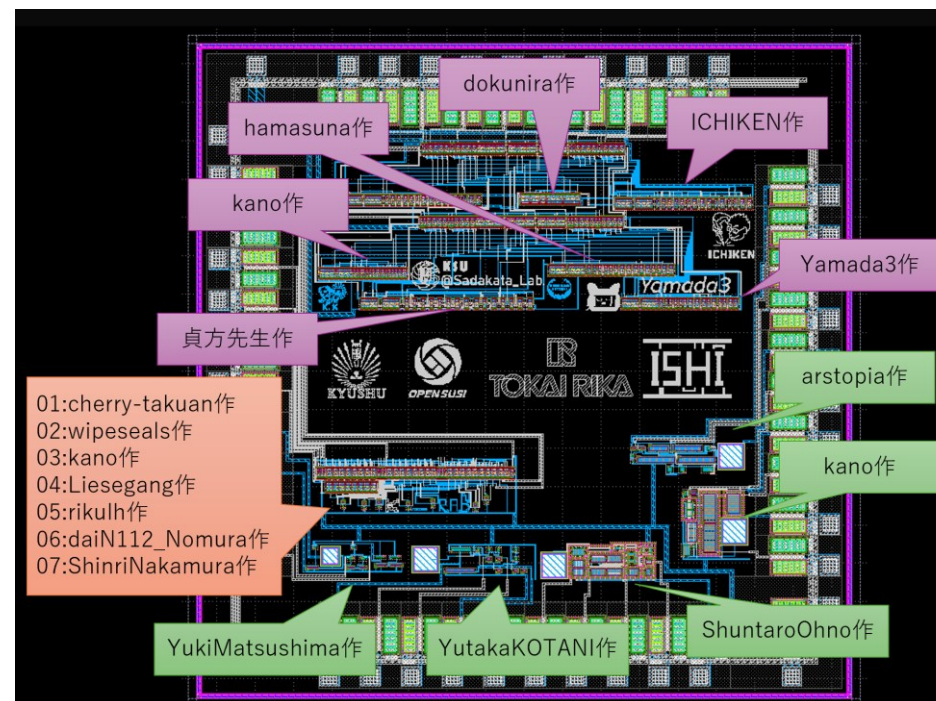
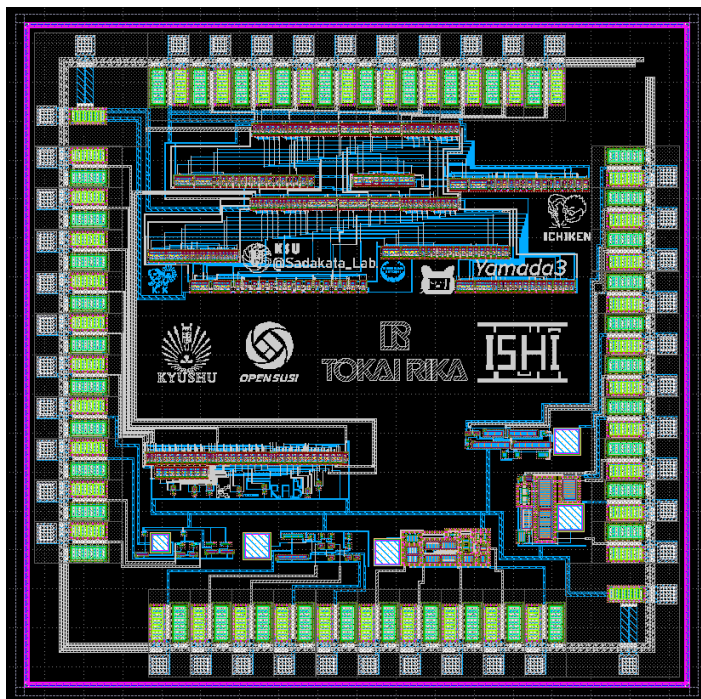
OpenSUSI MPW Users Layout

	0	1	2	3	4	5	6	7
0	TEG %_lum_000_system_img Run: - Title: (0,0)	JunTokamura %_lum_JunTokamura_0-14 Run: 2385819825 Title: (1,0)	JunTokamura %_lum_JunTokamura_0-14 Run: 2385819825 Title: (2,0)	EMPTY %_lum_000_system Run: - Title: (3,0)	EMPTY %_lum_000_system Run: - Title: (4,0)	EMPTY %_lum_000_system Run: - Title: (5,0)	EMPTY %_lum_000_system Run: - Title: (6,0)	EMPTY %_lum_000_system Run: - Title: (7,0)
1	EMPTY %_lum_000_system Run: - Title: (0,1)	EMPTY %_lum_000_system Run: - Title: (1,1)	EMPTY %_lum_000_system Run: - Title: (2,1)	EMPTY %_lum_000_system Run: - Title: (3,1)	EMPTY %_lum_000_system Run: - Title: (4,1)	EMPTY %_lum_000_system Run: - Title: (5,1)	EMPTY %_lum_000_system Run: - Title: (6,1)	EMPTY %_lum_000_system Run: - Title: (7,1)
2	EMPTY %_lum_000_system Run: - Title: (0,2)	EMPTY %_lum_000_system Run: - Title: (1,2)	EMPTY %_lum_000_system Run: - Title: (2,2)	EMPTY %_lum_000_system Run: - Title: (3,2)	EMPTY %_lum_000_system Run: - Title: (4,2)	EMPTY %_lum_000_system Run: - Title: (5,2)	EMPTY %_lum_000_system Run: - Title: (6,2)	EMPTY %_lum_000_system Run: - Title: (7,2)
3	EMPTY %_lum_000_system Run: - Title: (0,3)	EMPTY %_lum_000_system Run: - Title: (1,3)	EMPTY %_lum_000_system Run: - Title: (2,3)	EMPTY %_lum_000_system Run: - Title: (3,3)	EMPTY %_lum_000_system Run: - Title: (4,3)	EMPTY %_lum_000_system Run: - Title: (5,3)	EMPTY %_lum_000_system Run: - Title: (6,3)	EMPTY %_lum_000_system Run: - Title: (7,3)
4	EMPTY %_lum_000_system Run: - Title: (0,4)	EMPTY %_lum_000_system Run: - Title: (1,4)	EMPTY %_lum_000_system Run: - Title: (2,4)	EMPTY %_lum_000_system Run: - Title: (3,4)	EMPTY %_lum_000_system Run: - Title: (4,4)	EMPTY %_lum_000_system Run: - Title: (5,4)	EMPTY %_lum_000_system Run: - Title: (6,4)	EMPTY %_lum_000_system Run: - Title: (7,4)
5	EMPTY %_lum_000_system Run: - Title: (0,5)	EMPTY %_lum_000_system Run: - Title: (1,5)	EMPTY %_lum_000_system Run: - Title: (2,5)	EMPTY %_lum_000_system Run: - Title: (3,5)	EMPTY %_lum_000_system Run: - Title: (4,5)	EMPTY %_lum_000_system Run: - Title: (5,5)	EMPTY %_lum_000_system Run: - Title: (6,5)	EMPTY %_lum_000_system Run: - Title: (7,5)
6	EMPTY %_lum_000_system Run: - Title: (0,6)	EMPTY %_lum_000_system Run: - Title: (1,6)	EMPTY %_lum_000_system Run: - Title: (2,6)	EMPTY %_lum_000_system Run: - Title: (3,6)	EMPTY %_lum_000_system Run: - Title: (4,6)	EMPTY %_lum_000_system Run: - Title: (5,6)	EMPTY %_lum_000_system Run: - Title: (6,6)	EMPTY %_lum_000_system Run: - Title: (7,6)
7	EMPTY %_lum_000_system Run: - Title: (0,7)	EMPTY %_lum_000_system Run: - Title: (1,7)	EMPTY %_lum_000_system Run: - Title: (2,7)	EMPTY %_lum_000_system Run: - Title: (3,7)	EMPTY %_lum_000_system Run: - Title: (4,7)	EMPTY %_lum_000_system Run: - Title: (5,7)	EMPTY %_lum_000_system Run: - Title: (6,7)	EMPTY %_lum_000_system Run: - Title: (7,7)



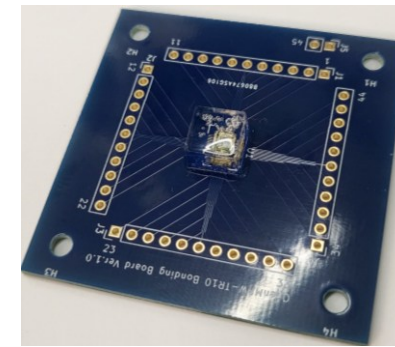
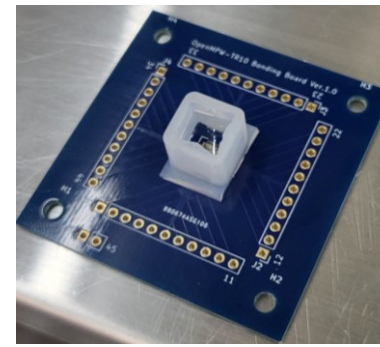
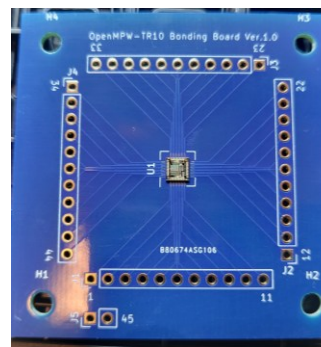
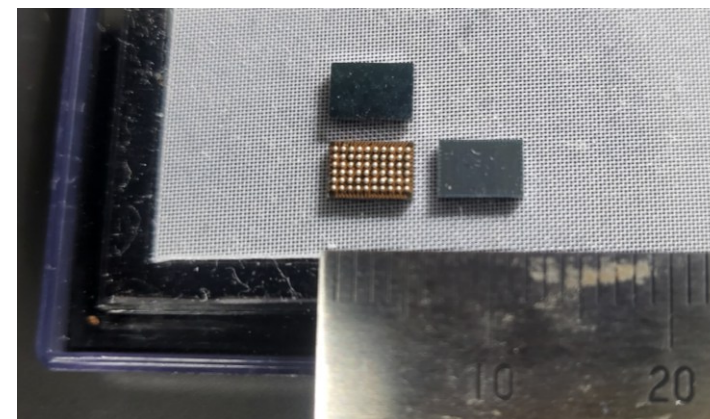
MPW : 今回の形態

- MPW (Multi Person/Project Wafer) という概念もある
 - 1つのフレームを複数人やプロジェクトでシェアする形態
- メリット : さらに安く作れる
- デメリット : パッド (ピン) やエリア (面積) 、チップをシェアすることになるため、取り合いになることがある



パッケージ

- パッケージの方法
 - チップ+基板+ワイヤリング+封入で構成される
- パッケージの分類
 - QFN, BGAなど
 - 一般的な方式。側面や背面などにピン（足）を出す方式
 - Chiplet, SiP(System in Package)など
 - 複数のチップを1つのパッケージにまとめる方式。封入法がQFN, BGAなどと同じであることが多い。
 - WLP (Wafer Level Package)
 - ウェハーのまま配線し、封入する方式
 - CoB(Chip on Board)
 - PCB基板に直接パッケージする方式
- 基板：PCB基板やセラミック基板などを利用
- ワイヤリング：ワイヤボンディングやフリップチップを利用
- 封入：樹脂やセラミックを利用

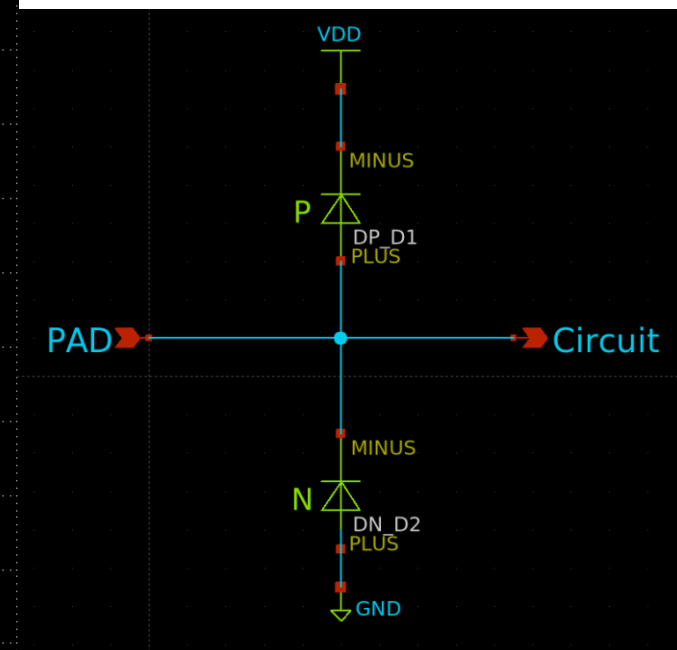
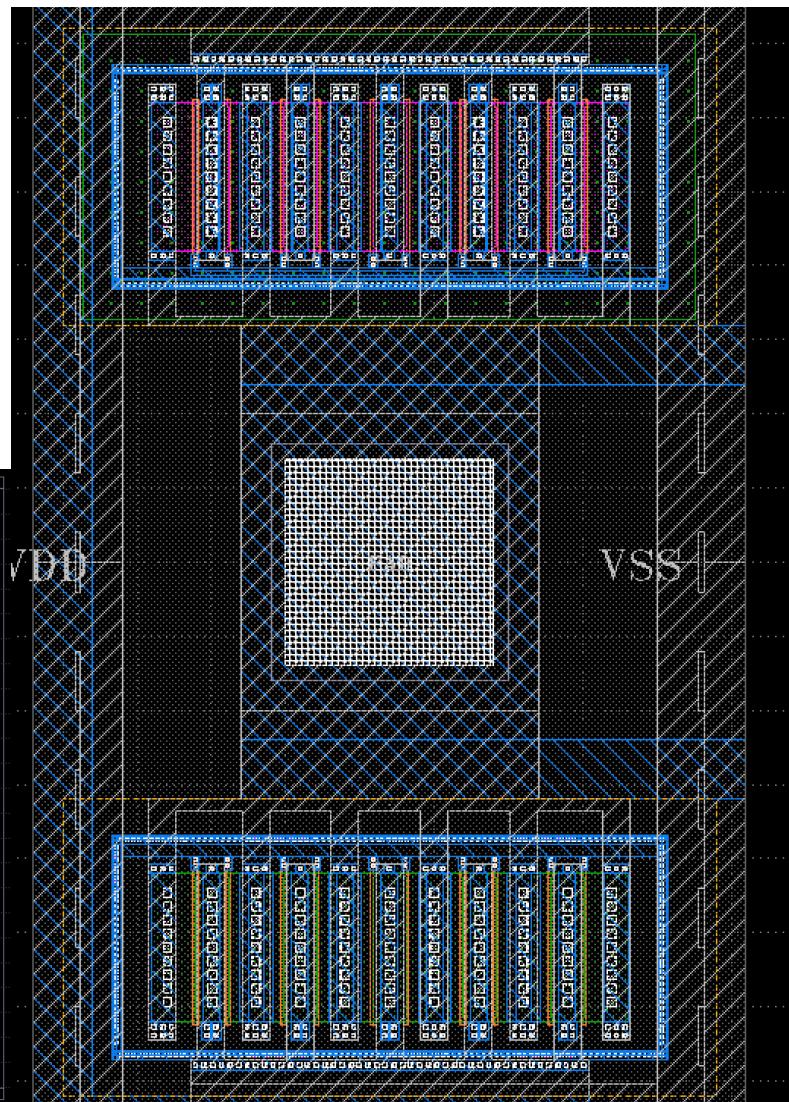
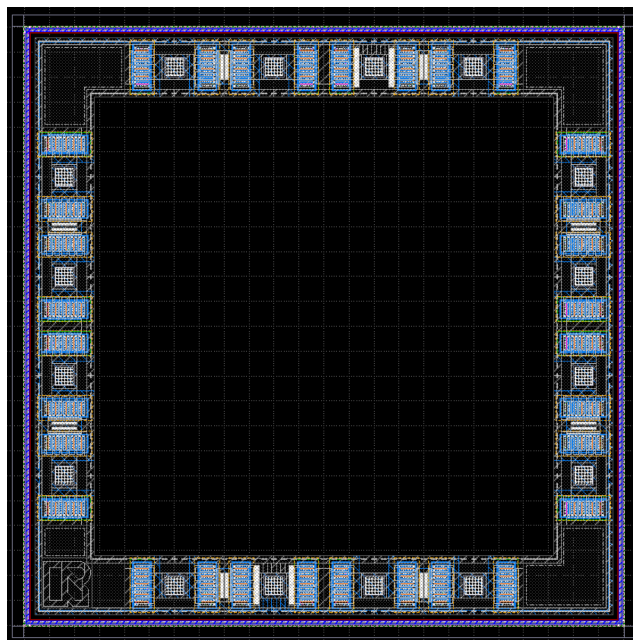


ファブの関係性：PCB基板との対比

- EDAツール（設計ツール）
 - 回路図とレイアウト
 - KiCAD -> XScem & Klayout
 - シミュレーター
 - LTSpice -> ngspice
- 製造会社：代理店（ファブからウェハーを購入する）
 - ユーザサポートを行う
 - レチクルの中身やフレームを提供する
 - P版.com -> OpenSUSI
- 製造会社：ファブ（前工程工場）
 - ウェハーやレチクルのサイズなどの製造仕様の提供する
 - テープアウト日を指定する（シャトルの場合）
 - PCB基板と違い、年に数回しか製造チャンスがない
 - 実際の製造を行う
 - PCB基板製造工場 -> 東海理化
- 製造会社：アセンブリ（後工程工場）
 - パッケージングを行う
 - PCB基板の部品実装工場 -> ODT

パッドとESD

- パッドからゲートにつなげる場合は要注意
 - ゲート酸化膜の厚さ = 19.5nm
 - 酸化膜の耐圧 : 6~10MV/cm
 - 19.5Vで破壊される

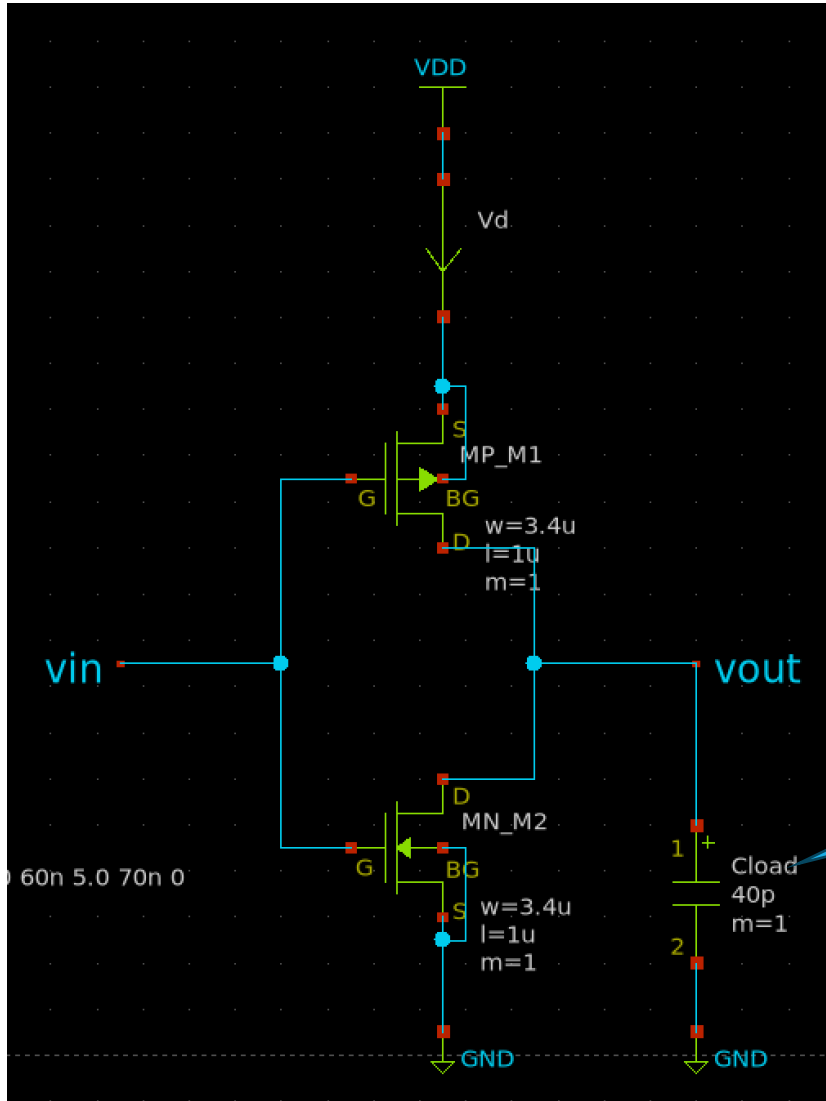


ドライブ能力

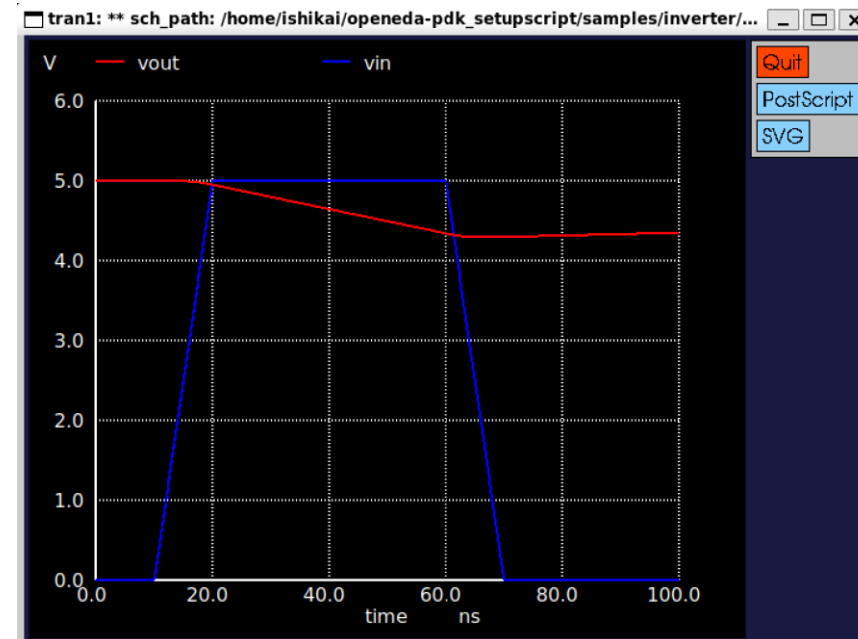
- **出力先がピンの外の場合は**IC内のロジックセル等の容量負荷よりも**膨大な容量負荷がかかる**
 - どれぐらいの負荷容量がかかるかはICの利用者に依存
 - 要因：ICパッケージ、（ソケット）、基板配線パターン、ケーブル など
- ターゲットの出力周波数を、想定した負荷容量で駆動できるサイズで設計する必要がある
 - 今回は外界の負荷容量を 40 pF に設定してシミュレーションを行う

参考：一般的なIC + ソケット + 30cm ケーブル + パッシブプローブの容量を測った時は28pFでした

ngspice : 過渡解析ベンチマークのCload=40pFにした例



- PMOSとNMOSの性能を上げる
- PMOSとNMOSのWを大きくする
- PMOSとNMOSのLを小さくする
- W/Lの比が同じ場合、PMOSとNMOSの性能バランスは保たれる



自分でフレームに載せる場合

1. フレームをKlayoutで読み込む

1. ~/OpenEDA-PDK_SetupScript/GDS/OpenSUSI-TR10/top_frame.gds

2. インバータ回路のGDSファイルをインポートする

1. File -> Import -> Other Files Into Current

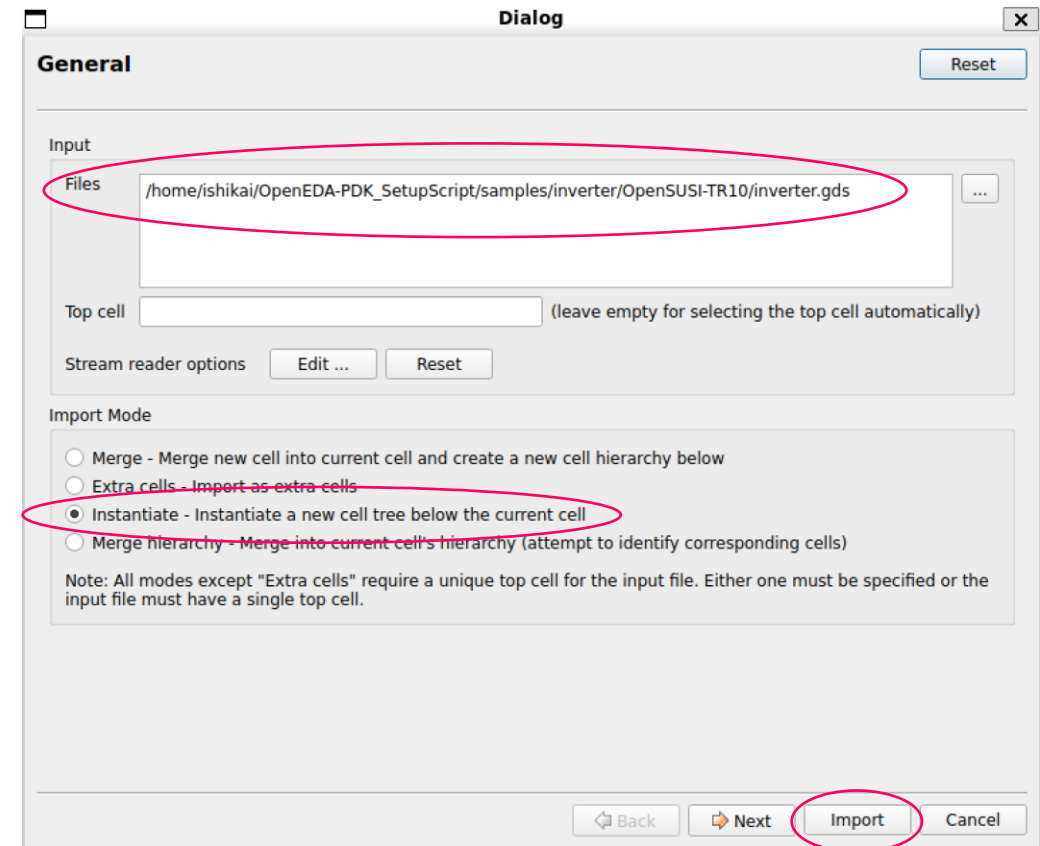
2. 右図に合わせて自作のインバータ回路のGDSファイルを読み込む

3. 指定の場所にインバータ回路のレイアウトを配置する

1. こちらで指定します

4. 指定のパッドにVDD,VSS,OUTピンを接続する

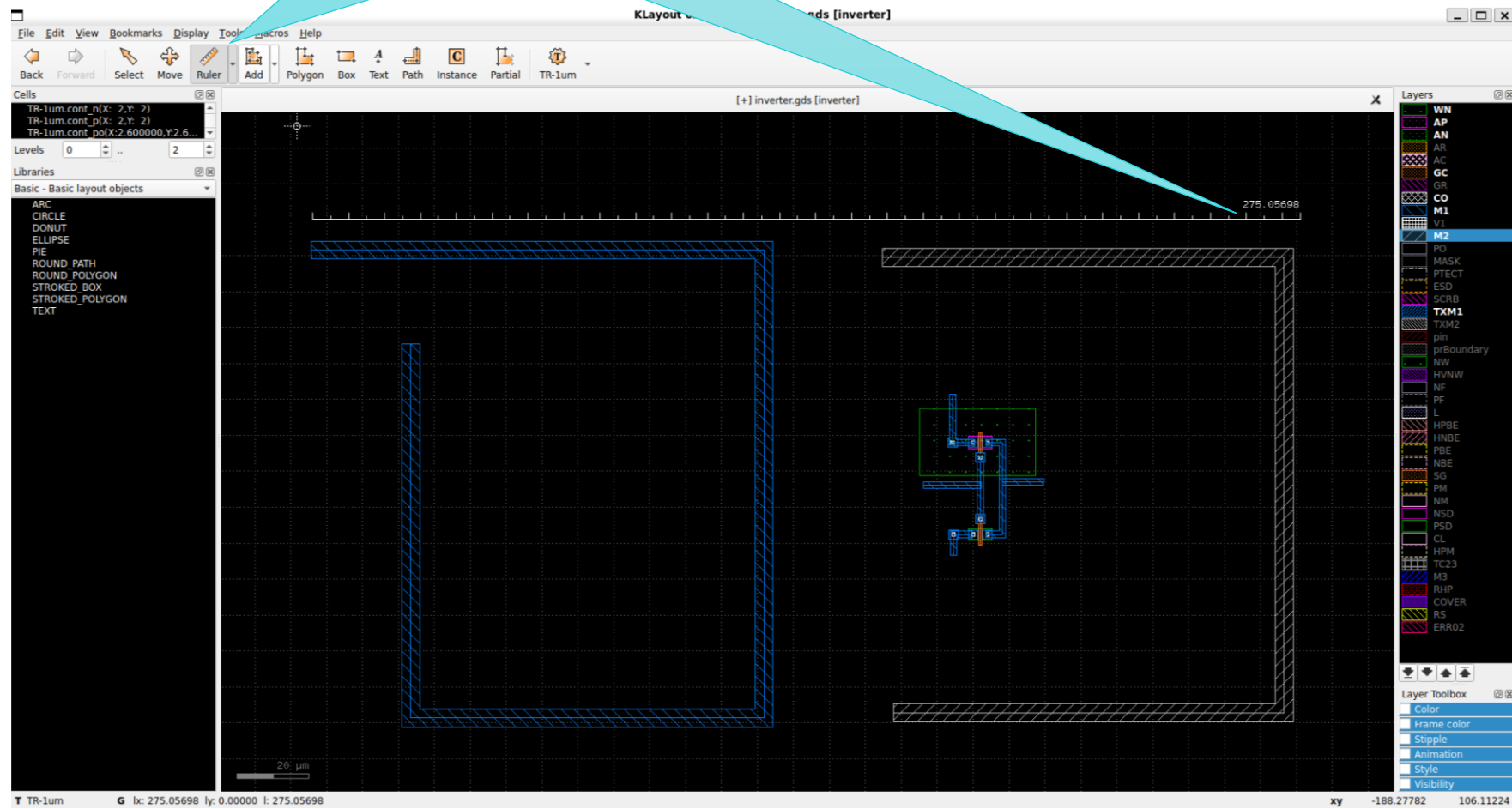
1. こちらで指定します



シリコンアート

Rulerで長さチェック可能

- サイズ制限
 - 200um x 200um以内
- 利用可能レイヤー
 - M1
 - M2
- 備考
 - DRCチェックをしてください
 - 状況により削除する可能性があります



提出

- 本日
 - 提出物：回路図、レイアウト、githubのアカウント、連絡先（チップ送付先）
 - 1, 自分のGithub上にリポジトリを作ってもらって、schファイルとgdsファイルをアップロードする
 - 2, READMEに感想などを記述する
 - 3, DiscordでgithubのURLを告知する
 - 4, ConnpassのIDもお願いします。
- 数日中
 - 提出物：公開用の一言など
- 半年後
 - イベント：チップが届きますので、お渡し会を兼ねたチップ測定会を開催します

1bit-CPU設計 (TR10)

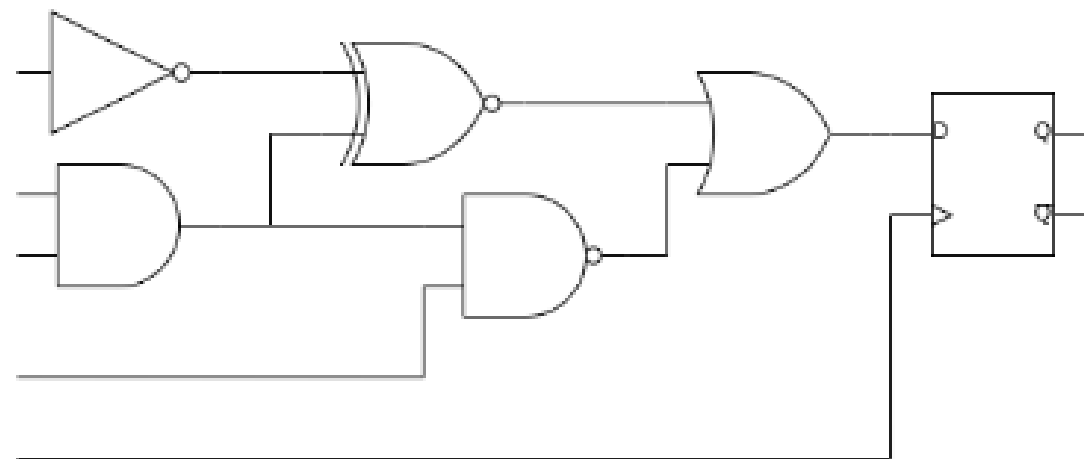
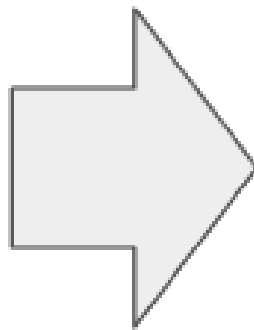
ISHI会

<https://ishi-kai.org/>

Mail: info@ishi-kai.org



HDL



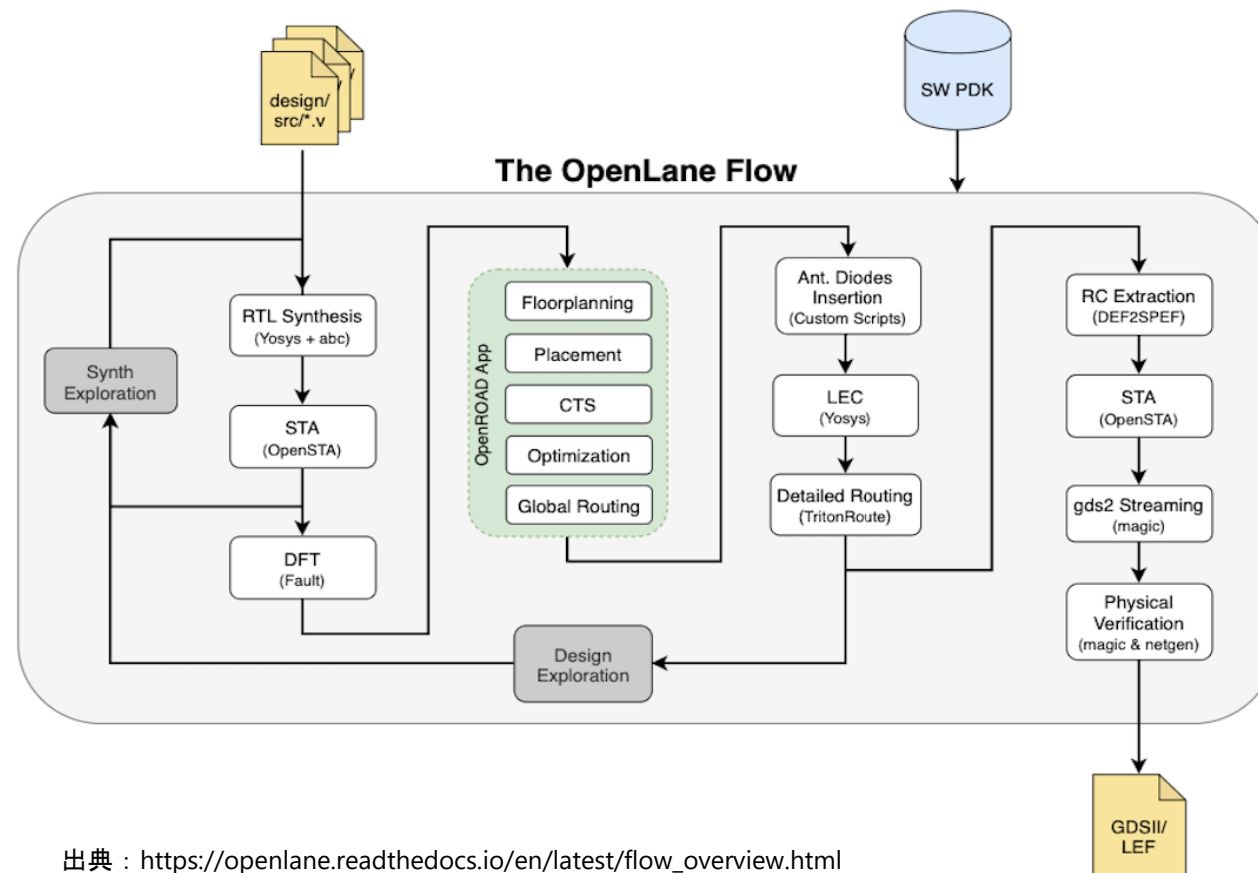
デジタル回路

論理合成のイメージ

論理合成

HDL（ハードウェア記述言語）で
記述して、論理回路（RTL）にする

OpenLANEによる生成フロー

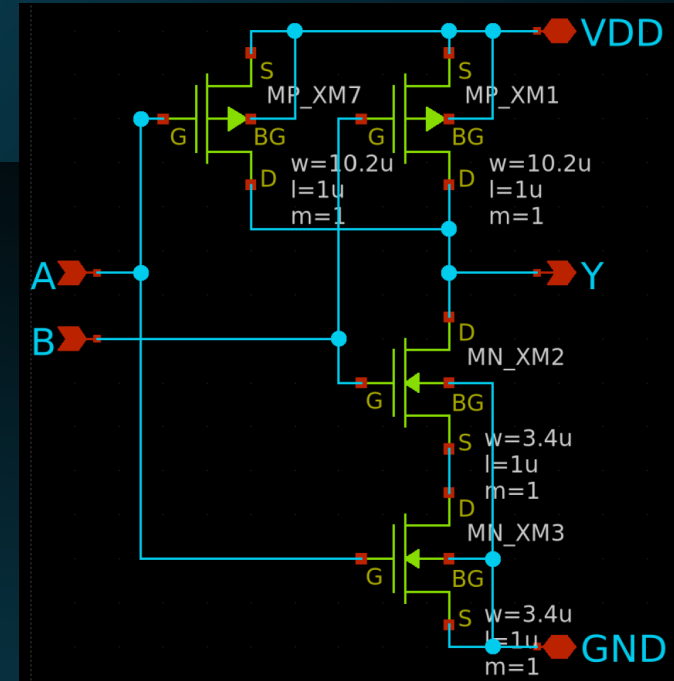
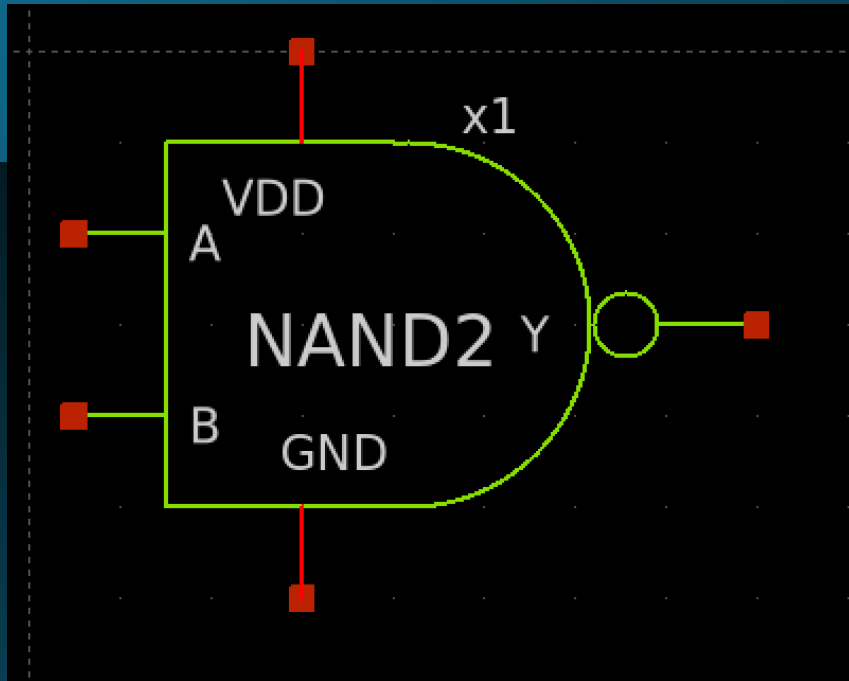


出典 : https://openlane.readthedocs.io/en/latest/flow_overview.html

生成フロー詳細

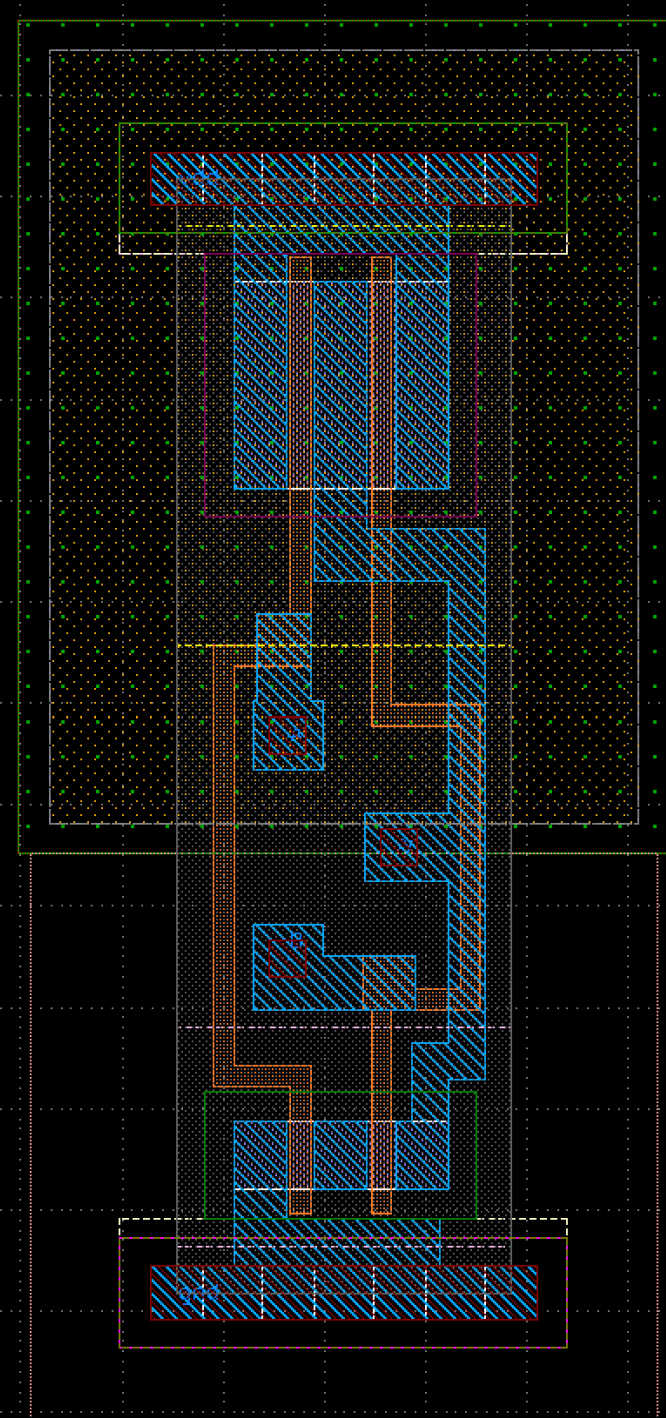
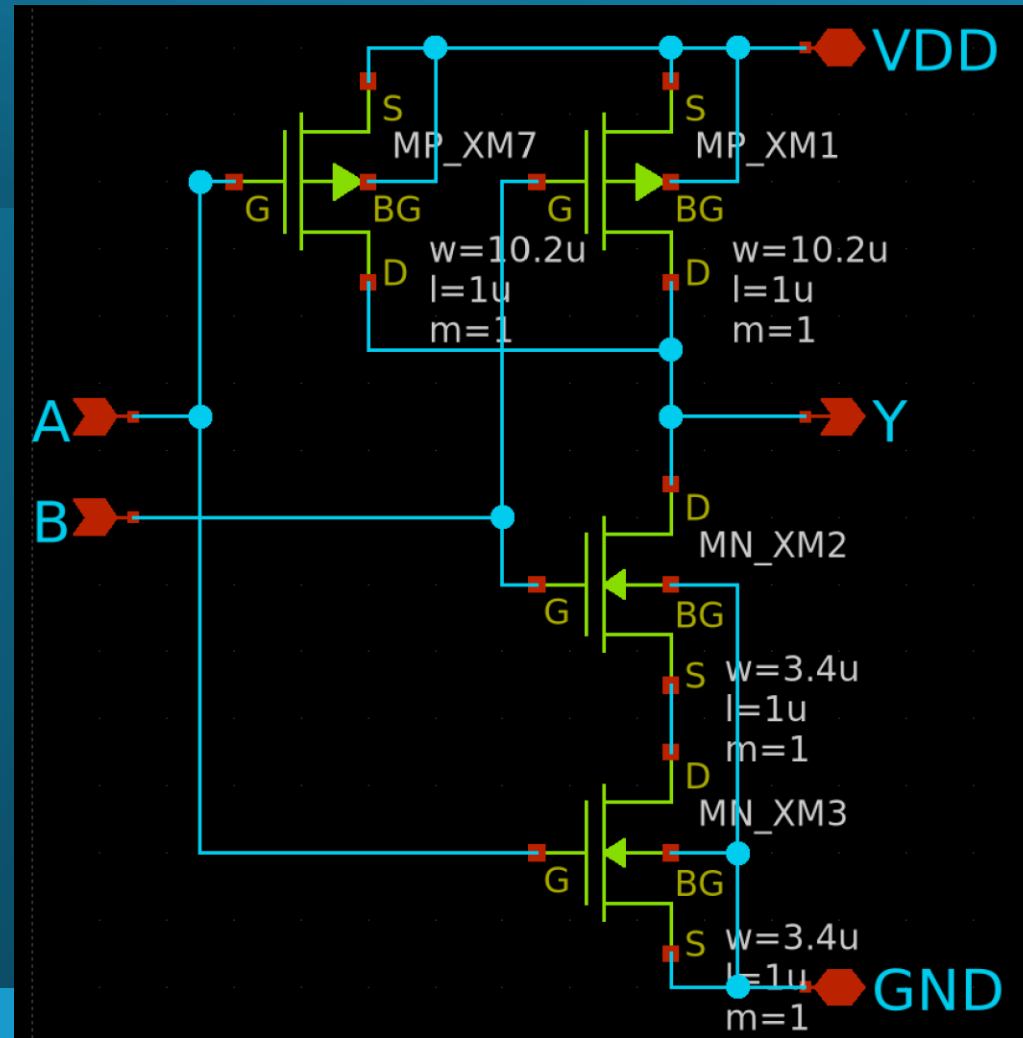
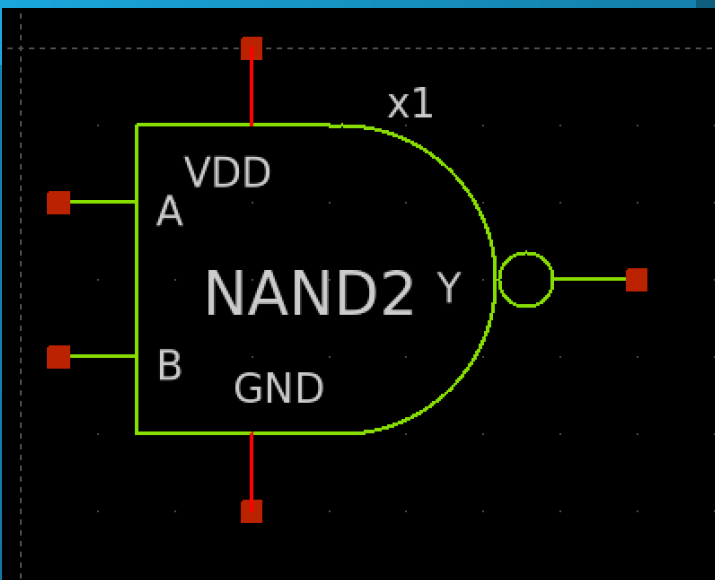
合成		フロアプランと電源供給		配置		クロックツリー合成と配線		GDS生成とチェック	
ツール名	機能内容	ツール名	機能内容	ツール名	機能内容	ツール名	機能内容	ツール名	機能内容
yosys	RTLを論理合成	init_fp	コア領域の定義	RePLace	グローバル配置	TritonCTS	クロックツリーの合成	Magic	GDSIIファイル生成
abc	PDKマッピング	ioplacer	入出力ポートの設置	Resizer	最適化	FastRouteとCU-GR	グローバル配線	Magic	DRC (Design Rule Check) とアンテナチェック
OpenSTA	静的タイミング解析	pdn	給電ネットワークの生成	OpenDP	ローカル配置	TritonRoute	ローカル配線	Netgen	LVS (Layout vs Schematic) チェック
		tapcell	タップとデキャップセルの挿入			SPEF_Extractor	寄生フォーマットの抽出	CVC	回路妥当性検証

論理回路の正体



中身はPMOSとNMOS

スタンダードセル



32.1

32.1

32.1

横の長さが何種類かある

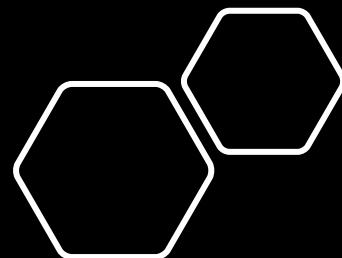
LIB..BUF_X1.

22 | :

•	•
•	•
•	•

Library: IP62_5_stdcell

No	Cell	種類	入力ピン数	出力ピン数	セル高さ[um]	セル幅[um]	MP W[um]	MP L[um]	MN W[um]	MN L[um]
1	AND2_X1	ANDゲート	2	1	55.0	22.0	10.2	1	3.4	1
2	AND3_X1	ANDゲート	3	1	55.0	27.5	10.2	1	3.4	1
3	AND4_X1	ANDゲート	4	1	55.0	33.0	10.2	1	3.4	1
4	BUF_X1	バッファ	1	1	55.0	16.5	10.2	1	3.4	1
5	BUF_X2	バッファ	1	1	55.0	22.0	10.2 x 2	1	3.4 x 2	1
6	BUF_X4	バッファ	1	1	55.0	33.0	10.2 x 4	1	3.4 x 4	1
7	BUF_X8	バッファ	1	1	55.0	49.5	10.2 x 8	1	3.4 x 8	1
8	BUF_X12	バッファ	1	1	55.0	66.0	10.2 x 12	1	3.4 x 12	1
9	BUF_X16	バッファ	1	1	55.0	93.5	10.2 x 16	1	3.4 x 16	1
10	CLKBUF_X1	クロックバッファ	1	1	55.0	38.5	10.2 x 5	1	3.4 x 5	1
11	CLKBUF_X2	クロックバッファ	1	1	55.0	66.0	10.2 x 10	1	3.4 x 10	1
12	CLKBUF_X4	クロックバッファ	1	1	55.0	115.5	10.2 x 20	1	3.4 x 20	1
13	CLKBUF_X8	クロックバッファ	1	1	55.0	203.5	10.2 x 40	1	3.4 x 40	1
14	CLKBUF_X12	クロックバッファ	1	1	55.0	291.5	10.2 x 60	1	3.4 x 60	1
15	CLKBUF_X16	クロックバッファ	1	1	55.0	379.5	10.2 x 80	1	3.4 x 80	1
16	DEL1	ディレイセル	1	1	55.0	38.5	10.2 x 2	1	3.4 x 2	1
17	DEL2	ディレイセル	1	1	55.0	60.5	10.2	1	3.4	1
18	DEL4	ディレイセル	1	1	55.0	104.5	10.2	1	3.4	1
19	DFFR	Dフリップフロップ	3	2	55.0	88.0	10.2	1	3.4	1
20	DFFS	Dフリップフロップ	3	2	55.0	88.0	10.2	1	3.4	1
21	INV_X1	インバータ	1	1	55.0	16.5	10.2	1	3.4	1
22	INV_X2	インバータ	1	1	55.0	16.5	10.2 x 2	1	3.4 x 2	1
23	INV_X4	インバータ	1	1	55.0	27.5	10.2 x 4	1	3.4 x 4	1
24	INV_X8	インバータ	1	1	55.0	44.0	10.2 x 8	1	3.4 x 8	1
25	INV_X12	インバータ	1	1	55.0	60.5	10.2 x 12	1	3.4 x 12	1
26	INV_X16	インバータ	1	1	55.0	77.0	10.2 x 16	1	3.4 x 16	1
27	MUX2	マルチプレクサ	3	1	55.0	49.5	10.2	1	3.4	1
28	NAND2	NANDゲート	2	1	55.0	16.5	10.2	1	3.4	1
29	NAND3	NANDゲート	3	1	55.0	22.0	10.2	1	3.4	1
30	NAND4	NANDゲート	4	1	55.0	27.5	10.2	1	3.4	1
31	NOR2	NORゲート	2	1	55.0	16.5	10.2	1	3.4	1
32	NOR3	NORゲート	3	1	55.0	22.0	10.2	1	3.4	1
33	NOR4	NORゲート	4	1	55.0	27.5	10.2	1	3.4	1
34	OR2	ORゲート	2	1	55.0	22.0	10.2	1	3.4	1
35	OR3	ORゲート	3	1	55.0	27.5	10.2	1	3.4	1
36	OR4	ORゲート	4	1	55.0	33.0	10.2	1	3.4	1
37	XNOR2	XNORゲート	2	1	55.0	33.0	10.2	1	3.4	1
38	XOR2	XORゲート	2	1	55.0	33.0	10.2	1	3.4	1



VDDライン

PMOS
Sが共通

ピン

NMOS
Sが共通

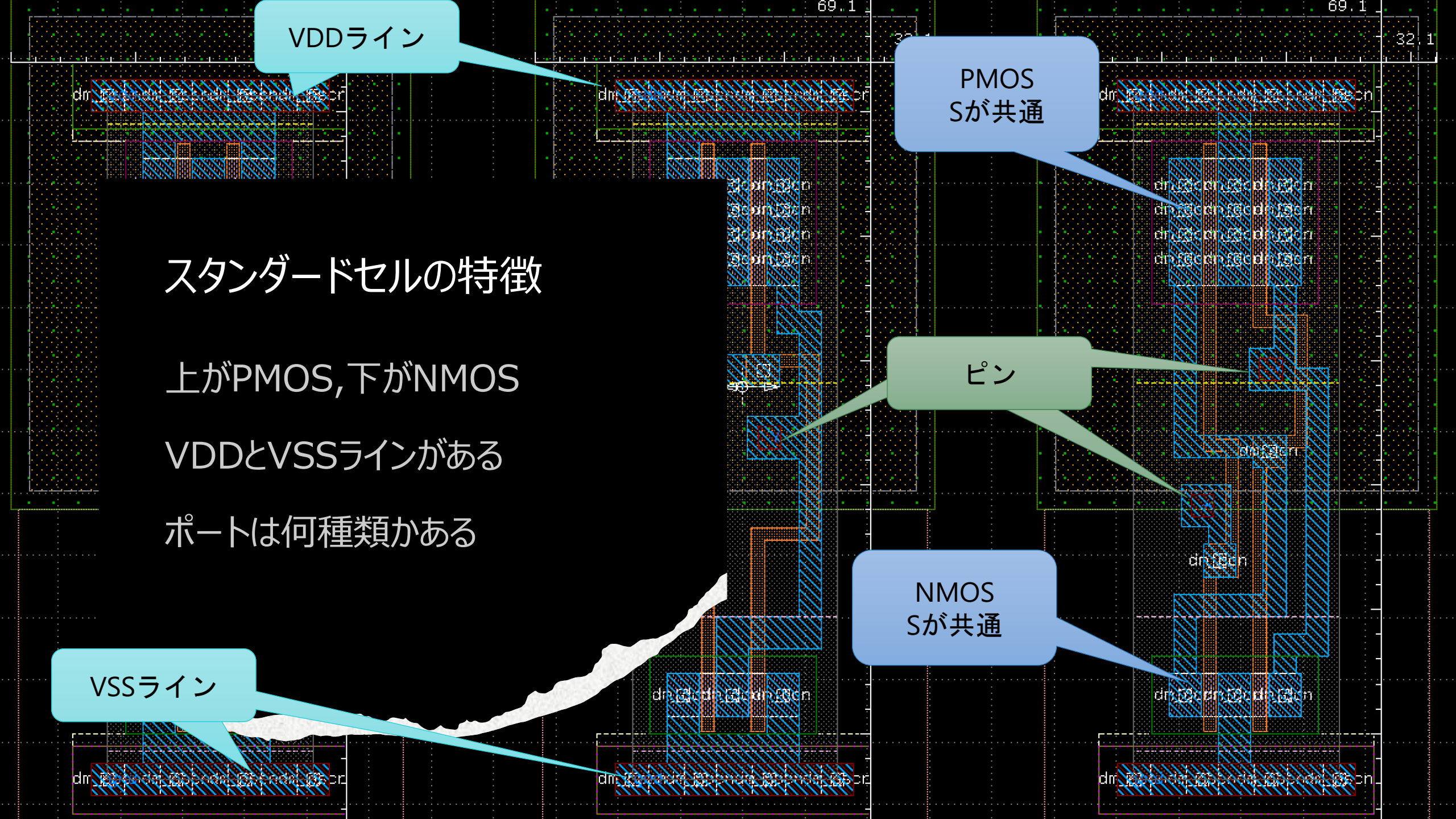
VSSライン

スタンダードセルの特徴

上がPMOS,下がNMOS

VDDとVSSラインがある

ポートは何種類もある



回路をレイアウトにする工程

